Cadence RTL-to-GDSII Flow v6.0

VideoTitle :Design Specification and RTL Coding

Time(HH:MM:SS) : 00:00:01

Module two design specification and RTL coding.

Time(HH:MM:SS) : 00:00:07

In this module, you will define the meaning of the design specification using an example of a simple counter design spec.

Time(HH:MM:SS) : 00:00:15

You will also implement the specified design through the RTL coding process.

Time(HH:MM:SS) : 00:00:21

You would identify what hardware description languages or HDL are.

Time(HH:MM:SS) : 00:00:26

You will implement a design spec for a given design criteria, and you will also code a very simple counter design given a design spec.

VideoTitle :Module Objectives

Time(HH:MM:SS) : 00:00:01

This is a flow chart of the cadence EDA flow RTL to GDS two.

Time(HH:MM:SS) : 00:00:06

The complete flow with all the phases that it goes through.

Time(HH:MM:SS) : 00:00:08

And next to each of the phase there are a snapshot of tools given here.

Time(HH:MM:SS) : 00:00:13

We start with the design specification.

Time(HH:MM:SS) : 00:00:15

Rtl coding.

Time(HH:MM:SS) : 00:00:16

Functional simulation.

Time(HH:MM:SS) : 00:00:18

Coverage analysis.

Time(HH:MM:SS) : 00:00:19

This is all the front end tasks.

Time(HH:MM:SS) : 00:00:21

Then comes the back end the logic synthesis design for test synthesis atpg vector generation.

Time(HH:MM:SS) : 00:00:29

Then the floor planning, the implementation, the power planning, the placement clock tree synthesis, routing.

Time(HH:MM:SS) : 00:00:36

Then we do have the timing analysis.

Time(HH:MM:SS) : 00:00:38

The static timing analysis with the output given from the static timing analysis.

Time(HH:MM:SS) : 00:00:44

We run the simulation again at the gate level.

Time(HH:MM:SS) : 00:00:47

This is called the gate level simulation.

Time(HH:MM:SS) : 00:00:49

Then we have the output from it a set of gates or the netlist.

Time(HH:MM:SS) : 00:00:53

And this is the GDS two.

Time(HH:MM:SS) : 00:00:55

In this module we are dealing with the design specification phase of this flowchart.

VideoTitle :Design Specification FlowChart

Time(HH:MM:SS) : 00:00:01

What is a design specification or a design spec

Time(HH:MM:SS) : 00:00:05

Design specification is an explicit set of requirements to be satisfied by a material, a product, or a service.

Time(HH:MM:SS) : 00:00:12

An idea begins with a specification always, which can be textual, graphical, or sometimes a software representation.

Time(HH:MM:SS) : 00:00:20

It is a detailed document providing information about the characteristics of a project.

Time(HH:MM:SS) : 00:00:24

To set the criteria the design engineers will need to meet to code their designs.

Time(HH:MM:SS) : 00:00:29

This is an example snapshot here on the right side.

Time(HH:MM:SS) : 00:00:32

It's an apb Uart project specification.

Time(HH:MM:SS) : 00:00:36

As you can see, there is an introduction that explains the APB, Uart design and all the different criteria it has to contain.

Time(HH:MM:SS) : 00:00:43

And there is a diagram here that shows all the input and the output signals and the different blocks that go into this complex design.

Time(HH:MM:SS) : 00:00:50

And this spec will continue giving more and more details about this APB Uart design.

Time(HH:MM:SS) : 00:00:56

So this is how a design spec looks.

Time(HH:MM:SS) : 00:01:01

This slide shows an example of the design criteria of a two bit synchronous binary counter.

Time(HH:MM:SS) : 00:01:07

Based on all these points given here, these are all called design criteria.

Time(HH:MM:SS) : 00:01:12

A design specification can be formed as a document using this design criteria.

Time(HH:MM:SS) : 00:01:18

We also come up with a circuit diagram and the truth table as shown in the slide.

Time(HH:MM:SS) : 00:01:23

Let's go over what are the design criteria for this binary counter?

Time(HH:MM:SS) : 00:01:27

The two bit synchronous binary counter.

Time(HH:MM:SS) : 00:01:30

The binary counter is an example of a simple synchronous digital circuit.

Time(HH:MM:SS) : 00:01:35

It has no data inputs and no combinational output logic circuit at each clock pulse.

Time(HH:MM:SS) : 00:01:41

The counter takes up a new state and thus goes through a specific count sequence.

Time(HH:MM:SS) : 00:01:46

We shall now write a design spec for a binary up counter that is counting upwards with two inputs, which go through the following sequence zero, zero, zero one, one zero, one one, and back to zero zero.

Time(HH:MM:SS) : 00:01:58

It keeps on repeating.

Time(HH:MM:SS) : 00:02:00

The counter should have the following inputs.

Time(HH:MM:SS) : 00:02:03

It should have a reset which is supposed to be synchronous and is active low.

Time(HH:MM:SS) : 00:02:07

It should have a clock signal.

Time(HH:MM:SS) : 00:02:09

The counter should have the following output count one down to zero.

Time(HH:MM:SS) : 00:02:13

As we have seen the sequence, it's a two bit.

Time(HH:MM:SS) : 00:02:15

So the count one down to zero.

Time(HH:MM:SS) : 00:02:18

The block diagram structure and state transition diagram of a two bit binary counter built with two D-type flip flops, as shown in here.

Time(HH:MM:SS) : 00:02:27

All these criteria, as well as these diagrams will go into the design spec.

Time(HH:MM:SS) : 00:02:32

The truth table for the circuit is also shown here.

VideoTitle :What is Design Specification

Time(HH:MM:SS) : 00:00:01

What is a hardware description language or HDL

Time(HH:MM:SS) : 00:00:05

An HDL is a programming language with special constructs for modeling hardware concurrency, and timing.

Time(HH:MM:SS) : 00:00:11

It supports design at multiple levels of abstraction at behavioral level, structural level, and also regarding timing.

Time(HH:MM:SS) : 00:00:20

An HDL contains features and constructs to support description of the following three types.

Time(HH:MM:SS) : 00:00:25

Behavioral modeling, both serial and parallel, are available.

Time(HH:MM:SS) : 00:00:30

In serial behavior, you pass the output of one functional block to the input of another, which is similar to the behavior of a conventional software language.

Time(HH:MM:SS) : 00:00:39

However, in parallel behavior, you can pass a block output to the input of a number of blocks acting in parallel, where many separate events happen at the same moment in time.

Time(HH:MM:SS) : 00:00:50

Structural modeling.

Time(HH:MM:SS) : 00:00:52

This is supported both physical, such as hierarchical block diagrams and component netlists, and software such as subroutines.

Time(HH:MM:SS) : 00:01:00

This allows you to describe large complex systems and manage their complexity.

Time(HH:MM:SS) : 00:01:06

At the timing level, the programming languages have no concept of time.

Time(HH:MM:SS) : 00:01:10

An HDL has to model propagation delays, clock periods, and timing checks.

Time(HH:MM:SS) : 00:01:16

An HDL typically supports multiple abstraction levels as I just described.

Time(HH:MM:SS) : 00:01:21

You can describe hardware behaviorally without and with sufficient detail for logic synthesis, and is a structured netlist of predefined components that can themselves be as simple as a transistor and as complex as another behavioral design.

Time(HH:MM:SS) : 00:01:35

On the right side of the slide, I have shown pictorially how the behavioral level or structural modeling and timing looks in the form of design.

Time(HH:MM:SS) : 00:01:46

Why do we want to use a hardware description language or an HDL?

Time(HH:MM:SS) : 00:01:51

Benefits of using an HDL include the following.

Time(HH:MM:SS) : 00:01:54

You can capture and modify the design more quickly in an HDL than by schematic capture, which was an old procedure.

Time(HH:MM:SS) : 00:02:02

Earlier design captured at the higher abstraction level means earlier simulation, which facilitates design alternative exploration to produce a more optimal architecture and partitioning, and allows a more thorough verification.

Time(HH:MM:SS) : 00:02:15

Use of a standard HDL facilitates reuse of designs from previous projects or from commercial intellectual property providers IP providers.

Time(HH:MM:SS) : 00:02:25

You can move or switch between different tools and vendors without re-entry, reformat or translation of the design description.

Time(HH:MM:SS) : 00:02:33

Your RTL level or the register transfer level implementation is almost 100% independent of the implementation technology, so you can defer selection of the target ASIC or FPGA technology until the design is mostly entered, and then you can switch technology or vendor with a minimum of redesign.

Time(HH:MM:SS) : 00:02:51

This is the beauty of using HDL or the hardware description language.

Time(HH:MM:SS) : 00:02:59

A quick glimpse into before and after HDL.

Time(HH:MM:SS) : 00:03:02

Before the Verilog and VHDL languages were invented, designs were small and much less complex and were captured using schematics.

Time(HH:MM:SS) : 00:03:10

It was also simulated using SPICE.

Time(HH:MM:SS) : 00:03:13

With the advent of the Verilog and VHDL languages, designs are now much more larger and more complex.

Time(HH:MM:SS) : 00:03:20

They are designed using software and are simulated with Verilog or VHDL simulator.

Time(HH:MM:SS) : 00:03:28

You can describe a system as a group of hierarchical models of varying amount of detail.

Time(HH:MM:SS) : 00:03:34

As I explained earlier, an HDL supports multiple levels of such detail and supports multiple levels of abstraction accordingly.

Time(HH:MM:SS) : 00:03:42

The three main levels of abstraction are listed and described below.

Time(HH:MM:SS) : 00:03:46

As you can see in the slide on the left side, you can see the behavioral level, the register transfer level, or the RTL, the gate level, and the switch level.

Time(HH:MM:SS) : 00:03:56

On the right side, I have pictorially represented what it means, what goes into these levels, the algorithms, the nets and registers, the built in and user defined primitives and built in switch primitives.

Time(HH:MM:SS) : 00:04:09

The behavioral level in here you describe the system using mathematical equations.

Time(HH:MM:SS) : 00:04:14

You can omit the timing the system may simulate in zero time, like a software program in register transfer level or RTL.

Time(HH:MM:SS) : 00:04:24

You partition the system into combinational and sequential logic, as I have shown in the diagram here, using constructs and coding styles supported by logic synthesis.

Time(HH:MM:SS) : 00:04:34

You define timing in terms of cycles based on one or more defined clocks.

Time(HH:MM:SS) : 00:04:39

The structural level you instantiate and interconnect predefined components.

Time(HH:MM:SS) : 00:04:44

You can include vendor provided macrocells and can include logic primitives built into the language.

Time(HH:MM:SS) : 00:04:50

Similarly.

Time(HH:MM:SS) : 00:04:51

The gate level and switch levels are the structural levels.

Time(HH:MM:SS) : 00:04:55

So this is what I just described.

Time(HH:MM:SS) : 00:05:00

This slide shows the trade offs at different abstraction levels that you use.

Time(HH:MM:SS) : 00:05:04

The HDL simulation effort is proportional to the detail.

Time(HH:MM:SS) : 00:05:09

The less detail, the faster it is for design, entry in the simulation is faster.

Time(HH:MM:SS) : 00:05:15

More details slower is the design entry and slower is the simulation.

Time(HH:MM:SS) : 00:05:20

So behavioral level has less detail, just algorithms, hence faster simulation.

Time(HH:MM:SS) : 00:05:26

As we go towards the RTL level which is nets and register slightly more detailed than the simulation will decrease in speed.

Time(HH:MM:SS) : 00:05:35

So the simulation takes longer and the gate levels switch levels have far more details as it includes the timing information also.

Time(HH:MM:SS) : 00:05:43

So more detail, slower design entry and slower simulation users of an HDL model at different levels of abstraction.

Time(HH:MM:SS) : 00:05:52

The block architects are at the behavioral level.

Time(HH:MM:SS) : 00:05:55

They model functionality at this level for maximum simulation speed, and for the ease with which they can quickly modify the architecture as they explore alternatives.

Time(HH:MM:SS) : 00:06:05

Block implementers,At the RTL level, they refine the functional blocks to the RTL level for a logic synthesis tool.

Time(HH:MM:SS) : 00:06:13

The library developers at the structural level that is, at the gate level and switch level model cells at the gate level for simulation and the physical level for placement, place and route tools.

VideoTitle :What is Hardware Description Language

Time(HH:MM:SS) : 00:00:01

We are looking at this flowchart multiple times in this course, as it shows all the phases that the design goes through till we get the output in the form of GDS two.

Time(HH:MM:SS) : 00:00:11

The next section, the next few slides that I'm going to present will be dealing with RTL coding.

Time(HH:MM:SS) : 00:00:17

So this phase is highlighted in this flowchart.

VideoTitle :RTL Coding FlowChart

Time(HH:MM:SS) : 00:00:01

What is RTL coding?

Time(HH:MM:SS) : 00:00:03

Converting the design specifications set of rules into a high level design using a hardware description language such as Verilog, VHDL, Systemverilog, Systemc, etc.

Time(HH:MM:SS) : 00:00:15

is called RTL coding.

Time(HH:MM:SS) : 00:00:17

Rtl is an acronym for Register Transfer level.

Time(HH:MM:SS) : 00:00:21

Instead of using schematic capture, like in the olden days to design a complex chip, the designers now use an HDL.

Time(HH:MM:SS) : 00:00:29

Here is a small snippet from a terminal which shows the coding of multiplexer or a mux.

Time(HH:MM:SS) : 00:00:35

So it is having a module, component which is named as mux, and the outputs and the inputs are defined.

Time(HH:MM:SS) : 00:00:40

And the functionality of these outputs and inputs are described in the always block.

Time(HH:MM:SS) : 00:00:45

And the module is ended with an end module, construct.

Time(HH:MM:SS) : 00:00:48

So this is how typically RTL code looks like.

Time(HH:MM:SS) : 00:00:55

This slide shows an example of divide by two operation and goes on to show that at each abstraction level of behavior, RTL and structural, how the code might look and what kind of components go into the design at each abstraction level.

Time(HH:MM:SS) : 00:01:09

At the behavioral level.

Time(HH:MM:SS) : 00:01:11

So we have a D in signal and a D out.

Time(HH:MM:SS) : 00:01:14

So always at the input and the D out which is the output is D and divided by two.

Time(HH:MM:SS) : 00:01:20

Always code block loops forever.

Time(HH:MM:SS) : 00:01:22

Here the block is executed on every change in the input din at RTL.

Time(HH:MM:SS) : 00:01:28

At the register transfer level we have a D flip flop, always at the edge of the clock.

Time(HH:MM:SS) : 00:01:34

The output coming out of the flip flop is the right shifted value of the input.

Time(HH:MM:SS) : 00:01:39

So this is divide by two operation right shifting once is divide by two operation special non-blocking assignment is used here at the structural level we have these components instance array op of cell type f d1.

Time(HH:MM:SS) : 00:01:54

And it says FD13 down to zero.

Time(HH:MM:SS) : 00:01:57

The d flip flop the FD1 and the clock and the D out.

Time(HH:MM:SS) : 00:02:02

This is how it is described in the structural level.

Time(HH:MM:SS) : 00:02:08

This slide is given here in order to show you the usefulness and the benefit of HDL and how the same designs were expressed before using schematic based design.

Time(HH:MM:SS) : 00:02:18

This is a legacy schematic based design, so the low level proprietary non-portable stimulus will be given to the captured netlist and then manually the observation of the waveforms will take place.

Time(HH:MM:SS) : 00:02:31

Verify every function and timing, which is extremely time consuming.

Time(HH:MM:SS) : 00:02:35

So the designers used to capture the design intent using a proprietary schematic capture tool, manually develop the test stimulus in a proprietary tabular format, and simulated the design using a proprietary simulator.

Time(HH:MM:SS) : 00:02:49

Those who had money to burn had graphical displays to examine the results, but most of us had plain old text terminals so we could examine the results only with the zero and one characters.

Time(HH:MM:SS) : 00:03:00

We did not use the Z and X characters, because those values do not exist in hardware, and we use simulation only to generate expected results for a device test machine.

Time(HH:MM:SS) : 00:03:13

What is design verification?

Time(HH:MM:SS) : 00:03:15

Design verification verifies that the RTL design's functionality, logic and quality meet the design specifications.

Time(HH:MM:SS) : 00:03:24

During the verification phase of the digital design process, engineers are responsible for evaluating the functionality of the designed model and ensuring that the system performs its intended tasks.

Time(HH:MM:SS) : 00:03:37

This critical step involves a thorough examination of the system's performance under various conditions, as well as the identification and resolution of any errors or malfunctions that may arise.

Time(HH:MM:SS) : 00:03:49

So, to simulate or verify the RTL, we need a testbench with all the inputs or test cases that can provide to RTL to verify the functionality of the complete design.

Time(HH:MM:SS) : 00:04:00

Why digital verification is essential.

Time(HH:MM:SS) : 00:04:03

It helps users avoid costly design verification and time consuming errors and bugs in the implemented design.

Time(HH:MM:SS) : 00:04:11

Rtl verification is crucial for improving the performance, efficiency and reliability of the design system by verifying the RTL design, it minimizes the risk of functional failures, timing violations, and power issues, ensuring that the system works seamlessly.

Time(HH:MM:SS) : 00:04:30

Reusing verified RTL components and modules across different projects and platforms significantly saves time and resources, thereby increasing efficiency and productivity.

Time(HH:MM:SS) : 00:04:42

We have the test bench, which the design intent and RTL using the same standard HDL, now simulate them together using a choice of tools running on a wide choice of third party platforms.

Time(HH:MM:SS) : 00:04:54

And we have high definition wide screen displays that we do not have to stare at quite so much because our tests, to a large extent pinpoint any problems with a high degree of accuracy.

Time(HH:MM:SS) : 00:05:06

So we have a high level behavioral testbench, and it uses the behavioral code to simulate.

Time(HH:MM:SS) : 00:05:12

So the code is simulated using the Testbench.

Time(HH:MM:SS) : 00:05:15

It interacts with the RTL design, the RTL code which responds to the stimulus.

Time(HH:MM:SS) : 00:05:22

So the testbench checks whether the actual results are the expected results are equal.

Time(HH:MM:SS) : 00:05:28

Testbenches can be self-checking and design and tests vendor are independent, and it produces the waveforms only for debugging.

Time(HH:MM:SS) : 00:05:36

So there's an option to produce the waveforms if needed.

Time(HH:MM:SS) : 00:05:40

And this is how the HDL based simulation happens.

Time(HH:MM:SS) : 00:05:46

Who uses hardware description languages.

Time(HH:MM:SS) : 00:05:49

So the following personnel use an HDL system architects verification personnel.

Time(HH:MM:SS) : 00:05:55

Hardware engineers and model developers.

Time(HH:MM:SS) : 00:05:58

System architects create system level HDL models for high level architectural exploration verification personnel.

Time(HH:MM:SS) : 00:06:06

They create HDL testbenches for testing the components and systems of the design.

Time(HH:MM:SS) : 00:06:12

Hardware designers implement the system level models as register transfer level HDL for synthesis.

Time(HH:MM:SS) : 00:06:19

Model developers describe system level IP and ASIC or FPGA macrocells in an HDL.

VideoTitle :What is RTL Coding

Time(HH:MM:SS) : 00:00:01

Now let's go through the exercise of coding a very simple generic counter.

Time(HH:MM:SS) : 00:00:06

Let's see how we can behaviorally model a generic eight bit synchronous counter with an active low reset using the Verilog language.

Time(HH:MM:SS) : 00:00:14

First we create the design source file design V, or with VD or SV or with any other HDL language of your choice, and then create a testbench V, which simulates the design with dot V or dot VD or SV, or with any other HDL language of your choice.

Time(HH:MM:SS) : 00:00:35

The following is the design block on which we are going to work.

Time(HH:MM:SS) : 00:00:39

It's a eight bit counter.

Time(HH:MM:SS) : 00:00:41

It has a load, a clock clk and the reset st as the inputs and the CNT out as the output.

Time(HH:MM:SS) : 00:00:48

Eight bit synchronous CNT out is the output.

Time(HH:MM:SS) : 00:00:54

Now let's first create the design file counter v.

Time(HH:MM:SS) : 00:00:59

We will use the Module, construct in Verilog and the always procedural block to model the counter design.

Time(HH:MM:SS) : 00:01:05

The Module, captures the inputs and outputs and the always procedural block, which describes the functionality of this inputs and outputs how they're connected with the function.

Time(HH:MM:SS) : 00:01:16

So let's go over each of the statements in this counter file.

Time(HH:MM:SS) : 00:01:20

Module, counter has the inputs clock reset and output count.

Time(HH:MM:SS) : 00:01:25

The input are clock and the reset output is in the form of a register seven down to zero called count always at Posedge and, clk or negedge reset.

Time(HH:MM:SS) : 00:01:35

It is an active low reset.

Time(HH:MM:SS) : 00:01:37

Then begin if.

Time(HH:MM:SS) : 00:01:39

If it is active low reset then the count is reset.

Time(HH:MM:SS) : 00:01:42

Count gets zeros.

Time(HH:MM:SS) : 00:01:44

Else count equals count plus one.

Time(HH:MM:SS) : 00:01:47

It keeps on counting and end.

Time(HH:MM:SS) : 00:01:49

This is the end of the always block and end.

Time(HH:MM:SS) : 00:01:51

Module, for ending the module, counter.

Time(HH:MM:SS) : 00:01:57

Now let's create the Testbench to simulate the design file we created in the previous slide.

Time(HH:MM:SS) : 00:02:04

Let's call this counter Test.csv.

Time(HH:MM:SS) : 00:02:06

We need this test bench to verify the counter design we have coded.

Time(HH:MM:SS) : 00:02:10

We will code this two in Verilog.

Time(HH:MM:SS) : 00:02:13

So the Module, counter test and the clock SST are register types and the count is wire type wire seven down to zero count.

Time(HH:MM:SS) : 00:02:22

So initially you begin clock is zero.

Time(HH:MM:SS) : 00:02:25

Reset is zero.

Time(HH:MM:SS) : 00:02:26

Then you add ten after ten.

Time(HH:MM:SS) : 00:02:28

Then the reset gets the value one reset is one end counter is counter one clock reset count.

Time(HH:MM:SS) : 00:02:37

It gets those values of these three segments.

Time(HH:MM:SS) : 00:02:39

Then always hash five clock equals negation clock.

Time(HH:MM:SS) : 00:02:43

So clock gets the opposite value of the clock and it keeps clocking.

Time(HH:MM:SS) : 00:02:47

And the module, is ended.

Time(HH:MM:SS) : 00:02:49

This is how a test bench is written by giving values to the inputs to stimulate the output.

Time(HH:MM:SS) : 00:02:55

Depending on the functionality we have designed in the previous slide.

VideoTitle :Creating a Simple Generic Counter

Time(HH:MM:SS) : 00:00:00

In this module, you define the meaning of a design specification using an example of a simple counter design spec.

Time(HH:MM:SS) : 00:00:08

You implemented the specified design through the RTL coding process.

Time(HH:MM:SS) : 00:00:13

You identified what hardware description languages or hdls are.

Time(HH:MM:SS) : 00:00:17

You implemented a design spec for a given list of design criteria.

Time(HH:MM:SS) : 00:00:22

You coded a simple counter design, given a design spec by coding the design.v and the Testbench counter_test.V.

VideoTitle :Module Summary

Time(HH:MM:SS) : 00:00:00

Module three design simulation using the Xcelium simulator tool.

Time(HH:MM:SS) : 00:00:04

In this module you will recognize the process of simulation.

Time(HH:MM:SS) : 00:00:09

You will know about the compilation process, the elaboration process and also the simulation process.

Time(HH:MM:SS) : 00:00:15

Finally then you will execute the single step Xrun command method of simulating a design in both graphical and batch modes of Xcelium simulator tool.

VideoTitle :Design Simulation Using the Xcelium™ Simulator

Time(HH:MM:SS) : 00:00:00

This is again the familiar flowchart of all the phases, from design conception to the RTL to GDS two phase.

Time(HH:MM:SS) : 00:00:06

And in this chapter and the next few slides, we are going to look at the functional simulation.

Time(HH:MM:SS) : 00:00:12

The tool used is the Xcelium simulator and the GUI is the SIM vision GUI.

Time(HH:MM:SS) : 00:00:17

And the concept of simulation by using a simple counterexample is shown in the GUI mode, as well as the batch mode.

VideoTitle :Module Objectives

Time(HH:MM:SS) : 00:00:00

Let's learn about cadence Xcelium simulator.

Time(HH:MM:SS) : 00:00:03

The cadence Xcelium simulator is an advanced third generation parallel simulator that provides superior verification quality, outstanding simulation performance, and excellent regression throughput.

Time(HH:MM:SS) : 00:00:16

It boasts an exceptional verification turnaround time, making it a top choice for engineers and designers who need to verify complex designs with high accuracy and efficiency.

Time(HH:MM:SS) : 00:00:27

The simulator utilizes state of the art technology to ensure that simulations are fast, accurate, and reliable, providing designers with the necessary confidence to deliver high quality designs.

Time(HH:MM:SS) : 00:00:41

With its advanced capabilities and features, the Cadence Xcelium simulator is a powerful tool for any design and verification team.

Time(HH:MM:SS) : 00:00:50

When it comes to complex design scenarios, the efficiency of verification and debugging time is crucial.

Time(HH:MM:SS) : 00:00:57

The Xcelium simulator offers various features such as parallel simulation, intelligent inbuilt utilities support for multiple input file formats, High Performance System, Verilog, UVM, Testbenches, and low power verification.

Time(HH:MM:SS) : 00:01:13

These features help engineers achieve their verification goals and deliver high quality designs.

Time(HH:MM:SS) : 00:01:20

Working of Xcelium simulator Xcelium simulator runs in three steps compilation, elaboration and simulation.

Time(HH:MM:SS) : 00:01:30

Compilation is the process of reading and source code and analyzing the source code for syntax and semantic errors during compilation.

Time(HH:MM:SS) : 00:01:39

Xcelium simulator creates an intermediate data structure like system C data QCD for system C, abstract syntax tree AST for VHDL or Verilog.

Time(HH:MM:SS) : 00:01:53

Syntax tree VST for Verilog for each design unit in an efficient, accessible and interpretable format.

Time(HH:MM:SS) : 00:02:02

The HDL versions of these objects also contain pseudocode from which a code card object is created, containing machine specific executable code.

Time(HH:MM:SS) : 00:02:13

Elaboration is the process that occurs between parsing and simulation.

Time(HH:MM:SS) : 00:02:18

It binds modules to module instances, builds the design hierarchy, computes parameter values, resolves hierarchical names, establishes net connectivity, and prepares all this for simulation.

Time(HH:MM:SS) : 00:02:34

The collaborator generates an initial simulation snapshot containing information on the values of simulation objects such as nets, signals and variables, process state, and the execution point many more.

Time(HH:MM:SS) : 00:02:49

Simulation is the process by which the design model, coded in the HDL, gets executed after a successful compilation and elaboration based on a given execution model.

Time(HH:MM:SS) : 00:03:01

All these steps are achieved by a single command in the Xcelium simulator called xrun.

VideoTitle :Functional Simulation FlowChart

Time(HH:MM:SS) : 00:00:01

This slide shows the language support provided by the Xcelium simulator.

Time(HH:MM:SS) : 00:00:06

Please take some time to go through this table in detail and understand all the support provided by the Xcelium simulator for different languages.

VideoTitle :What is Xcelium Simulator

Time(HH:MM:SS) : 00:00:00

You can see the Xrun command format where we provide the file name and options.

Time(HH:MM:SS) : 00:00:06

This is a Linux command for the Xcelium simulator.

Time(HH:MM:SS) : 00:00:09

The Xcelium simulator supports specifying all input files and command line options using the Xrun utility.

Time(HH:MM:SS) : 00:00:17

This is the most basic way to use Xrun.

Time(HH:MM:SS) : 00:00:20

List the files to comprise the simulation on the command line, along with all the command line options that Xrun will pass to the appropriate compiler.

Time(HH:MM:SS) : 00:00:29

Elaborate, and then the simulator.

Time(HH:MM:SS) : 00:00:32

With its built in flexible use model, you can use Xrun to enable single core or multi-core engine features from one or more command lines by running the tool in single step or multi-step mode.

Time(HH:MM:SS) : 00:00:44

The first time you run the simulator with the Xrun command, it creates a scratch directory called Xcelium and creates required subdirectories with library.

Time(HH:MM:SS) : 00:00:56

Invokes the appropriate compiler for each file specified on the command line.

Time(HH:MM:SS) : 00:01:01

Invokes the collaborator to elaborate the design and generate a simulation snapshot.

Time(HH:MM:SS) : 00:01:08

Invokes the simulator to simulate the snapshot.

Time(HH:MM:SS) : 00:01:11

Let's see how single step and multi-step xrun invocation are done when using the Xrun utility in single step mode.

Time(HH:MM:SS) : 00:01:20

Xcelium supports specifying all input files and all command line options on one command line.

Time(HH:MM:SS) : 00:01:27

The Xrun command uses the following simple syntax in single step mode.

Time(HH:MM:SS) : 00:01:32

The compilation, elaboration and simulation processes are executed one by one automatically with the single Xrun command.

Time(HH:MM:SS) : 00:01:41

This mode calls the desired compiler by looking at file extensions and handles multiple libraries.

Time(HH:MM:SS) : 00:01:48

That's why cadence recommends using Xrun single step simulation.

Time(HH:MM:SS) : 00:01:53

Let's understand with an example.

Time(HH:MM:SS) : 00:01:56

This example has different types of input files and several options.

Time(HH:MM:SS) : 00:02:01

The files toport v and V are recognized as Verilog files and are compiled by the Verilog parser.

Time(HH:MM:SS) : 00:02:08

Xmvhdl or log.

Time(HH:MM:SS) : 00:02:10

The IEEE 1364 option is passed to the XML log compiler.

Time(HH:MM:SS) : 00:02:17

The file middle dot v is recognized as a VHDL file and is compiled by the VHDL parser.

Time(HH:MM:SS) : 00:02:24

Xmvhdl or.

Time(HH:MM:SS) : 00:02:27

The V 93 option is passed to the Xmvhdl or compiler.

Time(HH:MM:SS) : 00:02:32

The file verifier is recognized as a Specman E file and is compiled using sn_compile.sh.

Time(HH:MM:SS) : 00:02:42

After compiling the files, Xrun causes the XML elab to elaborate the design.

Time(HH:MM:SS) : 00:02:48

The access option is passed to the collaborator to provide read access to all the simulation objects created during elaboration.

Time(HH:MM:SS) : 00:02:57

After the collaborator has generated a simulation snapshot, Xmsim which is invoked for simulation.

Time(HH:MM:SS) : 00:03:04

This is how single step xrun execution takes place.

Time(HH:MM:SS) : 00:03:08

Let's see.

Time(HH:MM:SS) : 00:03:08

Multi-step xrun invocation when using the Xrun utility in multi-step mode.

Time(HH:MM:SS) : 00:03:15

Xcelium supports a flexible use model with the same behavior and feature set as traditional direct invocation.

Time(HH:MM:SS) : 00:03:23

Here we have three separate commands for compilation, elaboration and simulation.

Time(HH:MM:SS) : 00:03:29

During compilation, Xrun parses and compiles the specified source files, but does not elaborate.

Time(HH:MM:SS) : 00:03:37

During elaboration, Xrun parses and compiles source files, elaborates the design and generates a simulation snapshot, but does not simulate it.

Time(HH:MM:SS) : 00:03:48

During simulation, Xrun simulates the last snapshot generated by the xrun utility.

Time(HH:MM:SS) : 00:03:55

Let's understand with an example.

Time(HH:MM:SS) : 00:03:58

Consider the shown example in which dash compile option with xrun will compile the Verilog source files by calling Verilog parser and VHDL files by calling VHDL parser.

Time(HH:MM:SS) : 00:04:11

After compilation, elaboration is done using elaborate and elab only options with Xrun.

Time(HH:MM:SS) : 00:04:18

During this command execution, initial snapshot is generated by default.

Time(HH:MM:SS) : 00:04:24

The elaborate option compiles and elaborates the design to only elaborate a design.

Time(HH:MM:SS) : 00:04:31

Elab only option is used with elaborate option with Xrun command, so user can execute one command to perform compilation and elaboration together or separately using these commands.

Time(HH:MM:SS) : 00:04:45

At last, simulation is done using Dash capital R option with Xrun.

Time(HH:MM:SS) : 00:04:51

If dash small R is used, the user must provide the initial snapshot name generated on the elaboration step.

Time(HH:MM:SS) : 00:04:59

This is how the Xrun utility is used to perform compilation, elaboration and simulation in multiple steps.

VideoTitle :Language Support for Xcelium Simulator

Time(HH:MM:SS) : 00:00:00

Xcelium library and directory structure.

Time(HH:MM:SS) : 00:00:03

Here we'll go through the generated library and directories after the Xrun invocation Xcelium library structure is organized according to a library cell view approach.

Time(HH:MM:SS) : 00:00:16

Let's understand these terms.

Time(HH:MM:SS) : 00:00:18

Library A collection of related cells that describe components of a single design or standard.

Time(HH:MM:SS) : 00:00:24

Components used in many designs.

Time(HH:MM:SS) : 00:00:27

Cell A cell is an object with a unique name stored in a library.

Time(HH:MM:SS) : 00:00:33

Each Verilog module macro Module, UDP, VHDL, entity architecture package, package, body or configuration is a unique cell view.

Time(HH:MM:SS) : 00:00:46

A view is a version of a cell.

Time(HH:MM:SS) : 00:00:48

Xcelium automatically creates a work library called work lib in the Xcelium directory.

Time(HH:MM:SS) : 00:00:55

The work library contains one PAC file.

Time(HH:MM:SS) : 00:00:59

The Xrun utility also creates a scratch subdirectory under the Xcelium directory.

Time(HH:MM:SS) : 00:01:06

Xcelium can generate different directories and libraries if it is specified in the command line option with xrun.

Time(HH:MM:SS) : 00:01:13

Consider a Verilog design to visualize the generated directory structure a module might ship instantiates two other modules, M1 and M2.

Time(HH:MM:SS) : 00:01:24

The Verilog description of my chip is written in the file my chip V, but multiple descriptions of M1 and M2 have also been generated for M1.

Time(HH:MM:SS) : 00:01:35

A behavioral and an RTL description is described in M1, VB and M1 ver respectively.

Time(HH:MM:SS) : 00:01:44

For M2, there is both an RTL and a synthesized gate level representation described in m2 ver and m2 vg respectively.

Time(HH:MM:SS) : 00:01:56

Here we can see the explained designs, generated directory and library structure.

Time(HH:MM:SS) : 00:02:02

All source files reside in the src subdirectory from which tools are invoked.

Time(HH:MM:SS) : 00:02:08

The work library, is saved in the Xcelium directory by default.

Time(HH:MM:SS) : 00:02:14

The CDs lib and hdl var files are in the src directory.

Time(HH:MM:SS) : 00:02:20

The source file and intermediate files are also found in the src directory.

Time(HH:MM:SS) : 00:02:26

The generated directories and libraries hierarchy are simple and in a simplified version.

VideoTitle :Invoking xrun command

Time(HH:MM:SS) : 00:00:00

Some of the frequently used xrun options..

Time(HH:MM:SS) : 00:00:03

Let's review the frequently used command line options, which are essential for every user.

Time(HH:MM:SS) : 00:00:09

These options are listed in a table with their corresponding functionalities and usage.

Time(HH:MM:SS) : 00:00:15

We will go over each of these options and their details in the second column.

Time(HH:MM:SS) : 00:00:19

The F or capital F, followed by the file name, reads options or arguments from the specified file.

Time(HH:MM:SS) : 00:00:27

The small F scans the argument or arg file for files relative to xrun in location directory.

Time(HH:MM:SS) : 00:00:34

The dash capital f option, followed by the file name, first looks for files relative to the location of the argument file, and then re scans relative to the invocation directory.

Time(HH:MM:SS) : 00:00:46

The Dash srv option forces system Verilog compilation.

Time(HH:MM:SS) : 00:00:50

If you have any system Verilog files with SRV extension, then you need to use SRV in your command line along with Xrun command.

Time(HH:MM:SS) : 00:01:01

The access followed by RWC will turn on the access to read, write or connectivity access that is RW or C dash.

Time(HH:MM:SS) : 00:01:11

B93 will enable VHDL 93 features to compile, elaborate and simulate VHDL files.

Time(HH:MM:SS) : 00:01:20

Add this option.

Time(HH:MM:SS) : 00:01:22

Dash line debug allows breakpoints on lines of source code.

Time(HH:MM:SS) : 00:01:26

This also executes dash access plus RWC.

Time(HH:MM:SS) : 00:01:31

The Xrun help option provides access to numerous additional options that can be explored for further functionality.

Time(HH:MM:SS) : 00:01:38

Let's discuss defining and utilizing Xrun HDL variables.

Time(HH:MM:SS) : 00:01:43

If you want to enter a string of options that you would normally use on the command line, you can define the Xrun ops variable.

Time(HH:MM:SS) : 00:01:51

Although you can still enter more options on the command line, some options cannot be specified in this variable.

Time(HH:MM:SS) : 00:01:58

The utility requires to know the 64 bit append underscore log CDs lib HDL var no copyright or version options before it examines the Xrun opts variable to define the Xrun ops variable as per your requirements, you first need to include the path to project HDL var.

Time(HH:MM:SS) : 00:02:20

If you want to compile different source files with different options, you can define the file underscore opt, underscore, map variable.

Time(HH:MM:SS) : 00:02:28

It maps specific sets of xrun options to individual files or directories, including Verilog or VHDL source files, or directories that contain Verilog or VHDL files.

Time(HH:MM:SS) : 00:02:41

This allows you to compile different source files with different options.

Time(HH:MM:SS) : 00:02:46

Additionally, you can define the work variable to specify the work library, in which to place the compiled simulation units.

Time(HH:MM:SS) : 00:02:54

This defines a work library, where you can place all your simulation units.

VideoTitle :Directory-Library-Design Hierarchy

Time(HH:MM:SS) : 00:00:00

What is Verisium Debug?

Time(HH:MM:SS) : 00:00:02

Nowadays, digital designs, like S O C and IPs, are more complex, and sizes are increasing.

Time(HH:MM:SS) : 00:00:10

This brings challenges for design engineers, verification engineers, integrators, and validation engineers are also increasing to deliver a robust design.

Time(HH:MM:SS) : 00:00:21

Debugging any design is the most time taking phase for any engineer. To make little life easier for the engineers, Cadence has Verisium Debug.

Time(HH:MM:SS) : 00:00:31

Cadence Verisium™ Debug is an advanced debugging tool that helps design engineers, verifications engineers, integrators, and validation engineers explore, analyze, and debug complex designs and test benches, regardless of their size and language.

Time(HH:MM:SS) : 00:01:24

Verisign debug uses modern and innovative debug concepts like automated root cause analysis, RCA Smart Log, advanced Search hierarchy, browser source code, Viewer., HDL schematic browser, and advanced data exploration techniques using VeriSign debug.

Time(HH:MM:SS) : 00:01:44

An engineer can uncover bugs up to 50% faster than traditional sample based debug approaches.

Time(HH:MM:SS) : 00:01:52

Let's see what advantages VeriSign debug offers over traditional simulators.

Time(HH:MM:SS) : 00:01:57

Scalability and performance.

Time(HH:MM:SS) : 00:02:00

Verisign debug quickly loads large designs, search efficiently in the databases, and is capable of displaying large waveform databases.

Time(HH:MM:SS) : 00:02:10

Navigability.

Time(HH:MM:SS) : 00:02:11

This platform offers very efficient and fast navigation within the tool window like driver tracing, smart log and root cause analysis.

Time(HH:MM:SS) : 00:02:21

Smart log.

Time(HH:MM:SS) : 00:02:23

The VeriSign Debug Smart Log is a dynamic, hyperlinked, and colorful log file message viewer Viewer..

Time(HH:MM:SS) : 00:02:30

From the smart log, the user can locate the hierarchical object from where the message is printing and send messages to the waveform window.

Time(HH:MM:SS) : 00:02:39

Errors are automatically bookmarked and are in red color.

Time(HH:MM:SS) : 00:02:42

Many more user friendly features are present in the smart log.

Time(HH:MM:SS) : 00:02:47

Root Cause Analysis RCA.

Time(HH:MM:SS) : 00:02:49

With the VeriSign debug RCA tool, users can navigate through a complete failure scenario, including the Testbench and RTL code to diagnose the root cause of the failure.

Time(HH:MM:SS) : 00:03:01

Searchability VeriSign debug has a specialized pre-index search database for fast searching throughout the design.

Time(HH:MM:SS) : 00:03:10

It provides hyperlinked search results organized by source, value type, and log integrability.

Time(HH:MM:SS) : 00:03:18

The VeriSign debug provides a cross engine debugger that works with Xcelium, palladium, and embedded software.

Time(HH:MM:SS) : 00:03:27

Portability.

Time(HH:MM:SS) : 00:03:28

Users can save the complete debugging state like highlighted values, bookmarks, and notes for quick handoff to another engineer who can continue the debugging process.

Time(HH:MM:SS) : 00:03:40

With this feature, multiple teams can exchange debugging data efficiently.

VideoTitle :xrun frequently used options

Time(HH:MM:SS) : 00:00:01

This slide and the next slide show a quick overview of the SIM vision tool functions.

Time(HH:MM:SS) : 00:00:07

What are all the functions that can be performed using this tool?

Time(HH:MM:SS) : 00:00:10

Some of the main ones.

Time(HH:MM:SS) : 00:00:12

So using the different windows of the tool it has been explained what is the function they perform.

Time(HH:MM:SS) : 00:00:19

The first one is the console.

Time(HH:MM:SS) : 00:00:20

And in here you can type in your commands at the Xcelium prompt.

Time(HH:MM:SS) : 00:00:24

You have a SIM vision tab here and a simulator tab.

Time(HH:MM:SS) : 00:00:28

So you simulate your design in this console.

Time(HH:MM:SS) : 00:00:31

There is a design browser window wherein you can see the hierarchy of your design and keep on going to different inner levels.

Time(HH:MM:SS) : 00:00:37

And you can see the objects at each hierarchy.

Time(HH:MM:SS) : 00:00:40

And there are small tabs here at the top which go to the other windows once clicked.

Time(HH:MM:SS) : 00:00:45

There is a waveform window, and if you click the waveform symbol here in the Design Browser window, it takes you here.

Time(HH:MM:SS) : 00:00:51

Or you can open the windows on the top and click on waveform to get to this.

Time(HH:MM:SS) : 00:00:56

And you can observe the waveform and get to debug.

Time(HH:MM:SS) : 00:01:02

Continuing the quick overview of the SIM vision window functions from the previous slide.

Time(HH:MM:SS) : 00:01:07

In here we are showing the source browser window.

Time(HH:MM:SS) : 00:01:11

You can open the source code of the design using this and look at it in detail.

Time(HH:MM:SS) : 00:01:15

You might edit it to.

Time(HH:MM:SS) : 00:01:17

There is a schematic tracer option schematic Tracer window wherein you can trace the drivers of your design.

Time(HH:MM:SS) : 00:01:24

You can zoom in, go to the most detailed part of each object in your design.

Time(HH:MM:SS) : 00:01:29

So this is how it looks.

Time(HH:MM:SS) : 00:01:32

Each of these windows are explained in detail in the next few slides in this section, and also other windows apart from all this are also explained.

VideoTitle :Introduction to Verisium Debug

Time(HH:MM:SS) : 00:00:01

Executing Xrun in GUI mode.

Time(HH:MM:SS) : 00:00:04

The command Xrun followed by the source files, your design, and the Testbench followed by hyphen access with read, write, and connectivity options attached to it.

Time(HH:MM:SS) : 00:00:14

With hyphen, GUI will invoke the GUI mode with the SimVision and tool opening.

Time(HH:MM:SS) : 00:00:20

As you can see here in this box I have given the command Xrun source files.

Time(HH:MM:SS) : 00:00:24

Hyphen access rwc hyphen GUI.

Time(HH:MM:SS) : 00:00:27

Xrun takes files from different simulation languages such as Verilog, Systemverilog, VHDL, Verilog-AMS and VHDL AMS Specman E and files written in general programming languages like C and C plus plus and compiles them using the appropriate compilers.

Time(HH:MM:SS) : 00:00:46

After the input files have been compiled, Xrun automatically invokes XM e-lab to elaborate the design and then automatically invokes the XM sim simulator.

Time(HH:MM:SS) : 00:00:57

The most basic way to use Xrun is to list the files that are to comprise the simulation on the command line, just like I have shown above in the command box, along with all command line options that Xrun will pass to the appropriate compiler elaborate and then simulator hyphen GUI.

Time(HH:MM:SS) : 00:01:15

will invoke the SimVision and.

Time(HH:MM:SS) : 00:01:17

The screenshots here show the command I have given for a simple counter design file.

Time(HH:MM:SS) : 00:01:22

Counters and a Testbench counter underscore test dot v Xrun counter dot v counter underscore test v hyphen access RWC for read write connectivity options, and hyphen GUI..

Time(HH:MM:SS) : 00:01:37

This will elaborate.

Time(HH:MM:SS) : 00:01:38

Create a snapshot and then gives the control to the SimVision and.

Time(HH:MM:SS) : 00:01:42

So the SIM vision tool is opening here as you can see.

Time(HH:MM:SS) : 00:01:48

The SIM vision tool opens so the control from the terminal is relinquished to the SIM vision tool.

Time(HH:MM:SS) : 00:01:55

The design browser window and the console, which is where you give the commands at the Xcelium prompt are opened.

Time(HH:MM:SS) : 00:02:02

So in the design browser you can see the hierarchy simulator under the counter underscore test Testbench and the instance instantiation of counter one and the signals present in the objects window.

Time(HH:MM:SS) : 00:02:14

In the console window, you can click on the run button here in order to run the simulation.

Time(HH:MM:SS) : 00:02:22

You can view the source code using the source browser, and invoke the waveform window from your design

browser by clicking on the appropriate icon from that and wherever you click in the waveform, the appropriate source is.

Time(HH:MM:SS) : 00:02:35

Source code is shown in the source browser and in the waveform.

Time(HH:MM:SS) : 00:02:39

You can view it with different kinds of zooming options and also expanding options and expand the counter and then see if you can debug if there is a problem.

Time(HH:MM:SS) : 00:02:48

And in the terminal I have given here the screenshot of the terminal, you can see that waves shm is created Xcelium D is created according to the Xrun command execution and the xrun How history.

Time(HH:MM:SS) : 00:03:01

Xrun key Xrun dot log are also created.

Time(HH:MM:SS) : 00:03:06

These are all the files which get created after you execute the Xrun command.

Time(HH:MM:SS) : 00:03:13

Executing Xrun in batch mode.

Time(HH:MM:SS) : 00:03:16

So the command we use here is Xrun followed by the list of the source files.

Time(HH:MM:SS) : 00:03:21

Your design file, your Testbench, and the hyphen access option with RWC for read, write and connectivity options.

Time(HH:MM:SS) : 00:03:30

Hyphen node GUI.

Time(HH:MM:SS) : 00:03:31

option is given and the command runs in the background.

Time(HH:MM:SS) : 00:03:34

The control is not given to the SimVision and GUI., so this is just the batch mode.

Time(HH:MM:SS) : 00:03:40

The Xcelium prompt can be used to issue other commands to view the result of the simulation.

Time(HH:MM:SS) : 00:03:46

As you can see here in the terminal snapshot I have attached, I have my counter dot v counter underscore test dot v.

Time(HH:MM:SS) : 00:03:53

I have given the command xrun counter v counter underscore test v hyphen access RWC and you can see it compiles elaborates, creates a snapshot and it runs.

Time(HH:MM:SS) : 00:04:06

The run command is issued for simulation.

Time(HH:MM:SS) : 00:04:08

And I just interrupted the counting of the counter.

Time(HH:MM:SS) : 00:04:11

So it just stops.

Time(HH:MM:SS) : 00:04:13

It gets interrupted and stops and goes back to the Xcelium prompt, from where you can issue other commands to view the result of the simulation.

Time(HH:MM:SS) : 00:04:21

If you look at the directory where you executed the Xrun command, you can see that Xcelium dot d is created.

Time(HH:MM:SS) : 00:04:28

The Xrun dot history Xrun dot key Xrun dot log files are also created.

VideoTitle :Quick Overview of SimVision Functions

Time(HH:MM:SS) : 00:00:01

To summarize in this Module, three, we recognize the process of simulation, and the tool we used was the Xcelium simulator.

Time(HH:MM:SS) : 00:00:10

We looked at the compilation, elaboration and simulation aspects in detail, and we executed the single step Xrun method of simulation in both graphical by invoking the SimVision and tool as well as the batch mode in the xterm or the terminal.

VideoTitle :xrun in GUI and Batch Mode

Time(HH:MM:SS) : 00:00:01

This slide shows the several references and resources that are available to you apart from this course.

VideoTitle :Module Summary

Time(HH:MM:SS) : 00:00:01

Lab three.

Time(HH:MM:SS) : 00:00:02

In module three, we discuss the concept of the functional simulation.

Time(HH:MM:SS) : 00:00:07

So in this lab we are going to simulate a simple counter design.

Time(HH:MM:SS) : 00:00:11

Given a counter v and counter underscore test v, you will execute the xrun command in GUI.

Time(HH:MM:SS) : 00:00:17

mode by invoking the SIM vision tool, and also the batch mode to simulate the simple counter design with the Testbench provided.

Time(HH:MM:SS) : 00:00:24

As I mentioned, the counter v and counter underscore test V are given to you.

VideoTitle :Module 3 Reference Slide

Time(HH:MM:SS) : 00:00:00

Module 4 code coverage using the Integrated Metrics Center tool or the IMC tool.

Time(HH:MM:SS) : 00:00:05

In this chapter or module, you will identify the concept of coverage.

Time(HH:MM:SS) : 00:00:09

In verification process, we will identify the different kinds of coverage, functional code and also FSM or finite state machine.

Time(HH:MM:SS) : 00:00:19

You will identify the different kinds of code coverage the block, branch, expression and toggle.

Time(HH:MM:SS) : 00:00:26

You will also launch the integrated metrics tool IMC and go through the flow for code coverage.

VideoTitle :Lab

Time(HH:MM:SS) : 00:00:00

We are looking at this flowchart, which is familiar by now again, which shows all the phases from the design conception till the GDS two phase.

Time(HH:MM:SS) : 00:00:08

And in this module, the next set of slides cover the coverage analysis part of the flow.

VideoTitle :Code Coverage Using the Integrated Metrics Center

Time(HH:MM:SS) : 00:00:00

Coverage, goals and process.

Time(HH:MM:SS) : 00:00:02

Let's try to understand a little bit more about the role of coverage in verification.

Time(HH:MM:SS) : 00:00:06

The goal of coverage is to measure how well the Testbench verifies the design.

Time(HH:MM:SS) : 00:00:11

It identifies design areas in which to focus verification efforts and also estimates the remaining verification effort.

Time(HH:MM:SS) : 00:00:19

The process covered is to instrument the design to collect the coverage data, to reduce the coverage data, and then to analyze the coverage data.

Time(HH:MM:SS) : 00:00:27

Coverage is all about the verification environment, not about the design.

Time(HH:MM:SS) : 00:00:32

It identifies shortcomings of the verification environment that result in sufficient verification of the design.

Time(HH:MM:SS) : 00:00:38

What matters most is detecting what coverage points are not covered, as these are holes in the verification process that must be addressed by tracking verification progress over time and comparing it to that of previous projects, you can predict the additional effort required to reach the coverage target.

Time(HH:MM:SS) : 00:00:56

This section introduces the section of the slides, introduces the coverage process.

Time(HH:MM:SS) : 00:01:01

In step one, the tools automatically add coverage monitors to explicit or implicit coverage points, and the user manually adds coverage monitors for functional coverage.

Time(HH:MM:SS) : 00:01:11

In step two, the monitors count the coverage point hits throughout the simulation.

Time(HH:MM:SS) : 00:01:16

In step three, the tools reduce the coverage data to make it more manageable and record the data to coverage databases for later analysis.

Time(HH:MM:SS) : 00:01:24

In step four, the user analyzes the coverage data and diagnosis the reports to determine the additional test requirements.

Time(HH:MM:SS) : 00:01:32

Hence, the four steps are.

Time(HH:MM:SS) : 00:01:34

Instrument the design.

Time(HH:MM:SS) : 00:01:35

Collect the coverage data, reduce the coverage data, and analyze the coverage data to better understand the metric analysis that is supported by the Vmanager tool.

Time(HH:MM:SS) : 00:01:45

Let's go over some of the coverage types.

Time(HH:MM:SS) : 00:01:48

The basic definition of this coverage type, so that we can better understand the metrics analysis in the next few slides.

Time(HH:MM:SS) : 00:01:54

Code coverage it could be of type blog branch, expression, or toggle.

Time(HH:MM:SS) : 00:02:00

Code coverage analyzes that HDL code structure.

Time(HH:MM:SS) : 00:02:03

That is, it will give a sense of which blocks of design code are being executed, which branch, which expression and the toggles.

Time(HH:MM:SS) : 00:02:09

Also, it determines how fully code structure is exercised.

Time(HH:MM:SS) : 00:02:14

Functional coverage.

Time(HH:MM:SS) : 00:02:16

It could be of data oriented or control oriented.

Time(HH:MM:SS) : 00:02:19

It analyzes the design functionality, for example, which functions such as add and queue filled are executed and also control flows like how fully design behavior is exercised.

Time(HH:MM:SS) : 00:02:31

Finite state machine or FSM.

Time(HH:MM:SS) : 00:02:34

It could be of state type, transition type, or arc type.

Time(HH:MM:SS) : 00:02:38

It interprets the synthesis semantics of the HDL design and monitors the coverage of the FSM representation of control logic blocks in the design.

Time(HH:MM:SS) : 00:02:48

Analyzing code coverage.

Time(HH:MM:SS) : 00:02:50

Code coverage measures how thoroughly a testbench exercises the lines of HDL code that describe a design.

Time(HH:MM:SS) : 00:02:57

This coverage results reveal areas of the design that have not been fully tested, or that did not meet the desired coverage criteria.

Time(HH:MM:SS) : 00:03:05

Code coverage includes the following, which I described a couple of slides Lines before.

Time(HH:MM:SS) : 00:03:10

It includes block coverage, expression coverage, toggle coverage, etc..

Time(HH:MM:SS) : 00:03:15

Now let's look at the functional coverage definition as well as how it works.

Time(HH:MM:SS) : 00:03:20

Functional coverage focuses on functional aspects of design and provides a very good insight on how the verification goals set by a test plan or verification plan are being met.

Time(HH:MM:SS) : 00:03:31

It is generated by inserting Property Specification Language or TSL or Systemverilog assertions, SVA or the Systemverilog cover group statements into the source code and then simulating the design.

Time(HH:MM:SS) : 00:03:44

The functional coverage points specify scenarios, error cases, corner cases, and protocols to be covered, and also specifies analysis to be done on different values of a variable.

Time(HH:MM:SS) : 00:03:55

It can be of two types.

Time(HH:MM:SS) : 00:03:57

The assertion coverage, which is covered under SVA, and the cover group coverage, which are the cover group statements that are inserted into the code.

Time(HH:MM:SS) : 00:04:05

Now let us understand the difference between the code coverage and the functional coverage.

Time(HH:MM:SS) : 00:04:10

Code coverage basically focuses on the design code like block execution, execution of the expression terms in the code.

Time(HH:MM:SS) : 00:04:18

It is instrumented by tool, blindly focusing on individual items in your code.

Time(HH:MM:SS) : 00:04:23

It is more easy to set up and less easy to analyze.

Time(HH:MM:SS) : 00:04:27

Functional coverage focuses on design functions.

Time(HH:MM:SS) : 00:04:30

It focuses on the data values and the control sequences.

Time(HH:MM:SS) : 00:04:34

The data oriented and the control oriented.

Time(HH:MM:SS) : 00:04:36

Functional coverage.

Time(HH:MM:SS) : 00:04:38

As I explained earlier in one of the slides, it is instrumented by the user, specifies the scenarios, the corner cases, as well as the protocols.

Time(HH:MM:SS) : 00:04:47

It is less easy to set up, but more easy to analyze because specifically we define the scenarios, corner cases and protocols.

Time(HH:MM:SS) : 00:04:55

The user gets to do this and knows exactly what to analyze.

Time(HH:MM:SS) : 00:04:59

Let's learn a little bit more about the code coverage and functional coverage here.

Time(HH:MM:SS) : 00:05:03

Code coverage focuses on design code such as blogs, expressions, and signals.

Time(HH:MM:SS) : 00:05:09

For code coverage, the tool instruments the design, but the tool cannot understand the design functionality.

Time(HH:MM:SS) : 00:05:15

You can relatively easily specify what code coverage to collect, but to understand the code coverage metrics is

more difficult.

Time(HH:MM:SS) : 00:05:22

Functional coverage focuses on design functions such as data values and control sequences.

Time(HH:MM:SS) : 00:05:28

For functional coverage, the tool cannot understand the design functionality, so the user instruments the design to instrument.

Time(HH:MM:SS) : 00:05:36

The design is relatively more difficult, but then to understand the functional coverage metrics is more easy.

Time(HH:MM:SS) : 00:05:43

You perform functional coverage on functional coverage points that you specify that, hence the analysis is easier.

Time(HH:MM:SS) : 00:05:49

You specify these coverage points using Systemverilog cover groups to specify variable values and value transitions to cover and systemverilog assertions, which is SVA or the PSL Property Specification language to specify control scenarios, error cases, corner cases, and protocols to implement functional coverage, you must be familiar with the design.

Time(HH:MM:SS) : 00:06:11

Really thorough functional coverage can further be classified as control oriented or data oriented, as I mentioned earlier.

VideoTitle :Module Objectives

Time(HH:MM:SS) : 00:00:00

Analyzing code coverage.

Time(HH:MM:SS) : 00:00:02

Code coverage measures how thoroughly a testbench exercises the lines of HDL code that describe a design.

Time(HH:MM:SS) : 00:00:09

This coverage results reveal areas of the design that have not been fully tested or that did not meet the desired coverage criteria.

Time(HH:MM:SS) : 00:00:16

Code coverage includes the following which I described a couple of slides before.

Time(HH:MM:SS) : 00:00:21

It includes block coverage, expression coverage, toggle coverage, etc.

Time(HH:MM:SS) : 00:00:26

What is a code block?

Time(HH:MM:SS) : 00:00:28

A code block is a sequence of statements that always execute together.

Time(HH:MM:SS) : 00:00:33

A block is a contiguous set of statements that execute together, and any construct that breaks execution flow creates a new block.

Time(HH:MM:SS) : 00:00:40

The Incisive Comprehensive Coverage or the Integrated Comprehensive Coverage which it is called now, does not necessarily start the new block at the flow break construct.

Time(HH:MM:SS) : 00:00:50

For example, the Verilog at token @ and token # create a new block that starts on the following or the next statement.

Time(HH:MM:SS) : 00:00:58

For Verilog, blocks are lines of Verilog code within a function, procedural block or task.

Time(HH:MM:SS) : 00:01:04

The tool does not by default score.

Time(HH:MM:SS) : 00:01:07

Verilog continuous assignments, to score a continuous assignment, include the set_assign_scoring command in the coverage configuration file.

Time(HH:MM:SS) : 00:01:15

This scores it as one block.

Time(HH:MM:SS) : 00:01:18

To score the selections of a conditional expression, you should also include the set_branch_scoring command.

Time(HH:MM:SS) : 00:01:24

The coverage tool does not score Verilog primitives, and no option exists to change this behavior.

Time(HH:MM:SS) : 00:01:30

For VHDL, blocks are lines of VHDL code within a function, process or procedure.

Time(HH:MM:SS) : 00:01:37

The tool does not score VHDL concurrent signal assignment statements as the set_assign_scoring command does not apply to VHDL.

Time(HH:MM:SS) : 00:01:45

To score the selection of a concurrent signal assignment, include the set_branch_scoring command.

Time(HH:MM:SS) : 00:01:51

The coverage tool does not score VHDL or vital primitives, and no option exists to change this behavior.

Time(HH:MM:SS) : 00:01:58

I have included a table here for the flow break constructs and Verilog and VHDL has two columns and whatever I explained until now is given in more detail in this table.

Time(HH:MM:SS) : 00:02:07

Please take a few moments to go through the table and try to understand the concept I just explained.

Time(HH:MM:SS) : 00:02:12

In this slide, I have included two example code blocks, one for Verilog on the left and the one on the right is the VHDL.

Time(HH:MM:SS) : 00:02:20

I have named the blocks wherever there is a begin and end and how it is segregated within the code.

Time(HH:MM:SS) : 00:02:25

Any construct that breaks execution flow, creates a new block.

Time(HH:MM:SS) : 00:02:30

This illustrates for some typical Verilog and VHDL code where the tool considers a new block to begin.

Time(HH:MM:SS) : 00:02:37

For example, the tool considers a new block to begin with the Verilog begin statement and with the statement after the VHDL begin statement.

Time(HH:MM:SS) : 00:02:45

Here is a small exercise for you to make it a fun learning.

Time(HH:MM:SS) : 00:02:48

How many code blocks exist in this code?

Time(HH:MM:SS) : 00:02:51

So on the left, I have given a Verilog example with the line numbers and on the right, I have given the VHDL example.

Time(HH:MM:SS) : 00:02:58

You could go through the slides previously explained and also the notes section given here in the slide to understand more about the code blocks and how they are segregated within the Verilog and VHDL.

Time(HH:MM:SS) : 00:03:09

So let the fun begin.

Time(HH:MM:SS) : 00:03:11

Let's see how you can recognize the code blocks in this code.

Time(HH:MM:SS) : 00:03:14

What is a code branch?

Time(HH:MM:SS) : 00:03:16

A code branch is a statement that conditionally executes.

Time(HH:MM:SS) : 00:03:20

The branch coverage requires and extends block coverage.

Time(HH:MM:SS) : 00:03:24

It separately covers the branches of if and case statements.

Time(HH:MM:SS) : 00:03:27

These branches are already covered as blocks, with the exception of multiple Verilog case branches that execute the same one block.

Time(HH:MM:SS) : 00:03:35

It covers the branches of the Verilog ternary assignment, both as a procedural assignment and as a continuous assignment.

Time(HH:MM:SS) : 00:03:42

It covers the branches of the VHDL conditional and selected signal assignments.

Time(HH:MM:SS) : 00:03:48

So in here, I have given two code blocks, one for the Verilog continuous statement and VHDL concurrent example on the right.

Time(HH:MM:SS) : 00:03:56

So in the Verilog continuous assignment, assign y = c ?

Time(HH:MM:SS) : 00:04:00

or a or b.

Time(HH:MM:SS) : 00:04:02

So there is a branching, which happens as soon as y gets assigned a value.

Time(HH:MM:SS) : 00:04:07

So Similarly on the VHDL concurrent example, y gets the value of a when c = '1' else gets the value of b, so this is how the branching happens.

Time(HH:MM:SS) : 00:04:17

Block coverage identifies the lines of code that get executed during a simulation run.

Time(HH:MM:SS) : 00:04:22

It helps you to determine whether the various test-benches exercise the statements in a block.

Time(HH:MM:SS) : 00:04:28

You can analyze block coverage using the block page of Analysis Center GUI.

Time(HH:MM:SS) : 00:04:33

A blog is a contiguous set of statements that always execute together.

Time(HH:MM:SS) : 00:04:37

It is a block of statements.

Time(HH:MM:SS) : 00:04:39

Either none of the statements executes or all of the statements execute.

Time(HH:MM:SS) : 00:04:43

A branch or delay statement starts another block.

Time(HH:MM:SS) : 00:04:47

Block coverage is a basic code coverage mechanism that helps you to identify which lines in the code have executed and which have not.

Time(HH:MM:SS) : 00:04:54

Block coverage helps you to identify whether various test scenarios exercise the statements inside your block.

Time(HH:MM:SS) : 00:05:01

In general, block coverage is an essential first step in the overall verification process.

Time(HH:MM:SS) : 00:05:07

Block coverage is the simplest and easiest visualize metric for measuring code coverage.

Time(HH:MM:SS) : 00:05:12

For effective test cases, block coverage should ideally be near 100 percent.

Time(HH:MM:SS) : 00:05:18

However, even 100 percent block coverage does not necessarily imply verification completeness.

Time(HH:MM:SS) : 00:05:24

We will see more about this in the next several slides.

Time(HH:MM:SS) : 00:05:27

Let's understand how to perform very basic block analysis in your verification metrics page.

Time(HH:MM:SS) : 00:05:33

From the verification metrics hierarchy which you saw in the previous slide, you click on any hierarchy, any level, any instance and then the associated coverage is displayed in the toolbar the type of coverage which is associated with that particular level is highlighted.

Time(HH:MM:SS) : 00:05:48

So, for example, I have selected uart_ctrl_top here and the block coverage exists for this level and all the other coverage overall, local grade, code local grade, block local grade and statement are all displayed at the top here.

Time(HH:MM:SS) : 00:06:02

Then in this page, this is the main page once you click on the block button.

Time(HH:MM:SS) : 00:06:07

Then you can see the blocks associated at that particular instance, the type of the block, the source line in which it is present in the source code.

Time(HH:MM:SS) : 00:06:15

The number of hits that are seen for this particular block that is covered or uncovered status.

Time(HH:MM:SS) : 00:06:20

In here, they all green and they are covered at least once.

Time(HH:MM:SS) : 00:06:24

So the enclosing entity is the level, the hierarchy which I selected.

Time(HH:MM:SS) : 00:06:28

Then on the right side, you can see the source code of that particular block, as in when you click on each of this blocks, the respective code source code will be displayed on the right and the block attributes associated with this block will be displayed in here at the bottom.

Time(HH:MM:SS) : 00:06:42

Expression coverage.

Time(HH:MM:SS) : 00:06:44

This provides information on why a conditional piece of code was executed.

Time(HH:MM:SS) : 00:06:49

It provides statistics for all expressions in the HDL code.

Time(HH:MM:SS) : 00:06:53

You can analyze expression coverage using the expression page of the Integrated Metrics Center GUI or the vManager GUI.

Time(HH:MM:SS) : 00:07:00

To launch the expression page from the metrics page where you see the verification metrics hierarchy, which I discussed in the previous slide while discussing the block analysis.

Time(HH:MM:SS) : 00:07:10

You select the instance or type or the level at which you want to analyze expression coverage and just click the expression icon or the button in the analyse toolbar.

Time(HH:MM:SS) : 00:07:19

Alternatively, you can right click the instance or type which you have selected in the hierarchy and go down the menu which appears and select the expression analysis button.

Time(HH:MM:SS) : 00:07:28

Now let's see how to perform a simple expression analysis.

Time(HH:MM:SS) : 00:07:32

In the verification metrics hierarchy, as you did for the block analysis, select the instance or the type and then click on the expression button in the toolbar.

Time(HH:MM:SS) : 00:07:41

Then you get to a page in this manner, the index and the expressions index are displayed in here in this window in here top level expressions.

Time(HH:MM:SS) : 00:07:49

The overall coverage grade and overall uncovered and the enclosing entity which we selected as well as the source code line in which this is present is displayed.

Time(HH:MM:SS) : 00:07:58

As in when you click on each of these expressions here, the coverage tables and what is covered and not covered is displayed in detail in that coverage tables which is SOP table, which is Sum Of Product table for that particular expression.

Time(HH:MM:SS) : 00:08:10

Then on my right side, you can see in the info table the source code, the expression hierarchy of the expression which we selected and the full expression could be displayed in here.

Time(HH:MM:SS) : 00:08:20

The coverage quality tables, How much each of these T1, T2, T3 and T4 are covered or displayed in here.

Time(HH:MM:SS) : 00:08:28

And the respective attributes for each of these expressions are displayed on the Attribute pane.

Time(HH:MM:SS) : 00:08:34

So in the expression page you can identify covered and uncovered expressions, view the SOP the Sum Of Product table for the selected expression.

Time(HH:MM:SS) : 00:08:42

View source code expression hierarchy as well as the full expression of the entity that you have selected and then also view the attributes of the selected expression.

Time(HH:MM:SS) : 00:08:50

Continuing the explanation of the expression analysis page.

Time(HH:MM:SS) : 00:08:54

You can as in any page and any windows, you could toggle floating and toggle auto hide options by clicking on these two buttons here and the expression hierarchy you can see in here.

Time(HH:MM:SS) : 00:09:05

I have highlighted that and presented the snapshot of the window and the full expression of the one we have selected here will be seen here in full expression info.

Time(HH:MM:SS) : 00:09:13

Now, let's understand a little bit more about the toggle coverage.

Time(HH:MM:SS) : 00:09:17

What does toggle coverage do?

Time(HH:MM:SS) : 00:09:19

It monitors, collects and reports a design signal toggle activity.

Time(HH:MM:SS) : 00:09:24

Now, what is a design signal toggle?

Time(HH:MM:SS) : 00:09:27

It is a binary transition that is written after a finite delay to the original position of a DUT signal.

Time(HH:MM:SS) : 00:09:33

The signals may transition through but not terminate at an unknown state.

Time(HH:MM:SS) : 00:09:38

It can be a partial, full toggle.

Time(HH:MM:SS) : 00:09:40

Now, why bother with toggle coverage?

Time(HH:MM:SS) : 00:09:44

We have to understand this is the only code coverage available for a gate level netlist.

Time(HH:MM:SS) : 00:09:49

It verifies that design interconnect is connected and wiggles.

Time(HH:MM:SS) : 00:09:53

Now I will explain a little bit more about the toggle coverage.

Time(HH:MM:SS) : 00:09:56

As I mentioned earlier, toggle coverage monitors, collects and reports the signal toggle activity.

Time(HH:MM:SS) : 00:10:03

Toggle coverage does not apply to variables such as integers or the real numbers.

Time(HH:MM:SS) : 00:10:08

You can also optionally exclude module ports.

Time(HH:MM:SS) : 00:10:11

Integrated comprehensive coverage collects and report signal activity in the VHDL and Verilog portions of the design.

Time(HH:MM:SS) : 00:10:18

The simulator normally record 0-to-1 and 1-to-0 transitions and the reporting tool reports as full toggle the 0-to-1, then 1-to-0 back to 0, as well as 1-to-0 then 0-to-1 back transitions that pass the glitch filter and reports other transitions as partial toggles that is 0-to-1 and 1-to-0 are partial.

Time(HH:MM:SS) : 00:10:38

Full is 0-to-1 then back to 0 or 1-to-0 then back to 1 is considered full.

Time(HH:MM:SS) : 00:10:44

With the set-toggle-include-x configuration command that is set-toggle-include-x.

Time(HH:MM:SS) : 00:10:49

The simulator also records unknown to one and unknown to zero that is X-to-1 and X-to-0 transitions and with the set toggle includez configuration command the simulator also record z-to-1 and z-to-0 transitions that is unknown to 1 on a unknown to 0 transitions.

Time(HH:MM:SS) : 00:11:06

Signals must endure for at least the strobe interval.

Time(HH:MM:SS) : 00:11:09

The strobe interval is by default the simulation time precision.

Time(HH:MM:SS) : 00:11:13

To filter wider glitches, you can use the coverage configuration command to set a wider strobe signal.

Time(HH:MM:SS) : 00:11:19

Toggle coverage is difficult to interpret but it is the only code coverage available for the gate level netlist.

Time(HH:MM:SS) : 00:11:26

It verifies that the design interconnect is driven and exercised that is, it is wiggled.

Time(HH:MM:SS) : 00:11:31

In the verification hierarchy page, you can click on any hierarchy and if there is toggle coverage associated with it then the toggle button will highlight.

Time(HH:MM:SS) : 00:11:40

You can click on that and you could come to this toggle analysis page.

Time(HH:MM:SS) : 00:11:44

The variables and the signals associated with the particular hierarchy are listed out here and whatever is covered and uncovered are listed.

Time(HH:MM:SS) : 00:11:51

The overall average grade and overall covered is listed here and on the right side.

Time(HH:MM:SS) : 00:11:56

The source is also the source code could also be viewed by clicking on each of these variables or signals and for each of the clicked entity.

Time(HH:MM:SS) : 00:12:03

For example, I have clicked on address variable here and you could see the associated toggle related attributes of that particular entity.

Time(HH:MM:SS) : 00:12:11

So this is how you could perform a very basic toggle analysis and using this page, you can identify covered, uncovered signals and variables.

Time(HH:MM:SS) : 00:12:20

You could also view source code and view attributes of the selected signal or variable.

Time(HH:MM:SS) : 00:12:25

What toggle constructs are covered in calculating the overall coverage?

Time(HH:MM:SS) : 00:12:29

So there is a table here for SystemVerilog and VHDL.

Time(HH:MM:SS) : 00:12:33

The scopes are given in the first row and the types of each language are given in the second row.

Time(HH:MM:SS) : 00:12:38

The toggle coverage by default scores 0-to-1 and 1-to-0 transitions of Verilog nets and variables and ports of bits and logic types, including vectors and structures of those types.

Time(HH:MM:SS) : 00:12:49

VHDL signals and ports of bit and logic types, including vectors and records of these types.

Time(HH:MM:SS) : 00:12:56

You can optionally elect to also score transitions originating in high-impedance or unknown states.

Time(HH:MM:SS) : 00:13:02

You can also optionally elect to also score SystemVerilog static arrays of bit and logic types up to a maximum number of bits.

Time(HH:MM:SS) : 00:13:10

The maximum is by default 2 raised to the 12th power (4096) and you can change that 2 between 2 raised to the 10th power and 2 raised to the 24th power.

Time(HH:MM:SS) : 00:13:18

You can provide a file of items to exclude from scoring and SystemVerilog static arrays are covered only with set_toggle_scoring, objects within generate scopes are not covered.

Time(HH:MM:SS) : 00:13:29

Type-parameterized objects are not covered.

Time(HH:MM:SS) : 00:13:32

Value-parameterized objects are covered only with the setting of the statement set_parameterized_module_coverage Finite state machine coverage.

Time(HH:MM:SS) : 00:13:40

What is this FSM coverage?

Time(HH:MM:SS) : 00:13:43

This is the coverage that interprets the synthesis semantics of the HDL design.

Time(HH:MM:SS) : 00:13:48

It monitors the coverage of the FSM representation of control logic blocks in your design.

Time(HH:MM:SS) : 00:13:54

It answers the question, Did I reach all of the states and cover all possible transitions or arcs in a given state machine.

Time(HH:MM:SS) : 00:14:01

To analyze FSM coverage in vManager tool.

Time(HH:MM:SS) : 00:14:05

You should navigate through the design hierarchy on the metrics page and identify overall coverage of different state missions in the loaded run.

Time(HH:MM:SS) : 00:14:13

You could launch the FSM page to perform a detail analysis of different state missions.

Time(HH:MM:SS) : 00:14:18

To launch the FSM coverage analysis page in your vManager tool.

Time(HH:MM:SS) : 00:14:23

You have to select the instance or type in the verification hierarchy tree and then you might have to click the FSM icon in the Analyze toolbar.

Time(HH:MM:SS) : 00:14:31

Alternatively, in the level which you have clicked, you can right click and then a pull-down menu opens up and you can select the FSM analysis from this pull-down menu.

Time(HH:MM:SS) : 00:14:40

On the FSM page, you can view the list of FSMs as you can see here.

Time(HH:MM:SS) : 00:14:45

From this list, you can identify the FSM that you want to analyze in detail and then improve its coverage that is, select one of this for further analysis.

Time(HH:MM:SS) : 00:14:54

You can view the covered and uncovered states and transitions as you can see here and view the bubble diagram wherein it shows the transitions between the states and then the view the underlying source code are conditions as well as attributes as you can see here in the details pane you can see the arc conditions, the attributes as well as the source code.

Time(HH:MM:SS) : 00:15:12

So this is how you can perform a very basic FSM coverage analysis in the vManager tool.

Time(HH:MM:SS) : 00:15:18

Also, note that the local button on the locality toolbar allows you to change the rollup type.

Time(HH:MM:SS) : 00:15:24

Local is the default which shows state registers in the selected instance.

Time(HH:MM:SS) : 00:15:29

When you click local, the button is named as recursive and which allows you to display state registers within all the children of the selected instance.

Time(HH:MM:SS) : 00:15:37

If the button shows disabled, it indicates that you cannot change the roll-up type of the selected item.

Time(HH:MM:SS) : 00:15:43

For more detailed information, you could refer to the FSM coverage section in the cdnshelp documentation.

VideoTitle :Coverage Analysis FlowChart

Time(HH:MM:SS) : 00:00:00

This slide, and the next few slides explain the different types of functional coverage and how to perform their analysis in the Vmanager tool.

Time(HH:MM:SS) : 00:00:07

The different types of functional coverage are, as we have dealt with in the beginning of this chapter.

Time(HH:MM:SS) : 00:00:12

Assertion coverage as well as cover group coverage.

Time(HH:MM:SS) : 00:00:15

Functional coverage focuses on design functions such as data values and control sequences.

Time(HH:MM:SS) : 00:00:21

For functional coverage, the tool cannot understand the design functionality, so the user instruments the design to instrument the design.

Time(HH:MM:SS) : 00:00:30

It is literally more difficult, but to then understand the functional coverage metrics is more easier.

Time(HH:MM:SS) : 00:00:35

Assertion coverage is an extension of assertion based verification and identifies interesting functions directly.

Time(HH:MM:SS) : 00:00:42

Assertion coverage points are specified using PSL or SVA.

Time(HH:MM:SS) : 00:00:46

Assert, assume and cover directives.

Time(HH:MM:SS) : 00:00:49

The coverage to be measured is directly specified using the PSL or SVA statements, or is interpreted from them to analyze assertion coverage in Vmanager tool, you need to navigate through the design hierarchy on the metrics page and identify overall coverage of different assertions in the loaded run.

Time(HH:MM:SS) : 00:01:07

You could launch the assertion page.

Time(HH:MM:SS) : 00:01:09

to perform a detailed analysis of different assertions.

Time(HH:MM:SS) : 00:01:12

To get to the assertion analysis page in the vManager tool, you have to select one of the top level instances in their hierarchy tree, and then select the assertions tab in the toolbar, and also select the recursive checkbox as, which shows up on the right side of the verification metrics page.

Time(HH:MM:SS) : 00:01:28

Then you will get to this Assertion Analysis page here, which shows the list of all assertions for that particular hierarchy which you selected.

Time(HH:MM:SS) : 00:01:35

As you can see here, you can see the list of all the assertions and overall average grade, assertion, status, grade and enclosing entity, which is the hierarchy level which you selected according to the respective assertion.

Time(HH:MM:SS) : 00:01:46

It will show up here.

Time(HH:MM:SS) : 00:01:48

You could also view the underlying source code in here in the source pane here.

Time(HH:MM:SS) : 00:01:52

For all the assertions, whichever you click on the respective source code will open up showing the assertion in the source code.

Time(HH:MM:SS) : 00:01:58

And also you can view the attributes of the selected assertions in the attributes pane in here.

Time(HH:MM:SS) : 00:02:04

Note that the Show Local button on the locality toolbar in here allows you to change the roll up time.

Time(HH:MM:SS) : 00:02:10

Local is the default, which shows assertions in the selected instance.

Time(HH:MM:SS) : 00:02:14

When you click Show Local, the button is named as recursive, which allows you to display assertions within all the children of the selected instance that is the hierarchy.

Time(HH:MM:SS) : 00:02:22

Vice.

Time(HH:MM:SS) : 00:02:23

It will be displayed if the button shows disable.

Time(HH:MM:SS) : 00:02:26

It indicates that you cannot change the roll up type of the selected item.

Time(HH:MM:SS) : 00:02:30

What is cover group coverage?

Time(HH:MM:SS) : 00:02:33

This focuses on tracking data values.

Time(HH:MM:SS) : 00:02:36

It includes coverage of variable values, bin specification of sampling, and cross products.

Time(HH:MM:SS) : 00:02:42

It helps design engineers to identify untested data values or subranges.

Time(HH:MM:SS) : 00:02:47

Cover group coverage is specified using systemverilog constructs to analyze cover group coverage in the Vmanager tool.

Time(HH:MM:SS) : 00:02:55

You should navigate through the design hierarchy on the metrics page and identify overall coverage of different cover groups in the loaded run.

Time(HH:MM:SS) : 00:03:03

You could launch the Cover Groups page to perform a detailed analysis of different cover groups.

Time(HH:MM:SS) : 00:03:08

If you would like to perform the coverage analysis at a particular hierarchy in your verification metrics, then all you have to do is select the hierarchy level and click on the cover Group button in the toolbar or select the hierarchy level.

Time(HH:MM:SS) : 00:03:20

Right click with your mouse.

Time(HH:MM:SS) : 00:03:22

A pull down menu will open.

Time(HH:MM:SS) : 00:03:24

Then select Coverage Analysis from that pull down menu.

Time(HH:MM:SS) : 00:03:28

You get to a page in this manner with the cover groups listed associated with a particular hierarchy which you have selected.

Time(HH:MM:SS) : 00:03:35

And then if you click on one of these covered groups, all the cover items associated with the cover group are displayed in this pane here.

Time(HH:MM:SS) : 00:03:42

And if you double click on this cover item, the associated bins cover bins are displayed in this pane.

Time(HH:MM:SS) : 00:03:48

There are two tabs here Abstract and Expand.

Time(HH:MM:SS) : 00:03:51

Expand version will show you all the bins listed out, and abstract will give you an abstract summary of all the coverage space, number of valid bins, covered bins.

Time(HH:MM:SS) : 00:04:00

Uncovered bins, the total information.

Time(HH:MM:SS) : 00:04:03

And if you click on any of this bins here, you can see the source code in here.

Time(HH:MM:SS) : 00:04:07

And if the cover group is covered as a cover point, then it is green in color.

Time(HH:MM:SS) : 00:04:11

If it is not green then it is uncovered.

Time(HH:MM:SS) : 00:04:14

Cover point.

Time(HH:MM:SS) : 00:04:14

As you can see in this source code.

Time(HH:MM:SS) : 00:04:16

In this way, you can perform a very basic coverage analysis to the bins level by using the Vmanager tool.

VideoTitle :What is Coverage

Time(HH:MM:SS) : 00:00:01

Integrated Metrics Center Tool, or IMC.

Time(HH:MM:SS) : 00:00:05

Imc is a metric analysis tool that provides you an interactive way to evaluate both code as well as functional coverage.

Time(HH:MM:SS) : 00:00:12

The way the flow goes for IMC tool is you need to first generate the cover work directory, which contains all the coverage information by including the command line option hyphen coverage with the Xrun command.

Time(HH:MM:SS) : 00:00:25

Then you invoke the IMC tool by clicking the coverage symbol button from the console window of SimVision and tool.

Time(HH:MM:SS) : 00:00:32

You can then navigate through the tool options to analyze the coverage attached to your design.

Time(HH:MM:SS) : 00:00:40

The first step is to create the CoV underscore work directory which contains all the coverage information.

Time(HH:MM:SS) : 00:00:46

I have a simple counter example in my setup in this directory, and this is the command I give xrun counter dot v and counter underscore test dot v, which are the design files and the testbench files, and they provide the hyphen access option for read write connectivity.

Time(HH:MM:SS) : 00:01:01

And the most important option is the hyphen coverage all.

Time(HH:MM:SS) : 00:01:05

Then I give the hyphen GUI.

Time(HH:MM:SS) : 00:01:07

option to invoke the SimVision and to relinquish the control to the SimVision and, and through the SIM vision, I'm going to invoke the IMC tool.

Time(HH:MM:SS) : 00:01:15

So you need to generate a cube underscore work directory by including the command line option.

Time(HH:MM:SS) : 00:01:20

Hyphen coverage.

Time(HH:MM:SS) : 00:01:22

That's the important step.

Time(HH:MM:SS) : 00:01:23

The first step in the SIM vision GUI.

Time(HH:MM:SS) : 00:01:26

simulation Analysis.

Time(HH:MM:SS) : 00:01:27

environment SIM vision will open in this manner.

Time(HH:MM:SS) : 00:01:32

After the command given in the previous slide, the Xrun command with the hyphen GUI.

Time(HH:MM:SS) : 00:01:36

option.

Time(HH:MM:SS) : 00:01:37

Then, after the initial simulation snapshot is created, the control is relinquished to the SIM vision tool.

Time(HH:MM:SS) : 00:01:43

As you can see here in my first screenshot, then the design browser as well as the console window of the SIM vision opens.

Time(HH:MM:SS) : 00:01:51

You can see the design browser, the simulator.

Time(HH:MM:SS) : 00:01:53

Under that we have the counter test, the test bench under that the instantiation of counter one and the signals that go into it are shown on the right.

Time(HH:MM:SS) : 00:02:01

The console window.

Time(HH:MM:SS) : 00:02:02

You need to click on the run to run the simulation.

Time(HH:MM:SS) : 00:02:07

After the Xrun command is given and the SIM vision tool takes over the simulation, you can let the counter run for a while, then check in your directory.

Time(HH:MM:SS) : 00:02:15

If the cov underscore work directory is created.

Time(HH:MM:SS) : 00:02:18

Now go to your console window of the SIM vision tool and click on this button for coverage analysis.

Time(HH:MM:SS) : 00:02:23

This is the basis for invoking the IMC tool and performing the coverage analysis.

Time(HH:MM:SS) : 00:02:28

Also, it generates the UCD and UCM directories inside the CoV underscore work directory, which contains all the coverage information of your design.

Time(HH:MM:SS) : 00:02:38

Then the Integrated Metrics Center tool opens up.

Time(HH:MM:SS) : 00:02:41

This flow that I am showing invokes the IMC through the SIM vision tool, the IMC tool GUI..

Time(HH:MM:SS) : 00:02:48

We will open by default.

Time(HH:MM:SS) : 00:02:50

You should see the All metrics view here.

Time(HH:MM:SS) : 00:02:52

And you can see the verification metrics hierarchy.

Time(HH:MM:SS) : 00:02:54

And on the right side you can see the relative elements.

Time(HH:MM:SS) : 00:02:57

And on the bottom the different metrics associated with whatever hierarchy I click here.

Time(HH:MM:SS) : 00:03:02

You can also select the block coverage view from the views button.

Time(HH:MM:SS) : 00:03:06

You can pull down and go to the Dynamic Views and select the Block Coverage view.

Time(HH:MM:SS) : 00:03:10

And you can see the code blocks and the related source code, as well as the different blocks associated on the right side.

Time(HH:MM:SS) : 00:03:16

Another view is the same views you can go there and pull down the menu, select the dynamic views and go to Toggle Coverage View.

Time(HH:MM:SS) : 00:03:23

In here you can see the toggle coverage and the signals associated, the source code and all the code blocks within it.

VideoTitle :Types of Code Coverage and Analysis

Time(HH:MM:SS) : 00:00:00

To summarize in this module, we recognized coverage in the verification process.

Time(HH:MM:SS) : 00:00:05

What this coverage is about the concept of coverage and identified the different kinds of coverage such as functional code.

Time(HH:MM:SS) : 00:00:12

And also we went through the finite state machine coverage.

Time(HH:MM:SS) : 00:00:15

The FSM identified the different kinds of code coverage like block, branch, expression, toggle, etc.

Time(HH:MM:SS) : 00:00:23

we also launched the IMC or the integrated Metrics tool and went through the flow for code coverage with a simple counter example.

VideoTitle :Types of Functional Coverage and Analysis

Time(HH:MM:SS) : 00:00:01

Let's have some quiz time now.

Time(HH:MM:SS) : 00:00:03

Here are three very simple questions that you could answer.

Time(HH:MM:SS) : 00:00:08

If you have gone through the slides in this module and have understood the concepts that were discussed, very simple questions.

Time(HH:MM:SS) : 00:00:15

You could refer to the notes section later, but try to answer it yourself.

Time(HH:MM:SS) : 00:00:20

And then if you cannot, you can either refer the module, slides or go through the slide notes for answers.

Time(HH:MM:SS) : 00:00:26

The first question what is a code block?

Time(HH:MM:SS) : 00:00:30

The second, what is expression coverage do?

Time(HH:MM:SS) : 00:00:34

Third question true or false?

Time(HH:MM:SS) : 00:00:36

A bus transitions multiple times from the high impedance state to the same known value, and then back to the high impedance state.

Time(HH:MM:SS) : 00:00:44

The bus bits are considered to have fully toggled.

Time(HH:MM:SS) : 00:00:48

Is it true or false?

VideoTitle :How to Perform Coverage Analysis

Time(HH:MM:SS) : 00:00:00

Apart from the module.

Time(HH:MM:SS) : 00:00:01

In this course, you can find several articles and information pages on the Integrated Metrics Center or IMC at support.cadence.com or the course website.

Time(HH:MM:SS) : 00:00:11

You can find Training Bytes integrated Metric center one stop with several references within it.

Time(HH:MM:SS) : 00:00:16

Integrated metric center full course.

Time(HH:MM:SS) : 00:00:18

Also user manuals of different versions, and you can also find app notes and several articles on IMC within the course website.

VideoTitle :Module Summary

Time(HH:MM:SS) : 00:00:00

Lab time.

Time(HH:MM:SS) : 00:00:01

In this lab, you will execute the Xrun command using the hyphen coverage option along with it to create the coverage data and model for the simple counter design given to you, and then invoke the IMC tool to analyze the associated code coverage.

Time(HH:MM:SS) : 00:00:15

So this lab is titled Code Coverage flow for a simple counter design.

Time(HH:MM:SS) : 00:00:19

So please go through the instructions given in the lab manual and the Readme in the lab package and get on to executing this lab.

Time(HH:MM:SS) : 00:00:26

All the best.

VideoTitle :Quiz

Time(HH:MM:SS) : 00:00:00

This module covers synthesis stage of the flow, where we will discuss how you can carry out the synthesis in a step by step manner.

Time(HH:MM:SS) : 00:00:07

The learning goals for this module are will set up and run the basic synthesis flow.

Time(HH:MM:SS) : 00:00:12

Will discuss how to write the synthesis outputs will also set up for design for test DFT during synthesis.

Time(HH:MM:SS) : 00:00:20

And then we'll discuss and debug.

Time(HH:MM:SS) : 00:00:21

VLOGPTPT-46 error message for the synthesis run.

VideoTitle :References

Time(HH:MM:SS) : 00:00:00

In the complete RTL to GDS two flow.

Time(HH:MM:SS) : 00:00:02

We are currently at the logic synthesis and DFT synthesis stages.

Time(HH:MM:SS) : 00:00:07

Let's check out how you can carry out the synthesis process.

VideoTitle :Lab

Time(HH:MM:SS) : 00:00:00

Let's understand what happens during the synthesis process and what it is all about.

Time(HH:MM:SS) : 00:00:04

Logic synthesis is nothing but a process of transforming HDL, that is, hardware description language into a logic circuit, which is based on some specific library, and you provide some specific constraints for the optimization in terms of optimizing for area power and the timing results.

Time(HH:MM:SS) : 00:00:21

And the logic design is a representation of the functional design of a circuit in the form of the logic, operations, arithmetic operations, control flow, etc.

Time(HH:MM:SS) : 00:00:31

and these operations include Boolean and or Xrun and.

Time(HH:MM:SS) : 00:00:37

In short, logic synthesis means translation, mapping, and optimization translation of the HDL into the generic Boolean components.

Time(HH:MM:SS) : 00:00:46

That is, when it reads the HDL file, and then it maps to the target technology library, which has the standard cells as well as you optimize the design for the required constraints for area power and timing.

VideoTitle :The Synthesis Stage

Time(HH:MM:SS) : 00:00:00

So what is stylus common user interface that is stylus see how we the stylus is actually efficiency.

Time(HH:MM:SS) : 00:00:07

through unification.

Time(HH:MM:SS) : 00:00:08

Stylus common user interface that is stylus quite has been designed to use across various cadence tools like Genus, Modus test., Jules Innovus, Tempus, Voltus, etc.

Time(HH:MM:SS) : 00:00:21

so by providing a common interface from RTL to sign off, the common UI enhances the user experience by making it easier to work with cadence multiple products.

Time(HH:MM:SS) : 00:00:31

The UI simplifies command naming and aligns common implementation method across cadence, digital and signoff tools.

Time(HH:MM:SS) : 00:00:38

For example, we have the process of design, initialization, database access, command consistency, and metric collection and they all have been streamlined and simplified.

Time(HH:MM:SS) : 00:00:49

Not only this, in addition, you have the updated and shared method which has been added to run defined and deployed reference flows in common UI databases with set DB and get DB for Genus Synthesis Solution.

Time(HH:MM:SS) : 00:01:02

The default working mode is stylus common UI.

Time(HH:MM:SS) : 00:01:06

So with this common user interface, you have consistent commands across the whole digital flow, which helps in improving the usability and productivity.

Time(HH:MM:SS) : 00:01:15

Stylus unified metrics, that is, for capturing and reporting, which gives holistic metrics from synthesis to sign off and stylus flow Kit is for design, flow capture, and deployment.

Time(HH:MM:SS) : 00:01:26

If you talk about the common user interface, it has been updated and unification has been done for all aspects of the user experience across various tools like how you can access the database using set DB and get DB, you can access the database across various tools.

Time(HH:MM:SS) : 00:01:42

Setting up the tools using init and MMC flow various commands and attributes.

Time(HH:MM:SS) : 00:01:48

They are also common across the tools.

Time(HH:MM:SS) : 00:01:50

How we are driving the tools like what are the startup files and reference flows that are to be used, as well as how you can access the result and analyze the result.

Time(HH:MM:SS) : 00:02:00

So you have the command reporting format and styles.

Time(HH:MM:SS) : 00:02:03

And not only this, you have the common GUI across various tools, which is helpful for analyzing the results.

Time(HH:MM:SS) : 00:02:09

Let's understand some of the limitations which are associated with unique user interface across various tools.

Time(HH:MM:SS) : 00:02:16

If you're going with unique user interfacing, that is every tool behaves differently.

Time(HH:MM:SS) : 00:02:20

Every tool has a different approach of accessing the database and running the flow.

Time(HH:MM:SS) : 00:02:24

So with that case, you need to have multiple learning curves because you need to learn about the various tools that how it works.

Time(HH:MM:SS) : 00:02:32

At the same time, you need to duplicate the flow development needs as well, because the same command cannot

be applied to the other tool.

Time(HH:MM:SS) : 00:02:39

For example, if we talk about Genus, Innovus, and tempests, the way you find instances in Genus.

Time(HH:MM:SS) : 00:02:45

is different.

Time(HH:MM:SS) : 00:02:46

You need to use find command in Innovus.

Time(HH:MM:SS) : 00:02:48

Dbget is the command, and in Tempest you need to use get cells.

Time(HH:MM:SS) : 00:02:52

So all the three tools have a different approach to access the database, and you cannot use one command in another tool.

Time(HH:MM:SS) : 00:02:59

So what's the solution?

Time(HH:MM:SS) : 00:03:01

The solution is with the common DB access.

Time(HH:MM:SS) : 00:03:04

That is the common DB command for accessing library data, logical connectivity, physical geometries, timing and power results.

Time(HH:MM:SS) : 00:03:12

And hence you can use the same command across the tools for accessing that database.

Time(HH:MM:SS) : 00:03:17

For our example here, if you want to find the instances across various tools, it's simple through common database access, you simply have to write get db instance and the instances you are looking for.

VideoTitle :Module Objectives

Time(HH:MM:SS) : 00:00:01

If you are working in the Genus stylus common UI mode, then it uses the same startup scheme as other cadence tools that use common UI, such as Innovus and Tempus timing.

Time(HH:MM:SS) : 00:00:11

Genus looks for a Genus tool file, which is the initialization file for setup information, and it's going to look for these files in the different directories with a specific preference.

Time(HH:MM:SS) : 00:00:21

First, it is going to search in the installation root directory that is under etc.

Time(HH:MM:SS) : 00:00:26

synth.

Time(HH:MM:SS) : 00:00:27

You can find this startup file.

Time(HH:MM:SS) : 00:00:29

Then it's going to look for the cadence directory in the home directory.

Time(HH:MM:SS) : 00:00:33

Then in the cadence directory in the current directory.

Time(HH:MM:SS) : 00:00:36

So if this file Genus.

Time(HH:MM:SS) : 00:00:38

underscore startup dot tickle, it is present in the current design directory.

Time(HH:MM:SS) : 00:00:42

It contains project specific information.

Time(HH:MM:SS) : 00:00:45

If GUI.

Time(HH:MM:SS) : 00:00:46

is enabled, then the cadence directory in the home directory also has the GUI.

Time(HH:MM:SS) : 00:00:50

tickle file.

Time(HH:MM:SS) : 00:00:52

If the startup file Genus.

Time(HH:MM:SS) : 00:00:53

dot tickle is present at all the locations, then all encountered files will be read.

Time(HH:MM:SS) : 00:01:02

Let's check out how you can invoke Genus Synthesis Solution.

Time(HH:MM:SS) : 00:01:06

So once you have installed the required binaries of the tool, you can start Genus.

Time(HH:MM:SS) : 00:01:11

by using command Genus.

Time(HH:MM:SS) : 00:01:12

and specify the required option.

Time(HH:MM:SS) : 00:01:15

Say if you want to specify the LEC startup file, or do you want to start in batch mode, or you want to start in no GUI., etc..

Time(HH:MM:SS) : 00:01:23

So we are showing certain examples here for the various options.

Time(HH:MM:SS) : 00:01:27

One of the common options is the files that you are specifying, the file that is to be run in the Genus session during the startup string, say Genus.

Time(HH:MM:SS) : 00:01:35

hyphen f Run times TCL.

Time(HH:MM:SS) : 00:01:38

You can also specify the log file name if you want.

Time(HH:MM:SS) : 00:01:41

Say for this case you are mentioning Genus.

Time(HH:MM:SS) : 00:01:43

run.

Time(HH:MM:SS) : 00:01:44

So when the command file is generated, it also uses the same prefix as the log file.

Time(HH:MM:SS) : 00:01:50

Or if you want to specify the different name for the command file, you can pair it along with the log option.

Time(HH:MM:SS) : 00:01:55

Say.

Time(HH:MM:SS) : 00:01:56

In the example we are showing Genus.

Time(HH:MM:SS) : 00:01:57

hyphen f, you are specifying that TCL script hyphen log, and in the quotes you are specifying, run top dot log and run top test as the command file name.

Time(HH:MM:SS) : 00:02:08

Once the Genus.

Time(HH:MM:SS) : 00:02:09

starts, you can run a script by using the source command and specifying the proper path for the script file.

Time(HH:MM:SS) : 00:02:15

You can also set the search path so that software can locate the files that Genus uses.

Time(HH:MM:SS) : 00:02:20

You can set the script search path using the Attribute.

Time(HH:MM:SS) : 00:02:23

set db script search path, and then you can simply source the script file name to exit the Genus Synthesis Solution shell, you can use quit or exit to close the Genus..

Time(HH:MM:SS) : 00:02:33

If you are not writing out your data information to the disk, Genus.

Time(HH:MM:SS) : 00:02:36

automatically doesn't save the data.

Time(HH:MM:SS) : 00:02:39

So if you are exiting without saving, that data will be lost.

Time(HH:MM:SS) : 00:02:43

So in the example, we are showing that when you write exit, you have the normal exit and you can have the status also reported using the echo command.

Time(HH:MM:SS) : 00:02:52

If you are writing exit 12 it says abnormal exit and you can have the status reported.

Time(HH:MM:SS) : 00:03:00

Genus.

Time(HH:MM:SS) : 00:03:01

stylist See how we shell does a backward compatibility with the legacy shell.

Time(HH:MM:SS) : 00:03:05

If we want to execute any legacy command in Genus stylist C you are run, you simply use your rival legacy in the brackets where you can specify the script file or the command.

Time(HH:MM:SS) : 00:03:16

Similarly, if you want to execute any stylist see how we command in the Genus legacy shell, you can use the eval common command to run the command in legacy mode.

Time(HH:MM:SS) : 00:03:25

With the help of this command is common UI mode.

Time(HH:MM:SS) : 00:03:28

You can detect active interface mode, so if you run this command in Genus shell, if it returns one, it shows that you are in stylus c UI shell.

Time(HH:MM:SS) : 00:03:37

If it returns zero or false value, it shows that you are in the legacy mode.

VideoTitle :Synthesis Flowchart

Time(HH:MM:SS) : 00:00:00

So we are showing here virtual directory structure of Genus Synthesis Solution which contains information about objects and their attributes.

Time(HH:MM:SS) : 00:00:08

You can also find information related to libraries, HDL libraries, flows, text messages, object type, design, etc.

Time(HH:MM:SS) : 00:00:17

so the object belongs to object type like any kind of design instances Module, hierarchical instances, clock ports.

Time(HH:MM:SS) : 00:00:26

You can also get information about your timing, DFT power, etc..

Time(HH:MM:SS) : 00:00:32

So when you run synthesis, many attributes affect the synthesis as well as optimization of these object types.

Time(HH:MM:SS) : 00:00:39

So as the run goes on, you can find the change in the value of these objects and their attributes.

VideoTitle :What is Synthesis

Time(HH:MM:SS) : 00:00:00

Let's check out how you can navigate the virtual directory structure of Genus Synthesis Solution.

Time(HH:MM:SS) : 00:00:05

So in Genus, if you want to set the current directory in the design directory, you can use VCD to list the object in the current directory.

Time(HH:MM:SS) : 00:00:14

Vls is to be used if you want to change the name of an object in the design hierarchy, use rename underscore obj.

Time(HH:MM:SS) : 00:00:22

By using command v, push d, you can push the specified new target directory onto the directory stack, and by using vpop d, you can remove the topmost element of the directory stack to remove an object like clock definition may be a flow step, etc.

Time(HH:MM:SS) : 00:00:38

from the database, you can use delete underscore obj to find an object and query attribute..

Time(HH:MM:SS) : 00:00:45

You can use get underscore dv, and if you want to navigate the Unix or Linux disk from the Genus.

Time(HH:MM:SS) : 00:00:51

shell, you can use the same Unix and Linux command like pwd to show the current Unix or Linux path CD, which changes the current disk directory in LZ to list the content of the Unix or Linux shell.

Time(HH:MM:SS) : 00:01:03

So we are showing the example here.

Time(HH:MM:SS) : 00:01:05

For example, you want to rename a design test two as test three.

Time(HH:MM:SS) : 00:01:09

So you can use rename underscore obj test two.

Time(HH:MM:SS) : 00:01:12

Test three using delete underscore obj.

Time(HH:MM:SS) : 00:01:16

When you are finding it in the clock say get underscore db clock specific clock.

Time(HH:MM:SS) : 00:01:20

So this is going to remove the clock objects.

Time(HH:MM:SS) : 00:01:23

And if you want to report the lib cells you can use vls hyphen hyphen A and get the lib cells.

VideoTitle :What is Stylus Common UI

Time(HH:MM:SS) : 00:00:01

In Genus we have predefined attributes which are associated with various objects in the database.

Time(HH:MM:SS) : 00:00:07

So if you want to assign value to read write attributes, you can use command set db.

Time(HH:MM:SS) : 00:00:12

The object on which you want to set the attribute name and then value.

Time(HH:MM:SS) : 00:00:16

So out of the attributes some of the attributes are read only and some are read write.

Time(HH:MM:SS) : 00:00:21

Also what value you are setting on the attribute or when you want to set the attribute.

Time(HH:MM:SS) : 00:00:25

These are dependent on the stage of the synthesis flow.

Time(HH:MM:SS) : 00:00:29

In some cases, the object type of an attribute determines the stage in the synthesis flow at which the attribute can be set.

Time(HH:MM:SS) : 00:00:36

Some attributes can be set only at the root level.

Time(HH:MM:SS) : 00:00:39

Some attributes are required to set before and after elaboration, some after synthesis or before synthesis.

Time(HH:MM:SS) : 00:00:46

So depending upon the stage of the synthesis flow, you can set the attribute for the root attribute.

Time(HH:MM:SS) : 00:00:52

There are two ways through which you can specify it.

Time(HH:MM:SS) : 00:00:55

Firstly, you can use set db dot attribute name value, or you can simply abbreviate it as set db attribute name and value.

Time(HH:MM:SS) : 00:01:04

We are showing an example here related to the setting of the attribute at root level first.

Time(HH:MM:SS) : 00:01:09

So we are simply writing set db lp insert Clock gating is true.

Time(HH:MM:SS) : 00:01:15

But when you want to set the design attribute, you need to use the syntax set db.

Time(HH:MM:SS) : 00:01:19

Then you have the object that is the design here.

Time(HH:MM:SS) : 00:01:22

Design top Module,.

Time(HH:MM:SS) : 00:01:23

Then the attribute.

Time(HH:MM:SS) : 00:01:24

name that is lp Clock gating exclude and then the value that is true.

Time(HH:MM:SS) : 00:01:28

This command set db also returns a pair, showing how many times it was executed and values assigned.

Time(HH:MM:SS) : 00:01:35

So in the example we are showing, say we are setting set my variable as ten and when you are going to execute command set db hyphen quiet get db messages max print dollar my variable.

Time(HH:MM:SS) : 00:01:46

It is going to return how many times it was executed and the value assigned.

Time(HH:MM:SS) : 00:01:53

In Genus you can query Attribute.

Time(HH:MM:SS) : 00:01:56

using command get db.

Time(HH:MM:SS) : 00:01:58

If you want to retrieve the value of an attribute, you can use get db.

Time(HH:MM:SS) : 00:02:01

Specify the object and the attribute name with a dot.

Time(HH:MM:SS) : 00:02:05

This command works on a single object and as well as a list of the object.

Time(HH:MM:SS) : 00:02:09

In the example we are showing, first you want to get the attribute value for tns opto, so you can simply use get dv tns opto.

Time(HH:MM:SS) : 00:02:17

Then you want to get the value of the sources for the particular clock.

Time(HH:MM:SS) : 00:02:20

So we specify get db clock and then dot sources.

Time(HH:MM:SS) : 00:02:25

In the result.

Time(HH:MM:SS) : 00:02:26

You can see that what is the source for this particular clock.

Time(HH:MM:SS) : 00:02:29

One other possibility to retrieve the attribute value of multiple objects is to embed the command using TCL.

Time(HH:MM:SS) : 00:02:36

This is also possible in Genus say in the example we are showing get db.

Time(HH:MM:SS) : 00:02:41

You have this pin dot gated clocks.

VideoTitle :Start and Exit Genus

Time(HH:MM:SS) : 00:00:01

Let's check out some of the properties of this DB alignment.

Time(HH:MM:SS) : 00:00:05

So this get underscore db and set underscore db.

Time(HH:MM:SS) : 00:00:09

It is capabilities to the find filter get underscore attribute set underscore attribute of RC and Genus.

Time(HH:MM:SS) : 00:00:15

legacy as well as get underscore db, get underscore property user attributes or others of Innovus and Tempus timing.

Time(HH:MM:SS) : 00:00:23

Not only this, you have the chaining and integrated filter present for better performance, and as well as various options to filter and define your search using like foreach, invert if unique regular expression, match, hierarchical or index.

Time(HH:MM:SS) : 00:00:42

Genus.

Time(HH:MM:SS) : 00:00:43

also offers tab completion for the commands if the command is unique, and if the command is not unique, it's going to give a message that this is the ambiguous command and going to list out the command which starts with the name that you had specified, say.

Time(HH:MM:SS) : 00:00:56

In the example we are showing, define, underscore scan.

Time(HH:MM:SS) : 00:00:59

So this is an ambiguous command for the tool and it returns the define scan has different commands.

Time(HH:MM:SS) : 00:01:05

Starting with this name define scan abstract segment, define scan chain, etc..

Time(HH:MM:SS) : 00:01:11

As we mentioned, if you're not sure about the command, and if you just type the few letters of the command, and if you press tab.

Time(HH:MM:SS) : 00:01:16

So it's going to display a list of the commands that start with those letters as well as file completion also works from the command line.

Time(HH:MM:SS) : 00:01:24

Say you are sourcing the script, you're specifying the correct script path, and you are just typing the first few letters of the script file.

Time(HH:MM:SS) : 00:01:31

Then automatically it completes if the final name is unique.

Time(HH:MM:SS) : 00:01:35

If not, you will get the list of the file names which are starting from those particular letters.

VideoTitle :Genus Virtual Directory

Time(HH:MM:SS) : 00:00:01

You can also get help on all the commands by specifying the complete command name.

Time(HH:MM:SS) : 00:00:06

Say you want to get help for the read SDC.

Time(HH:MM:SS) : 00:00:08

When you specify help read underscore SDC, you get detailed information about this particular command.

Time(HH:MM:SS) : 00:00:15

If you want to view the man pages from the Unix or Linux shell, you can set the environment using set env man path cn synth route share synth man.

Time(HH:MM:SS) : 00:00:24

So basically this CN synth route points to the cadence installation of Genus Synthesis Solution and it should be included in your path.

VideoTitle :Navigating Design Hierarchy

Time(HH:MM:SS) : 00:00:01

Let's check out what our different inputs and outputs of Genus synthesis solution.

Time(HH:MM:SS) : 00:00:06

When you're running Genus., the required inputs are RTL in the form of Verilog VHDL directives Pragmas Systemverilog constraints SDC library lib or LDB format.

Time(HH:MM:SS) : 00:00:20

Optionally, you can also provide Power intent.

Time(HH:MM:SS) : 00:00:23

file in the format CPF UPF..

Time(HH:MM:SS) : 00:00:26

You can also provide the physical information in terms of the cap table, Sharc technology file and DEF and DEF.

Time(HH:MM:SS) : 00:00:33

If you talk about the outputs which are generated from Genus, it includes optimized netlist LEC, DEF file, Atpg, scan, DEF and others.

Time(HH:MM:SS) : 00:00:42

Constraints.

Time(HH:MM:SS) : 00:00:43

Physical design.

Time(HH:MM:SS) : 00:00:44

Input files.

VideoTitle :Setting and Querying Attributes

Time(HH:MM:SS) : 00:00:01

We can turn on the GUI.

Time(HH:MM:SS) : 00:00:03

by using command GUI.

Time(HH:MM:SS) : 00:00:04

underscore show.

Time(HH:MM:SS) : 00:00:06

Let's check out different components of the Genus.

Time(HH:MM:SS) : 00:00:08

common ui GUI.

Time(HH:MM:SS) : 00:00:10

The main GUI window has different components like the menu bar viewers, which include a design browser, layout viewer, schematic viewer, HDL viewer, and object attributes.

Time(HH:MM:SS) : 00:00:23

These viewer windows are dockable and you can move them around in your display area.

Time(HH:MM:SS) : 00:00:28

Then you have this layer control, which is applicable only to the layout viewer.

Time(HH:MM:SS) : 00:00:32

It allows selectivity and visibility on the various components of your physical design.

Time(HH:MM:SS) : 00:00:38

Then you have this toolbar.

Time(HH:MM:SS) : 00:00:39

Every viewer has its own toolbar.

Time(HH:MM:SS) : 00:00:42

This bar is present at the top of the viewer, and displays a collection of the icons that represent frequently used commands corresponding to the viewer.

Time(HH:MM:SS) : 00:00:51

And then you have the status bar, which is associated with the viewers.

Time(HH:MM:SS) : 00:00:54

Every viewer has its own status bar.

VideoTitle :DB Alignment Highlights

Time(HH:MM:SS) : 00:00:00

Let's check out details for the new graphical user interface using a small design here.

Time(HH:MM:SS) : 00:00:05

You can simply invoke GUI using GUI, underscore show or GUI underscore raise.

Time(HH:MM:SS) : 00:00:11

So once you write gui underscore show, you will be diverted to the GUI.

Time(HH:MM:SS) : 00:00:15

So this is our graphical user interface.

Time(HH:MM:SS) : 00:00:18

So the main GUI has the different components like menu bar viewers like design browser or your schematic viewer or your layout viewer.

Time(HH:MM:SS) : 00:00:29

Then it has the actual viewer as well.

Time(HH:MM:SS) : 00:00:31

Object attributes etc..

Time(HH:MM:SS) : 00:00:34

Layer control toolbar and status bar.

Time(HH:MM:SS) : 00:00:41

So this menu bar which is at the top, it is the horizontal bar across the top of the GUI which contains the different buckets of commands like file, DFT, floor plan, power, timing, tools, windows help, etc.

Time(HH:MM:SS) : 00:00:56

and each of these buckets has a drop down menu with associated commands like for file you can check the design report, different summary gate count, netlist statistics, data path elements, hide gear exit Similarly with the DFT have different options for checking the Violations..

Time(HH:MM:SS) : 00:01:15

for generating the scan chain, for checking the fail or pass, flip flop, etc.

Time(HH:MM:SS) : 00:01:21

floor plan.

Time(HH:MM:SS) : 00:01:22

You can check the floor plan and check the placement power.

Time(HH:MM:SS) : 00:01:25

You can report the power of your design, detailed report or the RTL power timing.

Time(HH:MM:SS) : 00:01:31

With the help of that, you can debug the timing tools where you can have the object attributes of your design, and windows where you have the workspace menus which are appearing like file, DFT, floor plan, power, timing, etc.

Time(HH:MM:SS) : 00:01:46

on your menu bar and then help.

Time(HH:MM:SS) : 00:01:48

So all other windows except the menu bar are referred as the viewers in Genus..

Time(HH:MM:SS) : 00:01:52

And all these viewers are detachable.

Time(HH:MM:SS) : 00:01:54

That is, they can be moved around freely within your display area, and this is useful feature when having an extended monitor setup, and the viewers give a peek into the design from different viewpoints.

Time(HH:MM:SS) : 00:02:05

For example, you have this layout view.

Time(HH:MM:SS) : 00:02:08

You can just click this tab plus and you can have the schematic view.

Time(HH:MM:SS) : 00:02:13

Hdl view Design Browser Object Browser Design Browser is already appearing here.

Time(HH:MM:SS) : 00:02:18

Say we click this schematic view so you will You'll have the schematic window here, which will give a schematic view of your design.

Time(HH:MM:SS) : 00:02:27

So this schematic viewer.

Time(HH:MM:SS) : 00:02:29

lets you see the design from a designer perspective.

Time(HH:MM:SS) : 00:02:31

And this layout view which appears when you run the physical synthesis.

Time(HH:MM:SS) : 00:02:36

So this gives an idea about the Di and the placement of the different modules inside the diarrhea.

Time(HH:MM:SS) : 00:02:41

So in Genus.

Time(HH:MM:SS) : 00:02:42

there are five different views to fulfill different requirements.

Time(HH:MM:SS) : 00:02:45

Namely like design browser.

Time(HH:MM:SS) : 00:02:47

You can also have say we just click the object attributes here.

Time(HH:MM:SS) : 00:02:53

So you have this object attributes.

Time(HH:MM:SS) : 00:02:57

You have HDL Viewer.

Time(HH:MM:SS) : 00:02:59

schematic Viewer.

Time(HH:MM:SS) : 00:03:01

and layout Viewer..

Time(HH:MM:SS) : 00:03:03

So each Viewer.

Time(HH:MM:SS) : 00:03:04

it has its own submenu on the top of the viewer window, and the status bar at the bottom.

Time(HH:MM:SS) : 00:03:09

So each window it has the menu at the top, say for layout you are having this toolbar.

Time(HH:MM:SS) : 00:03:16

For schematic.

Time(HH:MM:SS) : 00:03:17

We are having this toolbar for HDL view.

Time(HH:MM:SS) : 00:03:20

We are having this toolbar for object attributes.

Time(HH:MM:SS) : 00:03:23

We have this.

Time(HH:MM:SS) : 00:03:24

And similarly for the design browser, we have here this toolbar.

Time(HH:MM:SS) : 00:03:28

At the same time, at the bottom you have the status bar for the different kind of the views you can see here.

Time(HH:MM:SS) : 00:03:35

Another important part is the layer control.

Time(HH:MM:SS) : 00:03:39

So this is applicable only to the layer viewer..

Time(HH:MM:SS) : 00:03:42

It allows selectability and visibility on the various components of your physical design.

Time(HH:MM:SS) : 00:03:47

And you can even choose the colors and patterns for a specific component time as per your requirement and preference in a file, and reload the file whenever required.

Time(HH:MM:SS) : 00:03:57

And another important um part was the toolbar.

Time(HH:MM:SS) : 00:04:00

So as we have seen that each view has its own toolbar, and this toolbar is present at the top of the viewer.

Time(HH:MM:SS) : 00:04:09

So it displays a collection of the icons that represent frequently used commands corresponding to the viewers and Similarly status bar is also associated with the viewers.

Time(HH:MM:SS) : 00:04:18

Like for the different view, you can have the status appearing at the bottom of it.

Time(HH:MM:SS) : 00:04:23

So using this graphical user interface capability of Genus Synthesis Solution, you are more aligned with the Innovus UI, which is further helpful for you to debug issues which are related to your physical synthesis or timing and power by using different tabs and options which are present in GUI.

Time(HH:MM:SS) : 00:04:41

in terms of your power, in terms of debugging the timing.

VideoTitle :How to Get Help for Command in Genus Stylus CUI

Time(HH:MM:SS) : 00:00:01

Let's check out basic synthesis flow in Genus synthesis solution.

Time(HH:MM:SS) : 00:00:06

You start by reading the technology library and initial setup.

Time(HH:MM:SS) : 00:00:10

If you are running multi-mode multicore flow, then you can use read underscore MMC command to read the library and initial setup.

Time(HH:MM:SS) : 00:00:18

After this you can read the HDL files, elaborate your design and if you are going with MMC flow then you can initialize the design after this stage.

Time(HH:MM:SS) : 00:00:29

Then you set the timing and design constraints, apply the optimization directives, synthesize your design to generic technology mapping, and optimize it.

Time(HH:MM:SS) : 00:00:40

It is followed by reporting and analyzing the design.

Time(HH:MM:SS) : 00:00:43

Then you can write out the output files.

Time(HH:MM:SS) : 00:00:46

Check if you are meeting your desired constraints.

Time(HH:MM:SS) : 00:00:48

If not, then you can go back and modify the optimization directives as required.

Time(HH:MM:SS) : 00:00:54

And then you can hand off the files for Place and route.

VideoTitle :Inputs and Outputs of Genus Synthesis Solution

Time(HH:MM:SS) : 00:00:01

So one of the important settings for specifying the library is the lib search path.

Time(HH:MM:SS) : 00:00:07

So with the help of this attribute.

Time(HH:MM:SS) : 00:00:09

init lib search part, you can specify the list of Unix or Linux directories that Genus.

Time(HH:MM:SS) : 00:00:15

should search to locate the technology libraries and left files.

Time(HH:MM:SS) : 00:00:19

So you can specify this path before you read your libraries or left libraries.

VideoTitle :Genus Stylus Common UI GUI Components

Time(HH:MM:SS) : 00:00:01

To read Libraries in Genus..

Time(HH:MM:SS) : 00:00:03

You can use command read libs.

Time(HH:MM:SS) : 00:00:05

With the help of this command, it will search for libraries with a specified name.

Time(HH:MM:SS) : 00:00:10

If the libraries are not found, then the tool will look for the libraries in the library search path, and if the same file name is available at different places in the search path, only the first library which is encountered will be loaded.

Time(HH:MM:SS) : 00:00:27

To read libraries in Genus, you can use command read libs.

Time(HH:MM:SS) : 00:00:31

There are options associated with this command, which includes specifying the list of iucv libraries and CV libraries.

Time(HH:MM:SS) : 00:00:40

File names are the most common option, which specifies the list of libraries, and when you use this option, the tool will populate the minimum and maximum with the same libraries.

Time(HH:MM:SS) : 00:00:50

You can also explicitly specify the max libs and min libs using max libs and min libs option, but if you are using these options along with file names option, it will cause an error.

VideoTitle :Demo exploring stylus comman ui

Time(HH:MM:SS) : 00:00:01

Let's discuss what exactly MMC flow is.

Time(HH:MM:SS) : 00:00:05

If we talk about the single corner, that is when you have no multi-mode, no multi corner, you're just having a plain view.

Time(HH:MM:SS) : 00:00:12

You start with your library, read your HDL supply timing Constraint and synthesize with generic mapping and optimization.

Time(HH:MM:SS) : 00:00:20

When you have multi-mode multi corner this is how your flow looks like.

Time(HH:MM:SS) : 00:00:25

You need to read MMC file which contains details about your multi-mode multi corner's views etc..

Time(HH:MM:SS) : 00:00:32

Read physical information through LEF.

Time(HH:MM:SS) : 00:00:34

Read your HDL, elaborate your design, read DEF file, initialize the design using init design and then you can go with synthesis, generic mapping and optimization.

Time(HH:MM:SS) : 00:00:45

A normal and an example multi-mode file looks like this.

Time(HH:MM:SS) : 00:00:48

You have different commands and inputs related to libraries operating condition, timing condition, delay corners, constraints, modes, etc..

VideoTitle :Basic Synthesis Flow in Genus

Time(HH:MM:SS) : 00:00:01

To read multi-mode multi corner.

Time(HH:MM:SS) : 00:00:03

That is MMC view definition file and to populate the MMC related attributes use command read MMC.

Time(HH:MM:SS) : 00:00:11

You can specify option design and file name.

Time(HH:MM:SS) : 00:00:15

This file name can be specified with absolute paths in the MMC, or relative to the local directory, or to the library search path.

Time(HH:MM:SS) : 00:00:22

Attributes MMC refers to the full set of timing libraries, constraints, operating conditions, timing conditions, RC corners, delay corners, and the associated analysis.

Time(HH:MM:SS) : 00:00:35

views.

Time(HH:MM:SS) : 00:00:36

When this command is run, the MMC file is read and processed.

Time(HH:MM:SS) : 00:00:41

The majority of elements in the MMC control files are saved in the MMC caching attributes.

Time(HH:MM:SS) : 00:00:47

When this command is run, only those library files are read, which are associated with the first view defined by set Analysis.

Time(HH:MM:SS) : 00:00:54

view command in the MMC file.

Time(HH:MM:SS) : 00:00:57

At this time, no other data is loaded into the database.

Time(HH:MM:SS) : 00:01:01

This command does basic syntax checking on the MMC commands.

Time(HH:MM:SS) : 00:01:06

It rejects commands that have incorrect options and commands that are not MMC or Tcl commands.

Time(HH:MM:SS) : 00:01:12

However, no consistency checking is performed.

Time(HH:MM:SS) : 00:01:16

For example, if a Constraint mode is used in an analysis view but it is not defined, then it will not be flagged.

Time(HH:MM:SS) : 00:01:23

The example here shows snapshot of reading the MMC file, which sources the file and starts processing various commands which are defined in the MMC file.

VideoTitle :Setting the Library Search Path

Time(HH:MM:SS) : 00:00:01

HDL file contains design information which contains structural code for combining lower level modules, behavioral design specification or RTL implementation.

Time(HH:MM:SS) : 00:00:12

You can read both RTL or netlist in Genus.

Time(HH:MM:SS) : 00:00:16

For reading netlist, you can use command read_netlist to parse the source file and read_hdl loads one or more HDL files in the order given into the memory.

Time(HH:MM:SS) : 00:00:26

The example here shows how you can read RTL or mixed source design by specifying the different design files with the read_hdl command.

Time(HH:MM:SS) : 00:00:35

And you can also specify the HDL search path by using attribute init_hdl_search_path.

Time(HH:MM:SS) : 00:00:45

There are various options which are available with the read_hdl command to define a different kind of the file formats.

Time(HH:MM:SS) : 00:00:52

Various file format supported by Genus includes VHDL, SystemVerilog, Verilog 1995 2001 and you can also specify the netlist if you want.

Time(HH:MM:SS) : 00:01:05

So, if you do not specify either any option like V1995 V2001 -sv or vhdl, the default language format is the one which is specified by the HDL language attribute.

Time(HH:MM:SS) : 00:01:18

By default, the value is v2001.

VideoTitle :Reading Libraries in Genus Stylus CUI

Time(HH:MM:SS) : 00:00:01

Elaboration involves various design, checking and optimization, and it is a necessary step to proceed with synthesis.

Time(HH:MM:SS) : 00:00:08

The elaborate command can be used to elaborate design, which automatically elaborates the top level design and all of its references.

Time(HH:MM:SS) : 00:00:16

During elaboration, tool performs the task like it builds data structure, infers registers in the design, performs high level HDL optimization like dead code removal check semantics.

Time(HH:MM:SS) : 00:00:28

Identifies Clock gating and if there are any gate level netlist read within the RTL file, Genus.

Time(HH:MM:SS) : 00:00:33

automatically links the cell to their references in the technology library.

Time(HH:MM:SS) : 00:00:37

During elaboration, you do not have to issue an additional command for linking.

Time(HH:MM:SS) : 00:00:42

At the end of the elaboration, Genus.

Time(HH:MM:SS) : 00:00:45

displays any unresolved references immediately after the keyword done elaborating and also after elaboration.

Time(HH:MM:SS) : 00:00:51

Genus.

Time(HH:MM:SS) : 00:00:51

has an internally created data structure for the whole design, so you can apply constraints and perform other operations with elaborate command.

Time(HH:MM:SS) : 00:01:00

There is an optional parameter that you can use to specify the parameters for a particular module.

Time(HH:MM:SS) : 00:01:06

This parameter option will override the parameter which is already defined in the design.

Time(HH:MM:SS) : 00:01:11

Let's take the example here.

Time(HH:MM:SS) : 00:01:13

In the design top, we have defined the parameter data width one average period and data width two.

Time(HH:MM:SS) : 00:01:19

Now with this option hyphen parameter we are defining them as 12, eight and 16.

Time(HH:MM:SS) : 00:01:25

But when you elaborate your design, they will be overwritten and these new parameters will be used for elaboration.

VideoTitle :Synthesis Flow When Using Multiple Corner Libraries

Time(HH:MM:SS) : 00:00:01

When you run this command in it underscore design.

Time(HH:MM:SS) : 00:00:04

It initializes the database and ensures that the tool is ready for full execution.

Time(HH:MM:SS) : 00:00:09

When this command is run, the tool steps through the defined MMC object, the design object, and the power requirements and build a full representation of the design.

Time(HH:MM:SS) : 00:00:19

Once initialization is complete, the design is in a consistent state and is ready for manipulation, that is, for floor planning, synthesis or timing, and so on.

VideoTitle :Reading MMMC File

Time(HH:MM:SS) : 00:00:01

Once your design is elaborated, you can read the timing constraints either by using the SDC file, which is the preferred method, or you can specify Genus.

Time(HH:MM:SS) : 00:00:09

tickle constraints.

Time(HH:MM:SS) : 00:00:11

So you can read directly into Genus.

Time(HH:MM:SS) : 00:00:13

by using command read underscore SDC.

Time(HH:MM:SS) : 00:00:16

There are various options like if you want to stop on errors, no compress view, or you can specify then the SDC file name.

Time(HH:MM:SS) : 00:00:24

So it is recommended to always check for errors and failure commands.

Time(HH:MM:SS) : 00:00:28

Once you have read the SDC constraint file to check for any missing constraints or any syntax error, etc., you can review the log entries and summary at the end of the table.

Time(HH:MM:SS) : 00:00:39

So again it is recommended to run check timing intent hyphen verbose after reading the constraints to check for the constraints consistency to catch any issues related to the constraints.

Time(HH:MM:SS) : 00:00:49

The important thing to note is this command read SDC is useful only in the simple synthesis flow, but not in the MMC flow.

VideoTitle :Reading Designs in Genus

Time(HH:MM:SS) : 00:00:01

If you're using multi-mode multicore flow in Genus., then you can specify timing constraints in MMC file using command create Constraint mode for a specific mode of design.

Time(HH:MM:SS) : 00:00:12

If you are specifying the constraints in MMC file, they are automatically read during init underscore design command.

Time(HH:MM:SS) : 00:00:19

So in the example we are showing that you are defining the timing constraint for say functional mode slow.

Time(HH:MM:SS) : 00:00:25

You specify the constraint file Similarly.

Time(HH:MM:SS) : 00:00:29

For functional Whql fast mode, you are specifying the specific constraint file, and so on and so forth.

Time(HH:MM:SS) : 00:00:36

If you want to update or add any other timing constraint, once you have read the MMC file, you can do so by using command update constraint mode.

VideoTitle :Elaboration of Designs

Time(HH:MM:SS) : 00:00:01

This slide shows attributes that you can use to control the optimization of the design in terms of preserving instances and sub designs, grouping and ungrouping.

Time(HH:MM:SS) : 00:00:10

Optimizing and merging sequential logic.

Time(HH:MM:SS) : 00:00:13

Please take a moment to go through details.

VideoTitle :Initializing The Database,

Time(HH:MM:SS) : 00:00:01

Synthesis is the process of transforming your HDL or RTL into a gate level netlist.

Time(HH:MM:SS) : 00:00:07

You provide the specified constraints and optimization settings and try to meet the design's power, timing, and area results.

Time(HH:MM:SS) : 00:00:15

There are three different levels of synthesis.

Time(HH:MM:SS) : 00:00:18

One is generic, which does generic gate optimization.

Time(HH:MM:SS) : 00:00:22

To do so, you use the command syn underscore generic.

Time(HH:MM:SS) : 00:00:26

The second is the mapping stage, which maps your design or RTL to the specified technology library.

Time(HH:MM:SS) : 00:00:32

It is achieved by using the command syn underscore map.

Time(HH:MM:SS) : 00:00:36

And third is the optimization which is synthesized to the optimized gates.

Time(HH:MM:SS) : 00:00:41

To run this stage, use the command syn underscore opt options.

Time(HH:MM:SS) : 00:00:45

Hyphen, physical and spatial are used to run the physical synthesis in Genus., and these require Genus.

Time(HH:MM:SS) : 00:00:51

physical license to run.

Time(HH:MM:SS) : 00:00:53

Let's take the example code.

Time(HH:MM:SS) : 00:00:55

Here a generic gate is used for the instance count reg zero during the generic synthesis.

Time(HH:MM:SS) : 00:01:01

During the mapping stage, tool maps the generic gate with std cell dfrac one.

Time(HH:MM:SS) : 00:01:07

For instance, count reg zero and optimization stage optimizes the cell dfrac one to DFCs one for the instance count reg zero.

VideoTitle :Specifying Timing Constraints in Genus Stylus CUI

Time(HH:MM:SS) : 00:00:02

Before you run command for synthesis in underscore gen sin underscore map sin underscore opt, you need to specify effort level for carrying out a synthesis.

Time(HH:MM:SS) : 00:00:12

You can set effort level for synthesis command by using attributes in underscore generic underscore effort for sin generic command sin underscore map underscore effort for mapping stage command and sin underscore opt underscore effort for your sin opt command.

Time(HH:MM:SS) : 00:00:27

You can also specify global effort, which is to be used during all synthesis command that is, sin.

Time(HH:MM:SS) : 00:00:32

Underscore generic map and optimization by using attributes in underscore global underscore effort, and you can set it to different values.

Time(HH:MM:SS) : 00:00:40

If we talk about the default values for the different attributes for generic, it is medium for mapping and optimization.

Time(HH:MM:SS) : 00:00:46

That is sin map effort and sin opt effort.

Time(HH:MM:SS) : 00:00:49

It is high and none for the sin global effort.

Time(HH:MM:SS) : 00:00:52

You can use option Express to enable express flow both for logical and physical synthesis.

Time(HH:MM:SS) : 00:00:57

This express flow enables early feasibility analysis with much faster run times and reasonable quality of results.

VideoTitle :Specifying Design Constraints Through the MMMC File

Time(HH:MM:SS) : 00:00:01

Once your design is synthesized and optimized, you can analyze and generate different kinds of the reports in terms of area timing, power instances, memory messages, etc.

Time(HH:MM:SS) : 00:00:13

you can use different commands and generate these reports like report, underscore, area.

Time(HH:MM:SS) : 00:00:17

Provide detail about the area of the design report.

Time(HH:MM:SS) : 00:00:20

Underscore DPI prints Datapath resources.

Time(HH:MM:SS) : 00:00:23

It provides detail about the different Datapath elements.

Time(HH:MM:SS) : 00:00:26

Report underscore design.

Time(HH:MM:SS) : 00:00:28

Underscore rules.

Time(HH:MM:SS) : 00:00:28

Shares details about design rule violation report underscore gate provides information about the lib cell, total area,

instant count summary, etc..

Time(HH:MM:SS) : 00:00:38

Report underscore hierarchy provides detail about the report of the hierarchy.

Time(HH:MM:SS) : 00:00:43

Report.

Time(HH:MM:SS) : 00:00:43

Underscore instance generates a report on the instance of the current design report.

Time(HH:MM:SS) : 00:00:48

Underscore memory.

Time(HH:MM:SS) : 00:00:49

Prints the memory usage report.

Time(HH:MM:SS) : 00:00:51

Underscore messages.

Time(HH:MM:SS) : 00:00:52

Provides detail about the messages.

Time(HH:MM:SS) : 00:00:54

It could be error message, warning message, or information you're looking for.

Time(HH:MM:SS) : 00:00:58

Report underscore power provides and generate detailed report of the power as per the option that you are supplying in the command report.

Time(HH:MM:SS) : 00:01:05

Underscore queue or generates quality of result report.

Time(HH:MM:SS) : 00:01:08

Report.

Time(HH:MM:SS) : 00:01:08

Underscore timing.

Time(HH:MM:SS) : 00:01:09

Prints a timing report as per the different options that you can give with this command and report.

Time(HH:MM:SS) : 00:01:14

Underscore summary prints area timing and design rule.

Time(HH:MM:SS) : 00:01:18

Report summary.

VideoTitle :Controlling Optimization

Time(HH:MM:SS) : 00:00:01

To generate timing report in Genus.

Time(HH:MM:SS) : 00:00:03

you can use command report underscore timing.

Time(HH:MM:SS) : 00:00:06

You can further filter out the reporting as per various options.

Time(HH:MM:SS) : 00:00:10

Say if we want to generate timing report between some specific points, you can specify hyphen from and hyphen to.

Time(HH:MM:SS) : 00:00:16

You can further filter report as per the exceptions or path type, etc..

Time(HH:MM:SS) : 00:00:21

In the example we are showing, if we want to generate report from all inputs to all outputs, you can specify report underscore timing hyphen from all inputs to all outputs.

Time(HH:MM:SS) : 00:00:31

Similarly, we can generate timing report for all register to register path using hyphen from and hyphen to option as all underscore registers.

Time(HH:MM:SS) : 00:00:40

If you want, you can generate the timing report among path from certain clock domain to another clock domain.

Time(HH:MM:SS) : 00:00:45

You can do so by using command report underscore timing hyphen from particular clock domain to another clock.

Time(HH:MM:SS) : 00:00:57

Let's check out once you have generated a timing report of how it looks like and what all information you can get from it in the timing report at the top, you can find the details about the Module, operating condition, interconnect mode and area mode.

Time(HH:MM:SS) : 00:01:12

Then in the body you can get information about the timing slack calculation, which gives information whether the slack has met or not, the group start point clock, end point, clock, etc.

Time(HH:MM:SS) : 00:01:23

and then you have the information regarding the calculation.

Time(HH:MM:SS) : 00:01:27

And then the table shows the arrival time calculation where you can get the timing point, the flags like congested area, ideal net preserved and so on, arc detail, edge cell name, fanout, load delay, arrival, etc..

Time(HH:MM:SS) : 00:01:45

This example of timing report shows with path adjust constraints, you can find information related to your Module, operating condition, interconnect mode and area mode.

Time(HH:MM:SS) : 00:01:57

And then in the body part, you can get information regarding your arrival time calculation and the exception created by set underscore path underscore.

Time(HH:MM:SS) : 00:02:04

Adjust command.

Time(HH:MM:SS) : 00:02:06

You can see here path adjusts say -200.

Time(HH:MM:SS) : 00:02:09

And then you can get details about your load, transaction delay and arrival, etc..

VideoTitle :Exploring Synthesis Stages

Time(HH:MM:SS) : 00:00:01

Once the synthesis process is completed.

Time(HH:MM:SS) : 00:00:04

That is, once you have synthesized the design and optimized it, you can write out various output files from Genus.

Time(HH:MM:SS) : 00:00:10

One of the outputs is to generate the database file.

Time(HH:MM:SS) : 00:00:13

So with the help of this command write underscore db you can write the design to a database file.

Time(HH:MM:SS) : 00:00:19

So basically this command writes outs the netlist to the database file.

Time(HH:MM:SS) : 00:00:23

Or it returns a tickle list.

Time(HH:MM:SS) : 00:00:26

When you use this command route attributes with Non-default settings can be included in the database.

Time(HH:MM:SS) : 00:00:31

Or they are written out to a script, and by default, only route attributes affecting the show are written out.

Time(HH:MM:SS) : 00:00:37

So in the example we are showing that we are generating a database for the top design DTMF underscore recover underscore core, and we are diverting it to file designer db.

Time(HH:MM:SS) : 00:00:48

Another important file that you can generate from Genus is the netlist.

Time(HH:MM:SS) : 00:00:52

Once you have synthesized the design.

Time(HH:MM:SS) : 00:00:55

So write underscore netlist is the command that you can use to generate a gate level netlist.

Time(HH:MM:SS) : 00:01:00

You can also divert it to some file.

Time(HH:MM:SS) : 00:01:02

In the example we are showing that either you can divert it to say some TSV file, or if you are simply writing write underscore netlist command, then the netlist is printed on the Genus prompt itself.

Time(HH:MM:SS) : 00:01:21

If you want to generate the SDC Constraint file in that case, you can use the command, write SDC, specify the design, and you can divert it to some file.

Time(HH:MM:SS) : 00:01:31

In the example, we show that we are diverting the file to say some test dot SDC.

Time(HH:MM:SS) : 00:01:36

All the related constraints are written out in this.

VideoTitle :Setting Effort Levels Prior to Synthesis

Time(HH:MM:SS) : 00:00:00

Hello everyone!

Time(HH:MM:SS) : 00:00:02

In this video demonstration we are going to discuss basic synthesis flow in Genus Synthesis Solution stylus common UI us.

Time(HH:MM:SS) : 00:00:11

So once you have set the required binaries for running the Genus tool, you can simply invoke Genus.

Time(HH:MM:SS) : 00:00:18

in stylus UI mode by typing on the command prompt.

Time(HH:MM:SS) : 00:00:23

Linux command prompt Genus..

Time(HH:MM:SS) : 00:00:26

If you want, you can specify the script file here by using hyphen f option or log file hyphen using log, say Genus synthesis dot log.

Time(HH:MM:SS) : 00:00:39

This command will automatically generate a command file with the same name Genus.

Time(HH:MM:SS) : 00:00:45

Dodson dot cmd.

Time(HH:MM:SS) : 00:00:47

Or if you want, you can specify a separate command file name with the same string, say Genus.

Time(HH:MM:SS) : 00:00:55

dot basic dot cmd.

Time(HH:MM:SS) : 00:00:59

And if you are specifying the explicit command name when you are invoking the tool, put them into the single quotes and then you can invoke the tool.

Time(HH:MM:SS) : 00:01:10

So once you are into the Genus shell, the first thing we need to do is to read the setup file which contains information regarding various variables.

Time(HH:MM:SS) : 00:01:21

Setup search path for your HDL libraries or any other file like SDC constraints, etc.

Time(HH:MM:SS) : 00:01:29

or any other required variable.

Time(HH:MM:SS) : 00:01:31

Let's check out the setup file we have.

Time(HH:MM:SS) : 00:01:33

So this is the setup file for our run where we are specifying path for the scripts reports library path RTL path design the messages that we want to suppress.

Time(HH:MM:SS) : 00:01:47

Library list left library list RTL list.

Time(HH:MM:SS) : 00:01:53

Then we are specifying the shark text file.

Time(HH:MM:SS) : 00:01:56

Then attributes related to say HDL track file name row call is true and search path for library script, HDL or any other required variable.

Time(HH:MM:SS) : 00:02:08

In this setup file we are also reading the library.

Time(HH:MM:SS) : 00:02:12

Left library and reading the shark tech file.

Time(HH:MM:SS) : 00:02:15

The command that you can use for reading the library is read underscore libs, and you can specify the lib list or libraries here directly.

Time(HH:MM:SS) : 00:02:24

For reading left library, it's read underscore physical hyphen left and your left file.

Time(HH:MM:SS) : 00:02:30

And for shark attack file you can set it to attribute., set DB shark attack file and the specific tech file here.

Time(HH:MM:SS) : 00:02:40

So let's run the setup file in our session here.

Time(HH:MM:SS) : 00:02:47

So once the setup file is read, since we have read the libraries, it's time to read the article file.

Time(HH:MM:SS) : 00:02:55

So you can use command read underscore HDL.

Time(HH:MM:SS) : 00:02:58

And you can use the list here that you have defined in our setup file.

Time(HH:MM:SS) : 00:03:02

Next is to elaborate the design using command elaborate after elaborate command, make sure to check for any kind of unresolved references in your design.

Time(HH:MM:SS) : 00:03:17

Using command check underscore design hyphen unresolved.

Time(HH:MM:SS) : 00:03:20

You can do so.

Time(HH:MM:SS) : 00:03:21

For our case, the design is clean.

Time(HH:MM:SS) : 00:03:24

There are no unresolved references or empty modules in the design.

Time(HH:MM:SS) : 00:03:29

And then you can read the Constraint file using command read underscore SDC and specifying the SDC file here.

Time(HH:MM:SS) : 00:03:37

Also, make sure to check that there are no errors and failures when you run the Constraint.

Time(HH:MM:SS) : 00:03:41

You can check the same in the statistics of the commands which are executed by read underscore SDC.

Time(HH:MM:SS) : 00:03:47

So for our case, there are no failures.

Time(HH:MM:SS) : 00:03:50

All the commands have been successfully read and then we can go ahead with the synthesis.

Time(HH:MM:SS) : 00:03:55

For synthesis we have three step generic mapping and optimization.

Time(HH:MM:SS) : 00:04:00

And for each stage you can specify the required effort level.

Time(HH:MM:SS) : 00:04:04

For generic synthesis.

Time(HH:MM:SS) : 00:04:06

Medium is the required effort level.

Time(HH:MM:SS) : 00:04:08

If you want, you can change it using Attribute.

Time(HH:MM:SS) : 00:04:11

set underscore db.

Time(HH:MM:SS) : 00:04:12

And then you can specify syn generic effort as say high or low or medium.

Time(HH:MM:SS) : 00:04:19

For our case let's go with medium only which is the default.

Time(HH:MM:SS) : 00:04:24

And then using command syn underscore generic you can run the generic synthesis.

Time(HH:MM:SS) : 00:04:32

During generic optimization Genus.

Time(HH:MM:SS) : 00:04:35

performs optimizations like datapath synthesis, resource sharing, speculation, mux optimization, and carry save rhythmatic case optimization, and you can also see the related information in the log file when generic synthesis is run in the tool, so you can get the information with the various messages which appears.

Time(HH:MM:SS) : 00:04:56

So after generic stage it's time for mapping to the technology library.

Time(HH:MM:SS) : 00:05:00

So during technology mapping tool does restructuring and mapping the design including optimization like splitting pins mapping buffering pattern matching.

Time(HH:MM:SS) : 00:05:09

And then you have the global incremental optimization where it is targeted for error optimization and kind of power optimization.

Time(HH:MM:SS) : 00:05:18

So for mapping also you can set the efforts using Attribute.

Time(HH:MM:SS) : 00:05:22

sin map effort as medium, high or low.

Time(HH:MM:SS) : 00:05:26

High is the default value and if you want you can go ahead with the medium effort.

Time(HH:MM:SS) : 00:05:31

And then you can issue the command sin underscore map to do the mapping.

Time(HH:MM:SS) : 00:05:36

After mapping you can go for the optimization where during incremental optimization, Genus.

Time(HH:MM:SS) : 00:05:41

improves timing and area and also fix DRC Violations...

Time(HH:MM:SS) : 00:05:46

So for this stage, also you can set the effort level using Attribute.

Time(HH:MM:SS) : 00:05:52

sin opt effort.

Time(HH:MM:SS) : 00:05:54

Again, for this attribute, default value is high and let's go ahead with the medium.

Time(HH:MM:SS) : 00:06:00

And the command to optimize is sin.

Time(HH:MM:SS) : 00:06:02

Underscore opt.

Time(HH:MM:SS) : 00:06:04

So after this stage your synthesis is complete.

Time(HH:MM:SS) : 00:06:07

And now you can write out the different output files in terms of your HDL in terms of any interfacing with other tool.

Time(HH:MM:SS) : 00:06:14

Or you can analyze the result in terms of area timing or power.

Time(HH:MM:SS) : 00:06:19

So if you say example, you are generating the QR report using Command report and the QR and it gives information about your timing clocks instance count area, you can also get information about max Fanout min Fanout runtime and what is the memory usage during the run.

Time(HH:MM:SS) : 00:06:40

We can write out different output files.

Time(HH:MM:SS) : 00:06:42

Say write underscore HDL.

Time(HH:MM:SS) : 00:06:45

You can direct it to some name.

Time(HH:MM:SS) : 00:06:47

We are saying basic dot cin dot v and it will be saved at the location from where you are running this Genus and as well as you can write out other output file, say interfacing with LEC, Innovus, etc..

Time(HH:MM:SS) : 00:07:03

Let's summarize the basic synthesis flow that we have run in Genus.

Time(HH:MM:SS) : 00:07:07

stylus UI.

Time(HH:MM:SS) : 00:07:09

So this is the script.

Time(HH:MM:SS) : 00:07:11

We sourced the setup file which had information about various path various list for RTL left library library Script search path.

Time(HH:MM:SS) : 00:07:23

Rtl search path or library search path.

Time(HH:MM:SS) : 00:07:26

And we are setting the library using read underscore libs command and left library using read underscore physical and attribute.

Time(HH:MM:SS) : 00:07:34

for shark tech file.

Time(HH:MM:SS) : 00:07:36

After this setting, we read the HDL, elaborated the design check for any kind of unresolved references or empty module,.

Time(HH:MM:SS) : 00:07:45

Read the constraints synthesize to generic using sin underscore generic.

Time(HH:MM:SS) : 00:07:51

You can also set the effort level for your synthesis using attribute.

Time(HH:MM:SS) : 00:07:56

sin underscore generic underscore effort for generic Similarly for mapping and optimization.

Time(HH:MM:SS) : 00:08:03

Also, you can set the effort.

Time(HH:MM:SS) : 00:08:05

You can run the mapping using command sin underscore map and then further optimize using sin underscore opt command.

Time(HH:MM:SS) : 00:08:13

After this, you can generate various output files and analyze the results using various reporting commands like report, core, report, area, timing, etc..

VideoTitle :Generating reports in Genus Stylus CUI

Time(HH:MM:SS) : 00:00:01

Dft that is designed for test techniques, provide different methods to comprehensively test the manufactured device for quality and coverage purposes.

Time(HH:MM:SS) : 00:00:10

With the help of the DFT, you can assure the detection of all faults in a circuit.

Time(HH:MM:SS) : 00:00:15

It is helpful to reduce the cost and time associated with test development, as well as reduce the execution time of performing test on fabricated chips.

Time(HH:MM:SS) : 00:00:24

You can detect different faults in DFT.

Time(HH:MM:SS) : 00:00:26

The common ones are stuck at faults, transection faults, stuck open, and shorts and bridging faults.

Time(HH:MM:SS) : 00:00:35

Before we discuss scan insertion flow in detail, let's see how a design with test circuit looks like.

Time(HH:MM:SS) : 00:00:42

The circuit here shows normal flops along with combinational logic, with data in, data out and clock which is feeding to the registers.

Time(HH:MM:SS) : 00:00:49

Now, when scan insertion is done, these normal flip flops are replaced by equivalent.

Time(HH:MM:SS) : 00:00:54

scan flip flops from the libraries.

Time(HH:MM:SS) : 00:00:56

These flip flops have ports like scan in, scan out, and shift enable.

Time(HH:MM:SS) : 00:01:01

The test process puts the circuit into one of the three test modes capture mode, scan shift mode, and system mode, out of which system mode is the normal or functional operation mode of the circuit.

Time(HH:MM:SS) : 00:01:13

Any logic dedicated to DFT purpose is not active in the system mode, while scan shift mode is the part of the test process in which registers act as a shift register in the scan chain.

Time(HH:MM:SS) : 00:01:24

In capture mode, it analyzes the combinational logic on the chip.

VideoTitle :Generating Timing Reports in Genus Stylus CUI

Time(HH:MM:SS) : 00:00:01

This is the top down DFT synthesis flow in Genus synthesis solution.

Time(HH:MM:SS) : 00:00:06

Once you have read the target libraries or MMC file, read the design and elaborate it.

Time(HH:MM:SS) : 00:00:12

Initialize the design if you are running MMC flow.

Time(HH:MM:SS) : 00:00:16

Set the timing and design constraints and apply required optimization directives.

Time(HH:MM:SS) : 00:00:21

Then you can do setup for DFT rule.

Time(HH:MM:SS) : 00:00:24

You need to check the DFT rule, fix any violations.

Time(HH:MM:SS) : 00:00:27

Add testability logic if required.

Time(HH:MM:SS) : 00:00:29

Synthesize your design to map.

Time(HH:MM:SS) : 00:00:31

Set up DFT configuration constraints, connect your scan chain, run incremental optimization and then analyze your design.

Time(HH:MM:SS) : 00:00:40

Check if you need to modify Constraint.

Time(HH:MM:SS) : 00:00:42

If not, if you're meeting the constraint, then you can write out the output files.

VideoTitle :Generating Outputs Files in Genus Stylus CUI

Time(HH:MM:SS) : 00:00:01

In this debugging module, we will be discussing and debugging error message , which appears while reading the design.

Time(HH:MM:SS) : 00:00:12

As a module objective.

Time(HH:MM:SS) : 00:00:14

We will be analyzing the error VLOGPT-46, which comes while reading the design file, and then we will fix the issue.

Time(HH:MM:SS) : 00:00:25

Hello everyone!

Time(HH:MM:SS) : 00:00:26

In this debugging section we will discuss how you can debug error.

Time(HH:MM:SS) : 00:00:31

VLOGPT 46 which appears while reading the Verilog designs.

Time(HH:MM:SS) : 00:00:38

So we are into the Genus shell.

Time(HH:MM:SS) : 00:00:40

So we'll start by first reading the libraries.

Time(HH:MM:SS) : 00:00:43

So we'll read the library, which is already available in the Dennis installation that is tutorial dot lib.

Time(HH:MM:SS) : 00:00:49

Once you have read the libraries, we'll now read our HDL file.

Time(HH:MM:SS) : 00:00:53

So we are reading our HDL using read underscore HDL hyphen sv specifying the file here fv dot v.

Time(HH:MM:SS) : 00:01:01

So when we are reading our HDL We are getting this error message.

Time(HH:MM:SS) : 00:01:05

VLOGPT 46, which says that an if statement is required at the top of an always block to infer a ledger flip flop.

Time(HH:MM:SS) : 00:01:14

And then we are also getting information at which line and column the error candidate is present.

Time(HH:MM:SS) : 00:01:20

We need to resolve this issue.

Time(HH:MM:SS) : 00:01:22

Let's check out how.

Time(HH:MM:SS) : 00:01:26

Let's summarize the problem.

Time(HH:MM:SS) : 00:01:27

The problem is related to failure while reading design in Genus.

Time(HH:MM:SS) : 00:01:31

As a given input, we read the technology library and while reading HDL file errors are seen.

Time(HH:MM:SS) : 00:01:37

The error message is related to VLOGPT 46, which is coming for the read underscore HDL command.

Time(HH:MM:SS) : 00:01:44

And the goal is to remove this error message and to get the read design clean without any errors.

Time(HH:MM:SS) : 00:01:52

Let's check out what are the different possible strategies and solutions that we can use to debug and analyze our error message.

Time(HH:MM:SS) : 00:01:59

First, we can start by checking the log file for an error or warning message, which appears after the read underscore HDL command.

Time(HH:MM:SS) : 00:02:07

This provides a direct hint regarding what kind of issue is there, and which can further help to pinpoint the issue related to the RTL file.

Time(HH:MM:SS) : 00:02:15

But then you cannot get much detail just by looking at the error or warning message.

Time(HH:MM:SS) : 00:02:20

Next is we can debug the RTL code.

Time(HH:MM:SS) : 00:02:23

We can check the RTL code, which is pointed by the error message, which can further help us to dig the root cause of the issue.

Time(HH:MM:SS) : 00:02:30

And for this, you need to thoroughly analyze the RTL file, look at the various steps and locate the error prone coding style.

Time(HH:MM:SS) : 00:02:38

So what is the strategy we are going to use?

Time(HH:MM:SS) : 00:02:41

We are going to use strategy two because this is the most efficient method to start debugging.

Time(HH:MM:SS) : 00:02:47

By using this, we can directly correlate the issue while reading the HDL with the corresponding code in the RTL file.

Time(HH:MM:SS) : 00:03:16

From the error message.

Time(HH:MM:SS) : 00:03:17

It seems that issue is related to the style of the RTL coding.

Time(HH:MM:SS) : 00:03:22

So let's check out RTL file and debug further.

Time(HH:MM:SS) : 00:03:26

So this is the RTL file which we are reading, where we are defining our input and output signal in the Module, list and then defining internal variables.

Time(HH:MM:SS) : 00:03:35

And then we have the code which talks about the flop definition with the always block.

Time(HH:MM:SS) : 00:03:41

So this, given RTL has an edge triggered always block, which is used for inferring the flop edge triggered always block are not always synthesizable.

Time(HH:MM:SS) : 00:03:51

So to distinguish between synthesizable and non synthesizable code, Genus.

Time(HH:MM:SS) : 00:03:57

does not allow the first statement of an edge triggered always block to be a loop statement.

Time(HH:MM:SS) : 00:04:03

That is for while, repeat, etc.

Time(HH:MM:SS) : 00:04:06

they are not allowed.

Time(HH:MM:SS) : 00:04:07

So we need to make this code synthesizable RTL and hence we need to modify the code which appears here.

Time(HH:MM:SS) : 00:04:15

So it should start with the if statement instead of the for loop.

Time(HH:MM:SS) : 00:04:19

And that's the cause of the error which we are getting on our Genus.

Time(HH:MM:SS) : 00:04:23

Xrun.

Time(HH:MM:SS) : 00:04:23

While reading the HTML file, that if statement is required at the top of an always block to infer a ledge or flip flop.

Time(HH:MM:SS) : 00:04:31

Let's check out how we can modify it and read the article successfully.

Time(HH:MM:SS) : 00:04:53

So this is our modified RTL file.

Time(HH:MM:SS) : 00:04:56

So the definitions are same for the input output for your internal variables, and for the code where we are defining the always block.

Time(HH:MM:SS) : 00:05:04

We have started the statement with the if instead of the for loop.

Time(HH:MM:SS) : 00:05:08

So we are defining the condition here for the if reset.

Time(HH:MM:SS) : 00:05:12

Said.

Time(HH:MM:SS) : 00:05:13

And then we have the condition for what the Q will be assigned to.

Time(HH:MM:SS) : 00:05:17

And then for else condition for enable, etc..

Time(HH:MM:SS) : 00:05:20

Let's run this code again in our Dennis run and see if tool is able to detect or synthesize this code or not.

Time(HH:MM:SS) : 00:05:27

We are into the fresh Genus.

Time(HH:MM:SS) : 00:05:29

session now.

Time(HH:MM:SS) : 00:05:31

So again we'll start by reading the libraries which is tutorial dot lib.

Time(HH:MM:SS) : 00:05:37

And then we'll read our HDL now which is the corrected one.

Time(HH:MM:SS) : 00:05:42

So we are going to use.

Time(HH:MM:SS) : 00:05:45

So now this time after using the correct HDL we are getting no error related to veloped 46.

Time(HH:MM:SS) : 00:05:52

After that you can elaborate the design.

Time(HH:MM:SS) : 00:05:55

You can also synthesize to generic, though we are not specifying any timing Constraint we are just running the generic synthesis to check out the netlist you can synthesize to map as well as you can optimize it.

Time(HH:MM:SS) : 00:06:09

We can check the GUI.

Time(HH:MM:SS) : 00:06:10

also using GUI.

Time(HH:MM:SS) : 00:06:11

underscore show.

Time(HH:MM:SS) : 00:06:12

So here we have the Genus.

Time(HH:MM:SS) : 00:06:13

GUI..

Time(HH:MM:SS) : 00:06:14

You can turn on the schematic using this tab here.

Time(HH:MM:SS) : 00:06:17

And this is how your design is going to synthesize into the final netlist form.

Time(HH:MM:SS) : 00:06:21

So to summarize, if you get error message VLOGPT GPT 46 in your design, check whether the code is synthesizable or not.

Time(HH:MM:SS) : 00:06:29

Synthesizable Genus.

Time(HH:MM:SS) : 00:06:31

does not allow the first statement of an edge triggered always block to be loop statement that is for while repeat.

Time(HH:MM:SS) : 00:06:38

So verify the RTL and update the code accordingly and check the synthesized result.

Time(HH:MM:SS) : 00:06:45

Let's summarize the problem and solution.

Time(HH:MM:SS) : 00:06:48

The problem was related to the failure of the design while using command read underscore HDL as given input, we read libraries and design file.

Time(HH:MM:SS) : 00:06:58

The goal was to fix the error message which was appearing while running the read underscore HDL command.

Time(HH:MM:SS) : 00:07:04

We selected the strategy of analyzing the RTL code to find the root cause of the issue, and finally, after correcting the code in the RTL file, we were able to read the design correctly without any errors.

VideoTitle :Demo Basic Synthesis Flow in Genus Synthesis Solution

Time(HH:MM:SS) : 00:00:01

Let's summarize what we discussed in this chapter.

Time(HH:MM:SS) : 00:00:04

We understood how to set up and run the basic synthesis flow and write out various output files.

Time(HH:MM:SS) : 00:00:09

We also saw the setup for design for test during synthesis as well as we analyzed and debugged the VLOGPTPT 46 error message.

VideoTitle :What Is DFT

Time(HH:MM:SS) : 00:00:01

This slide shows the reference content for the Genus present at the support portal.

Time(HH:MM:SS) : 00:00:06

You can find Genus.

Time(HH:MM:SS) : 00:00:07

training bytes and demo videos, user guides, articles, etc.

Time(HH:MM:SS) : 00:00:12

further, there is a one stop page to help you locate various content types in one place.

Time(HH:MM:SS) : 00:00:18

If you want to refer to the detailed training on Genus synthesis solutions, please check the course Genus synthesis solution with stylus common UI.

VideoTitle :RTL Top-Down DFT Flow

Time(HH:MM:SS) : 00:00:01

The lab exercises for this module are running the basic synthesis flow and running the synthesis flow with DFT.

VideoTitle :Quiz

Time(HH:MM:SS) : 00:00:00

This module discusses the test stage in RTL to GDS two file.

Time(HH:MM:SS) : 00:00:04

Our learning goal for this module is to run the basic Atpg flow in Modus test..

VideoTitle :Quiz

Time(HH:MM:SS) : 00:00:00

In the complete RTL to GDSII flow after logic synthesis and DFT insertion, it's time for ATPG vector generation.

VideoTitle :Debugging VLOGPT-46 Error Message

Time(HH:MM:SS) : 00:00:00

Modus test software is a fully integrated suite of tools designed from the beginning to interact and work with each other.

Time(HH:MM:SS) : 00:00:06

The capabilities include test synthesis, which is the process of inserting test logic into a design to help to make it testable.

Time(HH:MM:SS) : 00:00:13

Test analysis is the process of identifying the test logic and verifying that it has been inserted and integrated correctly, and identifying any design characteristics that will create testing problems.

Time(HH:MM:SS) : 00:00:24

Test generation is the process of creating and testing vectors that are applied at the tester.

Time(HH:MM:SS) : 00:00:30

Test diagnostics is the process of working backward from failure.

Time(HH:MM:SS) : 00:00:33

This compares to the probable cause and to help in the production and yield management of semiconductor lines.

Time(HH:MM:SS) : 00:00:39

We'll focus on the traditional atpg disciplines of test analysis and test generation.

Time(HH:MM:SS) : 00:00:45

This includes correct by design test structures using Genus.

Time(HH:MM:SS) : 00:00:48

in the form of verify compliance with test design rules.

Time(HH:MM:SS) : 00:00:51

A full range of tests for all types of digital testing and diagnostics for fault isolation, failure analysis and process monitoring.

Time(HH:MM:SS) : 00:00:59

This is the Modus test.

Time(HH:MM:SS) : 00:01:00

atpg flow, which is divided into two stages first, for preparing for Atpg and then running Atpg during the prepared stage for Atpg.

Time(HH:MM:SS) : 00:01:10

You provide the Verilog Libraries in netlist.

Time(HH:MM:SS) : 00:01:13

Build the model.

Time(HH:MM:SS) : 00:01:14

Build the test mode.

Time(HH:MM:SS) : 00:01:15

Verify test structure.

Time(HH:MM:SS) : 00:01:17

Internal scan.

Time(HH:MM:SS) : 00:01:18

Boundary scan and memory test.

Time(HH:MM:SS) : 00:01:20

If there are any violation, fix them and repeat the process.

Time(HH:MM:SS) : 00:01:25

If it is fine, you can build a fault model.

Time(HH:MM:SS) : 00:01:28

Then during the run stage you create and commit logic.

Time(HH:MM:SS) : 00:01:31

Test scan logic, transition memory, etc..

Time(HH:MM:SS) : 00:01:36

Finally, you write out the vectors for verification and manufacturing and send them to Xcelium, SimVision and, Voltus, etc.

Time(HH:MM:SS) : 00:01:44

and patterns to eight.

VideoTitle :Module Summary

Time(HH:MM:SS) : 00:00:00

Let's check out how you can invoke a Modus test..

Time(HH:MM:SS) : 00:00:03

Once you have installed the required binaries, you can simply invoke a Modus test.

Time(HH:MM:SS) : 00:00:07

by entering modus at the Unix or Linux prompt.

Time(HH:MM:SS) : 00:00:10

Cadences Modus test.

Time(HH:MM:SS) : 00:00:11

inherently supports two interfaces to the applications.

Time(HH:MM:SS) : 00:00:15

There is a command line which can accept both commands and scripts, and the graphical user interface.

Time(HH:MM:SS) : 00:00:20

Gui..

Time(HH:MM:SS) : 00:00:22

The command line interface is a Unix or TCL environment.

Time(HH:MM:SS) : 00:00:26

Modus test.

Time(HH:MM:SS) : 00:00:27

has its own commands to call functions, but it also supports the interfaces to traditional Unix commands.

Time(HH:MM:SS) : 00:00:33

If you want to invoke a Modus test.

Time(HH:MM:SS) : 00:00:35

in a graphical user interface, use the modus GUI.

Time(HH:MM:SS) : 00:00:38

command.

Time(HH:MM:SS) : 00:00:39

You can also invoke GUI.

Time(HH:MM:SS) : 00:00:40

with the help of command GUI.

Time(HH:MM:SS) : 00:00:42

underscore open from the Module, TCL console.

Time(HH:MM:SS) : 00:00:45

You also have the option to execute specific modus binary shell commands using modus hyphen E to specify a modus command or script.

Time(HH:MM:SS) : 00:00:53

With this method, you can either run a command without starting a shell, or you can also run a command script for example Atpg scripts generated from the Genus Synthesis Solution output.

VideoTitle :References

Time(HH:MM:SS) : 00:00:00

This slide shows the graphical user interface of modus GUI.

Time(HH:MM:SS) : 00:00:04

You can invoke GUI.

Time(HH:MM:SS) : 00:00:05

either by specifying the modus GUI.

Time(HH:MM:SS) : 00:00:07

command in the beginning.

Time(HH:MM:SS) : 00:00:08

When you start the tool, or use the GUI.

Time(HH:MM:SS) : 00:00:10

underscore open command from the command console.

Time(HH:MM:SS) : 00:00:13

This graphical user interface allows you to access Modus test.

Time(HH:MM:SS) : 00:00:17

tasks in several ways.

Time(HH:MM:SS) : 00:00:19

These tasks can be selected and executed using the GUI methodology.

Time(HH:MM:SS) : 00:00:23

The various menus, the GUI command line, or the GUI task menu.

Time(HH:MM:SS) : 00:00:29

There are various options in the GUI that helps you to process the task selected, or create your own methodology and perform project specific processing from the toolbar.

Time(HH:MM:SS) : 00:00:38

You can see the title bar, menu bar, how to exit the GUI, as well as the task view where you can import previous task history and refresh the task view.

Time(HH:MM:SS) : 00:00:48

Not only this, you can see the log area for the tasks.

VideoTitle :Labs

Time(HH:MM:SS) : 00:00:00

To summarize, in this module we explored the fundamentals of the Modus test ATPG flow.

VideoTitle :The Test Stage

Time(HH:MM:SS) : 00:00:01

Here are some additional reference materials for modus.

VideoTitle :Module Objectives

Time(HH:MM:SS) : 00:00:00

The lab exercise for this module includes running the basic Atpg flow in Modus test..

VideoTitle :Test Stage Flowchart

Time(HH:MM:SS) : 00:00:01

In this module, we'll discuss how you can do equivalency checking of a design.

Time(HH:MM:SS) : 00:00:09

The learning goals for this module include identifying the.

VideoTitle :Modus Test Disciplines

Time(HH:MM:SS) : 00:00:01

In the flowchart of RTL to GDS two flow.

Time(HH:MM:SS) : 00:00:04

We are at this logic equivalence checking stage.

Time(HH:MM:SS) : 00:00:08

Let's check out how you can do formal verification of the design.

VideoTitle :Starting a Modus Test

Time(HH:MM:SS) : 00:00:01

When you synthesize the design, it undergoes various complex transformations and optimization to meet the timing area and power.

Time(HH:MM:SS) : 00:00:09

We need to ensure that the original intent of the RTL is intact.

Time(HH:MM:SS) : 00:00:13

That is, the functionality doesn't change logic.

Time(HH:MM:SS) : 00:00:17

Equivalence checking is a process to ensure that the synthesized netlist is functionally equivalent to the original RTL design, because we need to make sure whatever optimization and transformations which have happened are not changing the original functionality.

Time(HH:MM:SS) : 00:00:30

Cadence Conformal Equivalence checking solutions are logical equivalence checking tools that verify RTL gate or transistor level designs.

Time(HH:MM:SS) : 00:00:40

Equivalence checking is required at numerous stages, because at each stage certain transformations and iterations happen in terms of logic synthesis, optimization, test insertion, clock synthesis, floor planning, etc.

Time(HH:MM:SS) : 00:00:54

we need to make sure whatever changes are done at each stage, it is not functionally affecting the design, so you need to do the logic equivalence checking.

VideoTitle :The Graphical User Interface

Time(HH:MM:SS) : 00:00:01

Let's discuss Equivalence checking flow.

Time(HH:MM:SS) : 00:00:04

The flow is divided into two stages.

Time(HH:MM:SS) : 00:00:06

One is the setup mode and the other is the LEC mode.

Time(HH:MM:SS) : 00:00:10

During setup mode, you provide the input files in terms of the original RTL or design standard libraries and optimized netlist.

Time(HH:MM:SS) : 00:00:19

Then you specify constraints and modeling options.

Time(HH:MM:SS) : 00:00:22

Once you are into the LEC mode, you specify various compare parameters, compare the design, check the results, and if there are any non-equivalents points, you do the diagnosis and then again specify some compare

parameters.

Time(HH:MM:SS) : 00:00:36

If not, it means that the design is equivalent.

Time(HH:MM:SS) : 00:00:39

and it's fine to go ahead.

Time(HH:MM:SS) : 00:00:43

We discuss that LEC flow is divided into two stages setup mode and LEC mode.

Time(HH:MM:SS) : 00:00:50

In setup mode, you do the setup of the design for comparison, like reading of the design, defining the design constraints, and specifying modeling options, specifying any black boxes, libraries, etc..

Time(HH:MM:SS) : 00:01:03

Then in LEC mode you do the design comparison.

Time(HH:MM:SS) : 00:01:06

Here mapping process takes place when you go from setup mode to LEC mode.

Time(HH:MM:SS) : 00:01:11

You resolve any unmapped key points if present before proceeding for the comparison.

Time(HH:MM:SS) : 00:01:16

You start the comparison process.

Time(HH:MM:SS) : 00:01:18

You diagnose and debug the non-equivalents key points of the design, and if everything is clean, then you come back with the Equivalent.

Time(HH:MM:SS) : 00:01:26

points and the Equivalence checking process is then complete.

VideoTitle :Quiz

Time(HH:MM:SS) : 00:00:00

Let's check out how you can start and exit LEC.

Time(HH:MM:SS) : 00:00:03

Once you've installed the required binaries of the tool, you can start Conformal LEC using various options.

Time(HH:MM:SS) : 00:00:10

If you want to start in menu mode, start by using L, XL, Gql, ECO, LP XL, and the LP Gql option, If you want to start in command mode, start by specifying L, XL Gql using option no Gui..

Time(HH:MM:SS) : 00:00:28

You can always switch between the menu that is the GUI and command modes by using the command set GUI.

Time(HH:MM:SS) : 00:00:33

on or off.

Time(HH:MM:SS) : 00:00:34

Lec also provides the flexibility to run in batch mode.

Time(HH:MM:SS) : 00:00:39

For this, you need to specify the command LEC and specify the do file in batch mode and option no.

Time(HH:MM:SS) : 00:00:45

Gui..

Time(HH:MM:SS) : 00:00:46

If you want to exit the LEC session, you can use command exit force.

Time(HH:MM:SS) : 00:00:51

Lec automatically closes all windows upon exit.

VideoTitle :Module Summary

Time(HH:MM:SS) : 00:00:01

The slide shows the graphical user interface.

Time(HH:MM:SS) : 00:00:04

Gui.

Time(HH:MM:SS) : 00:00:05

terminal of the Conformal tool.

Time(HH:MM:SS) : 00:00:07

You can switch back and forth between the graphical and Non-graphical mode any time during the session by using the command set GUI.

Time(HH:MM:SS) : 00:00:15

on off.

Time(HH:MM:SS) : 00:00:17

The graphical interface has a menu bar on the top with file setup, report, and other menus.

Time(HH:MM:SS) : 00:00:23

Below the menu bar is the icon bar with shortcuts to the debug tools.

Time(HH:MM:SS) : 00:00:28

The Design hierarchy window shows the golden design on the left and revised design on the right.

Time(HH:MM:SS) : 00:00:34

The transcript window shows all the messages issued by the tool.

Time(HH:MM:SS) : 00:00:38

The command entry window is where you enter all commands.

Time(HH:MM:SS) : 00:00:42

The bottom of the status bar indicates the state of your session.

Time(HH:MM:SS) : 00:00:46

Run time for the graphical user interface, GUI and Non-graphical user interface.

Time(HH:MM:SS) : 00:00:52

Noneq GUI or command mode might not be much different for small designs, but for large designs the difference can be significant.

Time(HH:MM:SS) : 00:01:01

In general, you can run the design in non-gui mode or command mode to speed up the process and switch to the GUI interface mode to use the diagnosis window for debugging, then switch back to command mode to rerun the software.

VideoTitle :References

Time(HH:MM:SS) : 00:00:01

Here we are showing the standard do file example for the Conformal run.

Time(HH:MM:SS) : 00:00:06

You can set the log file, read the library, read the design for the golden and for the revised you can report the design data and black boxes.

Time(HH:MM:SS) : 00:00:15

Report the Module, apply design constraints and modeling options.

Time(HH:MM:SS) : 00:00:20

Then you can set the system mode to LEC at all the compare points, compare diagnose and then you can check for the equivalence and non-equivalents points.

VideoTitle :Labs

Time(HH:MM:SS) : 00:00:00

Let's summarize what we learned in this module.

Time(HH:MM:SS) : 00:00:02

We identified the basics of the Equivalence checker.

Time(HH:MM:SS) : 00:00:05

software.

Time(HH:MM:SS) : 00:00:06

And we also saw how to set up a design for Equivalence checking.

VideoTitle :The Equivalency Checking Stage

Time(HH:MM:SS) : 00:00:01

Here are some additional reference materials for Conformal.

VideoTitle :Module Objectives

Time(HH:MM:SS) : 00:00:00

The lab exercises for this module include running the Equivalence checking flow and Conformal and creating a v format file from zlib format.

VideoTitle :Equivalency Checking Flowchart

Time(HH:MM:SS) : 00:00:00

In this module you will learn how to implement a design using Innovus implementation system.

Time(HH:MM:SS) : 00:00:07

Take a few moments to read these modules Objectives.

VideoTitle :What Is Logic Equivalence Checking

Time(HH:MM:SS) : 00:00:00

In this module, we will be covering how to implement a design using Innovus implementation system.

Time(HH:MM:SS) : 00:00:06

The main steps in implementation are floor planning, power planning.

Time(HH:MM:SS) : 00:00:11

Placement clock tree synthesis and routing.

VideoTitle :Equivalence Checking Flow

Time(HH:MM:SS) : 00:00:00

During the Digital implementation stage, you start with a gate level netlist, read in the technology files, and generate a placed and routed physical design that meets you power, performance, and area goals.

VideoTitle :Quiz

Time(HH:MM:SS) : 00:00:00

Before starting the software, open an xterm window.

Time(HH:MM:SS) : 00:00:04

This window becomes the input window for Tcl commands after you start the software.

Time(HH:MM:SS) : 00:00:10

To start an Innovus session in stylus mode, enter Innovus Hyphen Stylus.

Time(HH:MM:SS) : 00:00:16

Running the Innovus tool without the stylus option runs the tool in legacy mode, which has a different command structure and user interface.

Time(HH:MM:SS) : 00:00:24

This course is based entirely on the stylus common UI flow kit and metrics.

Time(HH:MM:SS) : 00:00:31

To display the command line options for Innovus type in Innovus hyphen help.

Time(HH:MM:SS) : 00:00:36

The Innovus implementation system creates two types of files log files and command files.

Time(HH:MM:SS) : 00:00:44

The dot log files are verbose versions of the log file, and contain additional information.

Time(HH:MM:SS) : 00:00:50

To suppress the verbose log file, enter the command Innovus hyphen Know how.

Time(HH:MM:SS) : 00:00:56

Here is an example of the command.

Time(HH:MM:SS) : 00:00:58

You can find the installation path as long as the path is set by typing in Innovus.

Time(HH:MM:SS) : 00:01:04

The Innovus implementation system binary is located in the following directory path.

Time(HH:MM:SS) : 00:01:10

The documentation is also installed relative to the installation directory under binary dot.

Time(HH:MM:SS) : 00:01:15

This is the Innovus implementation system window.

Time(HH:MM:SS) : 00:01:19

It includes pull downs, toolbar icons, floor planning icons, design views which are floorplan, amoeba, and physical, as well as the ability to make certain objects visible and selectable.

Time(HH:MM:SS) : 00:01:32

On the bottom right, you can see the status of the design, which shook as a design that is shown here is currently routed.

Time(HH:MM:SS) : 00:01:39

By pressing Q, you can display objects.

Time(HH:MM:SS) : 00:01:41

When you put your cursor over objects, the details of those objects are the same as pressing Q, and over here you see the number of objects that are selected.

VideoTitle :Starting and Exiting LEC

Time(HH:MM:SS) : 00:00:02

In this demonstration, I'm going to show you some of the features of the Innovus user interface.

Time(HH:MM:SS) : 00:00:09

I have read in a placed and rooted design.

Time(HH:MM:SS) : 00:00:13

If you see these arrows, it means that you should likely expand the window to see everything.

Time(HH:MM:SS) : 00:00:27

First, these are the views.

Time(HH:MM:SS) : 00:00:29

This is a floor plan view.

Time(HH:MM:SS) : 00:00:31

This is the amoeba view.

Time(HH:MM:SS) : 00:00:33

This is the physical view and this is the 3D view.

Time(HH:MM:SS) : 00:00:38

On the right hand side you see the visibility and the selectability of objects.

Time(HH:MM:SS) : 00:00:44

Here I am going to be in the physical view, and I can see the details of the nets and the standard cells roof.

Time(HH:MM:SS) : 00:01:00

If I just want to see the standard cells and don't want to see the nets, then I can unselect the visibility of the layers.

Time(HH:MM:SS) : 00:01:10

On the bottom right, you can see the status of the design.

Time(HH:MM:SS) : 00:01:14

In this case, it says that the design is rooted.

Time(HH:MM:SS) : 00:01:18

So you can easily tell if the design has been timing analyzed, rooted, placed, or whatever the current state is by looking at this bottom right area.

Time(HH:MM:SS) : 00:01:30

If I select an object, then I can see the number of selected objects also in the bottom right hand side.

Time(HH:MM:SS) : 00:01:36

I can press the shift key and select multiple objects, and you will see that the number of selected objects has changed from 1 to 2.

Time(HH:MM:SS) : 00:01:45

Click an empty space like somewhere in here to clear the selection.

Time(HH:MM:SS) : 00:01:52

If I press F on the keyboard then that will fit the design.

Time(HH:MM:SS) : 00:02:06

If you want to see the details of an object selected and press Q on the keyboard, that brings up the object Attribute editor form, and you can look at the attributes of that object and also change some of the attributes depending on the types of attributes they are.

Time(HH:MM:SS) : 00:02:23

So if you see a pencil type icon next to the attribute, that means that you can change the status.

Time(HH:MM:SS) : 00:02:34

Another way of seeing the details of an object is to press the Q here.

Time(HH:MM:SS) : 00:02:38

And then when you mouse over the objects, then there'll be a pop up that reveal the details about the object.

Time(HH:MM:SS) : 00:02:49

If you want more granular control over the visibility and selectability of the objects, then press all colors and it will give you a lot more options on changing the visibility and selectability of objects when compared to what's in this pane.

Time(HH:MM:SS) : 00:03:06

Some of the objects are both visible and selectable, whereas view only objects are only.

Time(HH:MM:SS) : 00:03:13

You can only change the visibility and not selectability.

Time(HH:MM:SS) : 00:03:16

So here you'll find objects such as congestion related objects, g cell objects, overflow, and so on that give you information about that object.

Time(HH:MM:SS) : 00:03:28

But you can't really change the selectability their view only objects.

Time(HH:MM:SS) : 00:03:39

In order to select an object, here is the icon.

Time(HH:MM:SS) : 00:03:41

The selection icon.

Time(HH:MM:SS) : 00:03:43

To move an object you would click this icon.

Time(HH:MM:SS) : 00:03:46

The move icon and move the object.

Time(HH:MM:SS) : 00:03:49

And to get back into selection mode you would press this icon again.

Time(HH:MM:SS) : 00:03:53

Or you can press A on the keyboard.

Time(HH:MM:SS) : 00:03:58

In order to view the design in 3D mode.

Time(HH:MM:SS) : 00:04:02

You can select the 3D icon and current view.

Time(HH:MM:SS) : 00:04:12

Here are instructions on how you would rotate, pan, select, and so on.

Time(HH:MM:SS) : 00:04:19

This feature is handy especially when you're looking at DRC's.

Time(HH:MM:SS) : 00:04:26

I can also auto play by selecting the auto revolving icon for a demo.

Time(HH:MM:SS) : 00:04:35

The menus are organized in the order that you will use them.

Time(HH:MM:SS) : 00:04:38

For example, on the left is reading in a design, and then as you move to the right, you get into floor planning, power planning, placement, clock tree synthesis, routing, timing analysis, and so on.

Time(HH:MM:SS) : 00:04:49

Additional tools are available under tools, for example the Design Browser.

Time(HH:MM:SS) : 00:05:00

Depending on the type of view that's your current view, you'll find that some of these icons will change depending on the selected view.

Time(HH:MM:SS) : 00:05:08

So here I've selected floor planning I'm going to see these floor planning related icons.

Time(HH:MM:SS) : 00:05:13

If I go into the physical view these icons will change so that they're more relevant to the physical view

Time(HH:MM:SS) : 00:05:22

The amoeba view shows you how the modules have been placed in your design.

Time(HH:MM:SS) : 00:05:27

If there had been many modules, then you can analyze the placement to see if the modules that have a high level of connectivity have been placed close together.

Time(HH:MM:SS) : 00:05:37

This view is relevant only after placement, and these are some of the ways that you can use the GUI.

Time(HH:MM:SS) : 00:05:44

of the Innovus implementation system.

VideoTitle :The Graphical User Interface (GUI)

Time(HH:MM:SS) : 00:00:00

There are multiple ways of running TCL command scripts in batch mode.

Time(HH:MM:SS) : 00:00:05

To run the tool in Non-graphical mode for the entire session, enter Innovus Hyphen Stylus hyphen Node GUI..

Time(HH:MM:SS) : 00:00:13

You can then source a tickle script as shown in this example.

Time(HH:MM:SS) : 00:00:17

If you want to run a process without this implementation system window type Innovus hyphen.

Time(HH:MM:SS) : 00:00:23

Stylus hyphen files followed by the name of the script you want to source.

Time(HH:MM:SS) : 00:00:28

To start the graphical mode from non-graphical, type Gui_show.

Time(HH:MM:SS) : 00:00:32

and to turn off the graphical mode type GUII..

VideoTitle :Standard Dofile Example

Time(HH:MM:SS) : 00:00:00

Here are the input files for the Innovus implementation system.

Time(HH:MM:SS) : 00:00:04

A netlist file that can be either in Verilog or O.

Time(HH:MM:SS) : 00:00:08

The floor plan file which you can create within Innovus.

Time(HH:MM:SS) : 00:00:11

Or you can import a FP or DEF file.

Time(HH:MM:SS) : 00:00:15

Clock tree spec files are created by Innovus and they are created from the s Scan chains information in the form of a scan.

Time(HH:MM:SS) : 00:00:23

Def or typical I o information in an I o pad file.

Time(HH:MM:SS) : 00:00:28

A GDS layer map file is required if you are going to be outputting a GDS.

Time(HH:MM:SS) : 00:00:33

Timing constraints in SDC format.

Time(HH:MM:SS) : 00:00:37

Timing libraries are lib files.

Time(HH:MM:SS) : 00:00:39

F libraries for Place and route and additional technology files for extraction, such as cap tables or Quantus tCKQ tech files.

Time(HH:MM:SS) : 00:00:49

Also, you can import a Power intent.

Time(HH:MM:SS) : 00:00:51

file in CPF or IEEE 1801 format.

Time(HH:MM:SS) : 00:00:56

When you save a design, the Innovus implementation directory that contains all of your design data.

Time(HH:MM:SS) : 00:01:03

The command write underscore db will create a directory that contains all of the design data.

Time(HH:MM:SS) : 00:01:09

To load a saved database, run the read underscore db command.

Time(HH:MM:SS) : 00:01:14

Here is an example of the files that are saved in the design directory, including placement, floor planning, and root file.

VideoTitle :Module Summary

Time(HH:MM:SS) : 00:00:00

When you save a design.

Time(HH:MM:SS) : 00:00:01

The Innovus implementation system creates a directory that contains all of your design data.

Time(HH:MM:SS) : 00:00:08

The command write underscore db will create a directory that contains all of the design data.

Time(HH:MM:SS) : 00:00:14

To load a saved database, run the read underscore db command.

Time(HH:MM:SS) : 00:00:19

Here are files that are saved in the design directory, including placement, floor planning, and route files.

VideoTitle :References

Time(HH:MM:SS) : 00:00:00

Here is a list of important input files required for implementation.

Time(HH:MM:SS) : 00:00:04

Some files are design related files, such as the netlist and constraints.

Time(HH:MM:SS) : 00:00:10

Other files, for example the LEF files, sharc technology files and Lib files are technology dependent and are provided by the foundry.

VideoTitle :Labs

Time(HH:MM:SS) : 00:00:00

This is the design implementation flow.

Time(HH:MM:SS) : 00:00:03

The steps in the flow start with design initialization where you import the design, create and load a floor plan and set up your timing.

Time(HH:MM:SS) : 00:00:12

Next is the priest's flow which includes scan definition, Jtag and cell placement, and also priest's setup optimization.

Time(HH:MM:SS) : 00:00:21

The post CTS flow start with clock tree synthesis and includes post CTS setup optimization.

Time(HH:MM:SS) : 00:00:29

Next is post CTS hold optimization.

Time(HH:MM:SS) : 00:00:33

After that you run the post route flow, which includes detail routing and optimization for both base and see delays for setup and hold.

Time(HH:MM:SS) : 00:00:41

Finally, there is the sign off stage where you add metal fill, run physical verification, and run final sign off timing analysis.

Time(HH:MM:SS) : 00:00:50

Notice all along the way you do run timing analysis whether you are in pre CTS, post CTS or post route.

VideoTitle :The Implementation Stage

Time(HH:MM:SS) : 00:00:00

In order to import the design interactively, select File Import Design.

Time(HH:MM:SS) : 00:00:06

This will bring up the design import form.

Time(HH:MM:SS) : 00:00:09

Here is where you enter in the Verilog files for your design.

Time(HH:MM:SS) : 00:00:13

The LEF files for place and route.

Time(HH:MM:SS) : 00:00:16

The I o assignment file that contains location information about where the I OS are placed around the core area.

Time(HH:MM:SS) : 00:00:23

Power and ground nets.

Time(HH:MM:SS) : 00:00:25

The CPF file, which is the common power format file for low power design.

Time(HH:MM:SS) : 00:00:31

And finally, the MK view definition file, which is used to specify the timing libraries and SDC constraints.

Time(HH:MM:SS) : 00:00:40

In order to import the design interactively, select File Import Design.

Time(HH:MM:SS) : 00:00:46

This will bring up the design import form.

Time(HH:MM:SS) : 00:00:49

Here is where you enter in the Verilog files for your design.

Time(HH:MM:SS) : 00:00:53

The LEF files for place and route.

Time(HH:MM:SS) : 00:00:56

The I o assignment file that contains location information about where the I OS are placed around the core area.

Time(HH:MM:SS) : 00:01:02

Power and ground nets.

Time(HH:MM:SS) : 00:01:05

The CPF file, which is the common power format file for low power design.

Time(HH:MM:SS) : 00:01:11

And finally, the MK view definition file, which is used to specify the timing libraries and SDC constraints.

Time(HH:MM:SS) : 00:01:20

Instead of interactively importing the design files, you can use Equivalent.

Time(HH:MM:SS) : 00:01:24

commands in a tickle script and source it to initialize the design.

Time(HH:MM:SS) : 00:01:29

After you import the design, if you click the floor plan view, you will see the pink Module, guide on the left hand side that contains the standard cells.

Time(HH:MM:SS) : 00:01:39

The I o around the periphery.

Time(HH:MM:SS) : 00:01:41

If it's a top level design, a core area and hard macros are custom blocks on the right hand side.

Time(HH:MM:SS) : 00:01:49

The command to resize the main window is GUI.

Time(HH:MM:SS) : 00:01:53

underscore set underscore UI.

VideoTitle :Module Objectives

Time(HH:MM:SS) : 00:00:00

Here is how you set up the Analysis view.

Time(HH:MM:SS) : 00:00:03

You start by creating a library set.

Time(HH:MM:SS) : 00:00:06

If you want to use an operating condition that's outside of the characterized operating condition, you can create an operating condition and have the tool interpolate.

Time(HH:MM:SS) : 00:00:15

You create a timing condition and an RC corner to specify the paths to the extraction libraries, which aggregate to a

delay corner.

Time(HH:MM:SS) : 00:00:24

On the right hand side, you see the Constraint mode that specify the Sdsc is to use for a particular analysis view.

Time(HH:MM:SS) : 00:00:32

The delay calculation corner and the Constraint mode together comprise the analysis view.

VideoTitle :Implementation Flowchart

Time(HH:MM:SS) : 00:00:00

Here are the commands to set up Analysis.

Time(HH:MM:SS) : 00:00:02

views in Innovus.

Time(HH:MM:SS) : 00:00:04

They associate timing Libraries in timing conditions, RC and delay corners, as well as the constraints to an analysis view.

Time(HH:MM:SS) : 00:00:13

Multiple analysis views can be set and used for a particular design for both timing analysis and optimization.

Time(HH:MM:SS) : 00:00:21

This script contains an example of how to create and set analysis views by first setting library sets, timing conditions, RC corners, and constraint modes.

VideoTitle :What is Digital Implementation

Time(HH:MM:SS) : 00:00:00

Floor planning is a process of deriving the die size, allocating spaces for soft blocks, planning power and macro placement.

Time(HH:MM:SS) : 00:00:09

In this example, we have a ten by ten micron die size.

Time(HH:MM:SS) : 00:00:14

The I O's have been placed around the periphery, and if this was a block level design instead of pads, these would be pins for hierarchical implementation.

Time(HH:MM:SS) : 00:00:24

Soft blocks A, B, and C have been created.

Time(HH:MM:SS) : 00:00:28

Optionally, you can place the critical macros such as the Rams in this example.

Time(HH:MM:SS) : 00:00:34

Perform power planning which is defining the global Vdd and VSS.

Time(HH:MM:SS) : 00:00:38

Perform macro placement check for early route congestion.

Time(HH:MM:SS) : 00:00:42

Check for early block utilization.

Time(HH:MM:SS) : 00:00:45

Power domain.

Time(HH:MM:SS) : 00:00:46

definition and flip chip bump placement.

Time(HH:MM:SS) : 00:00:49

These are the tasks for floor planning.

VideoTitle :Starting the software

Time(HH:MM:SS) : 00:00:00

To create a floor plan.

Time(HH:MM:SS) : 00:00:01

Select floor plan.

Time(HH:MM:SS) : 00:00:03

Specify floor plan.

Time(HH:MM:SS) : 00:00:05

Here is where you change the core utilization or specify certain floor plan defaults.

Time(HH:MM:SS) : 00:00:11

We will go over how to resize the floor plan.

Time(HH:MM:SS) : 00:00:13

How to create rows.

Time(HH:MM:SS) : 00:00:15

Use the floor planning toolbox and other floor planning functions.

Time(HH:MM:SS) : 00:00:20

This is the specify floor plan form and the basic tab.

Time(HH:MM:SS) : 00:00:24

The basic tab lets you specify what size the core should be by either specifying the ratio of the height to width, the aspect ratio, and the utilization, and then the tool has enough information to derive the core size.

Time(HH:MM:SS) : 00:00:37

This cell utilization we will get to later.

Time(HH:MM:SS) : 00:00:40

You can also specify dimensions of width and height of the core area, or the die area's width and height.

Time(HH:MM:SS) : 00:00:47

The core margins let you allow space between the core area and the I o, where you may want to create power rings for the core.

VideoTitle :Demo of the Innovus User Interface

Time(HH:MM:SS) : 00:00:01

You can assign IO pads or Pin locations by either reading in a DEF file, or by reading in an Innovus IO file.

Time(HH:MM:SS) : 00:00:09

The Innovus IO file format supports top level designs with pads, rectilinear block level designs, and I o ordering that is organized clockwise, counterclockwise or left to right, bottom to top when edges are specified.

Time(HH:MM:SS) : 00:00:25

Edge zero is the starting point for the IO assignment, and edge zero is the y zero that's on the leftmost side, which is in this case where you see this arrow pointing to.

Time(HH:MM:SS) : 00:00:38

If you do not specify an IO assignment file when you import a design, EOS will be assigned randomly by the tool.

Time(HH:MM:SS) : 00:00:45

You can then dump out an IO assignment file and then modify it and read it back in.

Time(HH:MM:SS) : 00:00:54

This is the IO file example for pad placement.

Time(HH:MM:SS) : 00:00:57

Refer to the data prep section of the user guide for examples of Pin placements for modules and area IOs.

VideoTitle :Running in Batch Mode

Time(HH:MM:SS) : 00:00:01

Power planning is a process of designing a global power distribution network that supplies sufficient power to all the instances of the design.

Time(HH:MM:SS) : 00:00:10

Power planning includes sizing the power wires and choosing the metal layers necessary to deliver the required power to different parts of the chip.

Time(HH:MM:SS) : 00:00:19

If you have inadequate power distribution, it can affect chip timing due to excessive rail voltage drop or air drop and ground bounce.

Time(HH:MM:SS) : 00:00:28

It can also lead to device failure due to M effects.

Time(HH:MM:SS) : 00:00:33

The effects of air drop and other power related issues can be limited by a good power grid design and sufficient Vdd and VSS pads.

Time(HH:MM:SS) : 00:00:46

There are several types of power planning structures.

Time(HH:MM:SS) : 00:00:49

One of them is rings and stripes.

Time(HH:MM:SS) : 00:00:51

Each block has its own ring structure, and each block has stripes that connect the top level to the block.

Time(HH:MM:SS) : 00:00:58

Rings can be shared between a butted blocks.

Time(HH:MM:SS) : 00:01:00

Changes in the design may require changes to the power structure.

Time(HH:MM:SS) : 00:01:04

The Rings and Stripes methodology requires less routing resources when compared to other methodologies, such as power mesh methodology.

Time(HH:MM:SS) : 00:01:15

Another methodology is power mesh methodology.

Time(HH:MM:SS) : 00:01:19

It includes a robust and redundant power network and is seen in microprocessors and large Asics.

Time(HH:MM:SS) : 00:01:25

The primary distribution is through upper Metal layers.

Time(HH:MM:SS) : 00:01:29

Power meshes consume more power than other methodologies such as rings and stripes.

VideoTitle :Input files and Database Structure

Time(HH:MM:SS) : 00:00:00

Choose power.

Time(HH:MM:SS) : 00:00:01

Power planning.

Time(HH:MM:SS) : 00:00:02

Add ring in order to add the core rings.

Time(HH:MM:SS) : 00:00:05

You can contour around core boundaries, selected objects, or along I o boundaries.

Time(HH:MM:SS) : 00:00:12

You can specify the layer, the width, the spacing and the offset to draw the core rings.

VideoTitle :Reading and Writing Innovus System Design Database

Time(HH:MM:SS) : 00:00:00

In order to add stripes, select power power planning, add stripe and that will bring up the add stripe form.

Time(HH:MM:SS) : 00:00:08

The command is add underscore stripes.

Time(HH:MM:SS) : 00:00:10

And just as for rings, you need to specify the net names, the layers, the width, as well as the spacing.

Time(HH:MM:SS) : 00:00:18

You also specify a pattern because it's a repeating pattern across the dye area.

Time(HH:MM:SS) : 00:00:24

So you specify the distance between sets of Vdd and VSS nets.

Time(HH:MM:SS) : 00:00:29

The set to set distance is defined as the distance between the left edge of the first net of the first set, to the next set's left edge.

VideoTitle :Significance of Input files

Time(HH:MM:SS) : 00:00:00

In a previous module, we talked about how to create rings and stripes in your design, and that's called power planning.

Time(HH:MM:SS) : 00:00:07

In this module we talk about how to complete the power connections and that is called power routing.

Time(HH:MM:SS) : 00:00:13

During this stage you complete the power connections to standard cells blocks and I o pads.

Time(HH:MM:SS) : 00:00:20

Here is what follow pin power routing looks like.

Time(HH:MM:SS) : 00:00:23

Here you see that the Vdd and the VSS of the standard cells are connected together.

VideoTitle :Flowchart of Implementation

Time(HH:MM:SS) : 00:00:00

In order to start power routing.

Time(HH:MM:SS) : 00:00:02

Select route.

Time(HH:MM:SS) : 00:00:03

Special route.

Time(HH:MM:SS) : 00:00:05

You can allow jogging, which is to allow the router to use a non-preferred direction for that particular layer, and you can also allow layer change that lets the tool go to a different layer with a V in order to make the power route.

Time(HH:MM:SS) : 00:00:19

You can control the top and bottom layers by selecting the names of the top and bottom layers to use.

Time(HH:MM:SS) : 00:00:25

In the example, you see the results of power planning, which is the addition of power rings and power stripes.

Time(HH:MM:SS) : 00:00:32

You also see the results of power routing, which is the addition of power rails.

Time(HH:MM:SS) : 00:00:38

Power rails are also known as follow pins.

VideoTitle :Importing the Design

Time(HH:MM:SS) : 00:00:00

The purpose of adding scan chains is to make the flip flops in the design controllable and observable.

Time(HH:MM:SS) : 00:00:06

In scan mode, the flip flops function as a shift register, and you can compare the scanned out values with expected values.

VideoTitle :Setting Up Analysis Views

Time(HH:MM:SS) : 00:00:00

If you reorder your scan chain, then you use fewer routing resources as compared to the original routing that was specified before scan reorder.

Time(HH:MM:SS) : 00:00:09

After scan reorder, you see that the net length has been minimized, and it also results in fewer routing resources being used.

VideoTitle :Setting up Analysis views

Time(HH:MM:SS) : 00:00:00

The Scan chains menu is under place Scan chains, which you can use to delete an existing scan chain or to reorder a scan chain.

Time(HH:MM:SS) : 00:00:09

Scan reordering is the process of reconnecting the elements in Scan chains to optimize their routing.

Time(HH:MM:SS) : 00:00:15

This process improves timing and congestion.

Time(HH:MM:SS) : 00:00:19

Typically, scan insertion is performed in the synthesis tool, whereas scan reordering is performed during placement or as a separate step during implementation.

VideoTitle :What is Floorplanning

Time(HH:MM:SS) : 00:00:00

Running placement while ignoring scan chain connections reduces routing congestion and wire lengths.

Time(HH:MM:SS) : 00:00:07

Scan cells are identified in the timing library or the dot lib file.

Time(HH:MM:SS) : 00:00:12

If scan cells are not in the timing library, use this command to load the information.

Time(HH:MM:SS) : 00:00:18

The most common method of reading and scan chain is by reading in a DEF file, commonly known as a scan DEF.

Time(HH:MM:SS) : 00:00:25

If you do not have a scan DEF file, you can instead load the scan chain with a TCL file containing scan chain and information with the create underscore scan underscore chain command.

Time(HH:MM:SS) : 00:00:37

This is an example of a scan DEF file written out by our synthesis tool.

Time(HH:MM:SS) : 00:00:42

In this example there are two scan chains auto chain one and auto chain two.

Time(HH:MM:SS) : 00:00:48

The information that's relevant to scans are the start pin and the stop pin information.

VideoTitle :Specifying a Floorplan

Time(HH:MM:SS) : 00:00:00

Placement is a process of placing the standard cells and blocks in a floor plan.

Time(HH:MM:SS) : 00:00:04

Design.

Time(HH:MM:SS) : 00:00:06

When a design is read in, the tool creates rows for the standard cells to be placed into by the placer.

Time(HH:MM:SS) : 00:00:13

A row is a multiple of a site that is defined in the DEF file.

Time(HH:MM:SS) : 00:00:17

There are several different types of rows, for example standard cell rows and I o rows.

VideoTitle :Creating an IO File

Time(HH:MM:SS) : 00:00:00

Optimization is the process of iterating through a design such that it meets timing area and power requirements.

Time(HH:MM:SS) : 00:00:08

In general, optimization can be broken down into these four areas.

Time(HH:MM:SS) : 00:00:13

Timing.

Time(HH:MM:SS) : 00:00:14

See power and area.

Time(HH:MM:SS) : 00:00:16

Optimization.

Time(HH:MM:SS) : 00:00:18

The design has to satisfy these multiple design objectives depending on the stage of the design.

Time(HH:MM:SS) : 00:00:24

Optimization can include the following operations.

Time(HH:MM:SS) : 00:00:27

Adding buffers.

Time(HH:MM:SS) : 00:00:29

Resizing cells.

Time(HH:MM:SS) : 00:00:30

Restructuring the netlist.

Time(HH:MM:SS) : 00:00:32

Remapping logic.

Time(HH:MM:SS) : 00:00:33

Swapping pins.

Time(HH:MM:SS) : 00:00:35

Deleting buffers.

Time(HH:MM:SS) : 00:00:36

Moving instances.

Time(HH:MM:SS) : 00:00:37

Applying useful skew layer optimization and track optimization.

VideoTitle :What is Power Planning

Time(HH:MM:SS) : 00:00:00

One of the main features of the placement optimization engine is that placement and optimization are interleaved.

Time(HH:MM:SS) : 00:00:07

The initial placement, incremental placement, and the several passes of incremental placement are all followed by optimization.

Time(HH:MM:SS) : 00:00:15

In the previous flow, there was a run of placement followed by optimization instead of giga place, which interleaves both placement and optimization.

VideoTitle :Adding Rings

Time(HH:MM:SS) : 00:00:00

Clocks are used to synchronize data communication.

Time(HH:MM:SS) : 00:00:03

In ideal mode, which is before a clock tree is built.

Time(HH:MM:SS) : 00:00:07

There is equal delay from the clock source to sinks.

Time(HH:MM:SS) : 00:00:11

You can create clock trees that have global skew balancing, which means that the delay from clock source to sinks are equal within some tolerance.

Time(HH:MM:SS) : 00:00:20

Optionally, you can use useful skew, which means that the clock path delays are adjusted to assist timing.

Time(HH:MM:SS) : 00:00:25

Slack optimization.

Time(HH:MM:SS) : 00:00:28

Strict balancing is not required, and the skew can be exploited to save clock area and power.

VideoTitle :Adding Stripes

Time(HH:MM:SS) : 00:00:00

You can generate the CCOpt clock tree spec file by running the command, create_clock_tree_spec.

Time(HH:MM:SS) : 00:00:07

The spec file is automatically generated by the ccopt_design command if you do not explicitly generate it using the create_clock_tree_spec command.

VideoTitle :What is Power Routing

Time(HH:MM:SS) : 00:00:00

You can generate the CC opt clock tree spec file by running the command Create clock tree spec.

Time(HH:MM:SS) : 00:00:07

The spec file is automatically generated by the clock underscore opt underscore design command if you do not explicitly generate it using the create clock tree spec command.

VideoTitle :Routing Power with Special Route

Time(HH:MM:SS) : 00:00:02

Bring up the clock tree debugger to view the clock trees and skew groups, and to cross probe between the debugger window, the physical view, the timing report and the schematic.

VideoTitle :What are Scan Chains

Time(HH:MM:SS) : 00:00:00

This demonstration will show you some of the features of the Clock Tree debugger.

Time(HH:MM:SS) : 00:00:05

Once you have created a clock tree, you can bring up the Clock Tree Debugger by selecting clock Cop.

Time(HH:MM:SS) : 00:00:11

Clock tree debugger.

Time(HH:MM:SS) : 00:00:17

Next, select view enable clock path browser.

Time(HH:MM:SS) : 00:00:21

Move it away from the main display window for now.

Time(HH:MM:SS) : 00:00:24

We will come back to it a little bit later.

Time(HH:MM:SS) : 00:00:29

Zoom into a leaf cell by clicking the right mouse button and dragging.

Time(HH:MM:SS) : 00:00:36

When you place your cursor over a leaf cell, it tells you some of the properties of the cell.

Time(HH:MM:SS) : 00:00:43

The numbers on the left is the time scale.

Time(HH:MM:SS) : 00:00:49

If you click and select a cell and go back to the physical window, you will see that that cell has been zoomed into.

Time(HH:MM:SS) : 00:01:00

Press F12 in order to dim the background so that you can see the cell better.

Time(HH:MM:SS) : 00:01:08

This feature helps you cross probe between the physical view and the clock tree.

Time(HH:MM:SS) : 00:01:13

Debugger.

Time(HH:MM:SS) : 00:01:14

A visualization window, you can press F on your keyboard to again fit all of the clock trees into this window.

Time(HH:MM:SS) : 00:01:24

This is the key tab and you can expand it.

Time(HH:MM:SS) : 00:01:28

The key tab lets you see what the colors and shapes map to in the main visualization window.

Time(HH:MM:SS) : 00:01:35

This is a view only window.

Time(HH:MM:SS) : 00:01:39

The visibility pulldown lets me select what objects I want visible, so I can select only certain clocks to be visible depending on what I'm trying to look at.

Time(HH:MM:SS) : 00:01:53

I can color by various criteria such as cell types, pin types, power domains, skew groups, and so on.

Time(HH:MM:SS) : 00:02:08

On the right hand side is a control panel, and here I can also select what I want to look at.

Time(HH:MM:SS) : 00:02:17

I can control the color of the various clock trees.

Time(HH:MM:SS) : 00:02:33

I can also color by transition time max transition violation, which I guess in this design there isn't one.

Time(HH:MM:SS) : 00:02:42

Net lengths.

Time(HH:MM:SS) : 00:02:45

Net length, violations..

Time(HH:MM:SS) : 00:02:46

and so on.

Time(HH:MM:SS) : 00:02:49

So now let's go back to the clock path browser.

Time(HH:MM:SS) : 00:02:53

So here I want to find what the maximum path is.

Time(HH:MM:SS) : 00:02:57

So I'm going to be selecting this queue group highlight by max path.

Time(HH:MM:SS) : 00:03:04

And I'm going to highlight with a color of red.

Time(HH:MM:SS) : 00:03:08

So this is the max path I can similarly find the min path.

Time(HH:MM:SS) : 00:03:19

Colored in blue.

Time(HH:MM:SS) : 00:03:24

Once I've highlighted the min and max path, I can go to the physical view and I see that the same colors are used to highlight the physical paths that correspond to what is highlighted in the clock browser.

Time(HH:MM:SS) : 00:03:37

So these have been some of the features of the clock tree debugger.

VideoTitle :What is Scan Reordering

Time(HH:MM:SS) : 00:00:00

Routing is the part of the implementation flow where the cells and macros are connected on metal layers, using the rules specified in the library exchange format or left file.

Time(HH:MM:SS) : 00:00:10

The router will make sure that the connections are DRC correct and the routing is timing and signal integrity aware.

Time(HH:MM:SS) : 00:00:18

The technology specific LEC file is most often provided by the foundry.

VideoTitle :Deleting and Reordering Scan Chains

Time(HH:MM:SS) : 00:00:00

This is the routing flow.

Time(HH:MM:SS) : 00:00:02

We start with global route where the design is divided into G cells and routes are assigned to tracks.

Time(HH:MM:SS) : 00:00:09

The congestion is analyzed.

Time(HH:MM:SS) : 00:00:11

Detail routing creates the routes and applies the full geometry rules and gives priority to critical nets.

Time(HH:MM:SS) : 00:00:18

The search and repair steps are surgical fixes for individual vias on wires.

Time(HH:MM:SS) : 00:00:24

Search areas are increased by three by three, five by five, and seven by seven.

Time(HH:MM:SS) : 00:00:29

In G cell areas, deletion and rerouting is a last resort if no other solution is found.

VideoTitle :Loading Scan Chain Information

Time(HH:MM:SS) : 00:00:00

To set the routing Attribute.

Time(HH:MM:SS) : 00:00:01

to timing driven.

Time(HH:MM:SS) : 00:00:02

Set DB route with timing driven true.

Time(HH:MM:SS) : 00:00:06

For C driven, the attribute to set is route with C driven true.

Time(HH:MM:SS) : 00:00:12

Run route underscore design to run global and detailed timing and C driven routing to route only selected nets.

Time(HH:MM:SS) : 00:00:20

Run route.

Time(HH:MM:SS) : 00:00:21

Underscore design.

Time(HH:MM:SS) : 00:00:22

Hyphen selected.

VideoTitle :What is Placement

Time(HH:MM:SS) : 00:00:00

Filler cells are used to fill the gaps or spaces between standard cell instances to provide continuity for the power and ground rails and end wells.

Time(HH:MM:SS) : 00:00:10

The fillers can be added interactively or by running the add underscore fillers command.

VideoTitle :What is Optimization

Time(HH:MM:SS) : 00:00:00

You can save the extracted data in several formats by selecting timing extract RC or by running the command extract underscore RC.

Time(HH:MM:SS) : 00:00:09

The most common is the Spef format.

Time(HH:MM:SS) : 00:00:12

Run the reset parasitics.

Time(HH:MM:SS) : 00:00:14

command to delete the extracted parasitic values, but keep the extraction attributes.

VideoTitle :Placement and Optimizat

Time(HH:MM:SS) : 00:00:00

What are DRC and LVS?

Time(HH:MM:SS) : 00:00:03

Drc are design rule checks that are flagged as violations and that need to be fixed or waived before tapeout.

Time(HH:MM:SS) : 00:00:11

Lvs is layout versus schematic and these errors occur where there is a mismatch in connectivity and open or a short for example, between what is placed and routed and what's in the Verilog netlist.

Time(HH:MM:SS) : 00:00:24

When DRC checks are run using place and route data, the models of the cells are in leaf or abstract format.

Time(HH:MM:SS) : 00:00:32

Layouts are not used because they contain more detail than the place and route tool needs to have.

Time(HH:MM:SS) : 00:00:38

The advantage of running DRC with a small data set is that it's fast, because it's working on a small database, and problems can be debugged and fixed fast in the place and route tool instead of going to a custom tool and fixing your violations...

Time(HH:MM:SS) : 00:00:53

However, there are some disadvantages that the checks are inaccurate because they are abstracts and they are not the GDS two or the layouts of the cells.

Time(HH:MM:SS) : 00:01:03

There aren't checks at the different levels of hierarchy, and the connections to pins could have violations...

VideoTitle :What Is Clock Tree Synthesis

Time(HH:MM:SS) : 00:00:00

Choose check check connectivity to report open nets, antennas, loops, and partial routing for all the nets or specified nets in your design.

Time(HH:MM:SS) : 00:00:11

This is a form that comes up when you select check Check connectivity.

Time(HH:MM:SS) : 00:00:16

If there are Violations..

Time(HH:MM:SS) : 00:00:17

violation markers are generated and in this example it's for open or unconnected rails.

Time(HH:MM:SS) : 00:00:23

You can choose Tools Violation browser.

Time(HH:MM:SS) : 00:00:26

And that will bring up a window of the DRC violations.

Time(HH:MM:SS) : 00:00:30

And you can click and zoom to those violations.

Time(HH:MM:SS) : 00:00:33

The command is check underscore connectivity.

VideoTitle :Automatically Generating a CTS Spec file

Time(HH:MM:SS) : 00:00:00

Select check.

Time(HH:MM:SS) : 00:00:01

Check DRC to check for rule violations that are specified in the tech LEF.

Time(HH:MM:SS) : 00:00:07

If there are violations, there will be DRC error markers in the physical view and details will be displayed in the violation browser.

VideoTitle :Clock Tree Synthesis Flow ,

Time(HH:MM:SS) : 00:00:00

After running power planning, you can run power and rail analysis.

Time(HH:MM:SS) : 00:00:05

Select power Power Analysis Setup in order to set up for power and rail analysis, and then run in order to run the analysis.

Time(HH:MM:SS) : 00:00:14

To run an air drop analysis, which will give you a map of the hotspots where the most air drop is in your design.

Time(HH:MM:SS) : 00:00:21

Select rail analysis and then follow the steps that are shown in this form.

Time(HH:MM:SS) : 00:00:26

This is the flow for early rail analysis and power planning.

Time(HH:MM:SS) : 00:00:30

First, you create an initial floor plan.

Time(HH:MM:SS) : 00:00:33

Next, you build a power grid, run an IR and Em analysis to see if there are areas like you see here that have unacceptable air drops or em violations..

Time(HH:MM:SS) : 00:00:45

You will refine your floor plan and your power grid.

Time(HH:MM:SS) : 00:00:48

Then we run your IR and Em analysis, and you see that the red has changed to yellow and green, which means that the IR drop and Em are now in acceptable ranges.

VideoTitle :Debugging the Clock Tree with the Clock Tree Debugger,

Time(HH:MM:SS) : 00:00:00

To write out a GDS two file, you need a GDS two map file that contains the layer name and the Equivalent.

Time(HH:MM:SS) : 00:00:07

layer number.

Time(HH:MM:SS) : 00:00:08

Here is an example of a layer map file.

Time(HH:MM:SS) : 00:00:11

The layer number is used by the custom tools such as virtuoso to interpret the layers in the same way as Innovus.

Time(HH:MM:SS) : 00:00:19

In addition to writing out the design in GDS two format, you can also include the GDS two library containing the cells and the macros in the design.

Time(HH:MM:SS) : 00:00:29

This feature is useful if you are not planning on using a custom tool for the chip finishing step, and this way you can write out both the netlist as well as the cells and the macros in GDS two format from Innovus.

VideoTitle :Demo How to Use Clock Tree Debugger in Innovus Stylus Common UI

Time(HH:MM:SS) : 00:00:00

Here are some additional reference materials for Innovus.

VideoTitle :What is Routing

Time(HH:MM:SS) : 00:00:00

It's time now to run the labs.

VideoTitle :Routing Flow

Time(HH:MM:SS) : 00:00:01

Module 9 gate level simulation or GLS.

Time(HH:MM:SS) : 00:00:06

In this module, you will identify the process of gate level simulation or GLS and describe the reason for performing it in the design flow.

Time(HH:MM:SS) : 00:00:15

You will identify the concept of SDF annotation.

Time(HH:MM:SS) : 00:00:20

You will invoke the Xcelium simulator and then simulate the gate level netlist of a very simple counter design.

VideoTitle :Timing SI and Power-Driven Routing

Time(HH:MM:SS) : 00:00:01

This is the Module, nine gate level simulation flowchart.

Time(HH:MM:SS) : 00:00:05

So I have highlighted the gate level simulation phase in this flowchart.

Time(HH:MM:SS) : 00:00:10

So this takes the input from the static timing analysis phase and then runs the simulation taking into account the timing information it gets from the STA process.

Time(HH:MM:SS) : 00:00:20

The next few slides in this module will discuss in detail what is GLS.

Time(HH:MM:SS) : 00:00:25

What is SDF standard delay format annotation and how to perform the gate level simulation using the Xcelium simulator tool.

VideoTitle :Adding Filler cells Basic Tab

Time(HH:MM:SS) : 00:00:01

What is gate level simulation or GLS for short?

Time(HH:MM:SS) : 00:00:05

Gls is a step in the design flow to ensure that the design meets the functionality after synthesis, or after placement and routing activities.

Time(HH:MM:SS) : 00:00:14

The motivation for running the GLS is it gives confidence in verification of low power structures, which is absent in register transfer level or the RTL, and added during the synthesis.

Time(HH:MM:SS) : 00:00:26

It is a method which can catch multi cycle paths.

Time(HH:MM:SS) : 00:00:29

Multi cycle paths are the ones in the design which take more than a single clock cycle.

Time(HH:MM:SS) : 00:00:35

The tools by default take all the paths to be a single cycle operation.

Time(HH:MM:SS) : 00:00:40

So multi cycle paths are a challenge in the design.

Time(HH:MM:SS) : 00:00:43

And this is a method which can catch these paths if tests exercising them are available to verify DFT structures, which is absent in RTL and is added during or after synthesis.

Time(HH:MM:SS) : 00:00:54

The GLS is performed.

Time(HH:MM:SS) : 00:00:56

Scan chains are generally inserted after the gate level netlist has been added has been created.

Time(HH:MM:SS) : 00:01:03

Hence gate level simulations are often used to determine whether scan chains are correct or not.

Time(HH:MM:SS) : 00:01:09

Gls is also required to simulate the atpg patterns.

Time(HH:MM:SS) : 00:01:13

Atpg is the short form for automatic test pattern generation.

Time(HH:MM:SS) : 00:01:18

Let's go over a little bit more about the history of simulations, and what is the industry standard and the norm followed in the EDA industry right now?

Time(HH:MM:SS) : 00:01:27

Simulations are generally an important part of the verification cycle.

Time(HH:MM:SS) : 00:01:31

In the process of hardware designing, it can be performed at varying degrees of physical abstraction, like at the transistor level, the gate level, and the register transfer level, or the RTL, which is what we have dealt with in the second chapter of the module.

Time(HH:MM:SS) : 00:01:46

In many companies, RTL simulation is the basic requirement to sign off the design cycle.

Time(HH:MM:SS) : 00:01:52

But lately there is an increasing trend in the industry to run the gate level simulation, or GLS before going to the last stage of chip manufacturing.

Time(HH:MM:SS) : 00:02:02

Improvements in static verification tools like static timing analysis, STA and Equivalence checking EQ have leveraged GLS to some extent, but so far none of the tools has been able to completely remove the function of GLS.

Time(HH:MM:SS) : 00:02:17

So gate level simulation still remain a significant step of the verification cycle footprint.

Time(HH:MM:SS) : 00:02:23

Here in this module, we focused mainly on the steps to run the GLS from verification perspective and the challenges faced while running it.

Time(HH:MM:SS) : 00:02:34

In this slide, I have shown a screenshot of the waveform from the SIM vision tool, wherein the gate level simulations are run on the counter design and the back annotated delay is applied and shown in the simulation.

Time(HH:MM:SS) : 00:02:46

How it makes a difference, and we will be looking at it in detail later.

Time(HH:MM:SS) : 00:02:51

But what we need to know at this point of time is as an input, we need the synthesized post routed netlist.

Time(HH:MM:SS) : 00:02:57

The first thing to run the GLS, the Testbench, the second thing, and the standard delay format file or the SDF file, which is the third thing for running this gate level simulation.

Time(HH:MM:SS) : 00:03:09

Let's look at more about this GLS in the next few slides.

VideoTitle :Extracting RC Data Interactively

Time(HH:MM:SS) : 00:00:01

What is SDF annotation or standard delay format annotation?

Time(HH:MM:SS) : 00:00:06

Standard Delay Format, or SDF, is an IEEE standard for the representation and interpretation of timing data for use at any stage of an electronic design process.

Time(HH:MM:SS) : 00:00:16

This uses the SDF file for performing the annotation process.

Time(HH:MM:SS) : 00:00:21

The SDF annotator is a tool which uses the SDF file as input for the annotation process.

Time(HH:MM:SS) : 00:00:28

The job of the annotator is to match data on the SDF file with the design, description and the timing models.

Time(HH:MM:SS) : 00:00:35

The Xcelium simulator tool is used as the tool for SDF annotation, as well as for running the gate level simulations or GLS.

Time(HH:MM:SS) : 00:00:43

Here is a Testbench, the counter Test.csv, which we will be looking at as an example in the next several slides to come.

Time(HH:MM:SS) : 00:00:50

And this is the modification I have to make in my testbench in order to add the back annotated delay, which is gotten from the SDF process.

Time(HH:MM:SS) : 00:00:59

And after adding this in the testbench, I run the simulation.

Time(HH:MM:SS) : 00:01:03

This is the GLS process.

Time(HH:MM:SS) : 00:01:08

What is SDF file?

Time(HH:MM:SS) : 00:01:10

The contents of this file identify regions of the design and provide timing data that applies to the timing properties of that region.

Time(HH:MM:SS) : 00:01:18

The SDF file supports hierarchical delay annotation.

Time(HH:MM:SS) : 00:01:22

A design hierarchy might include several different application specific integrated circuits or Asics, including cells or blocks within the Asics.

Time(HH:MM:SS) : 00:01:31

Each design hierarchy has its own SDF file.

Time(HH:MM:SS) : 00:01:35

Throughout the design process, you can use several different SDF files.

Time(HH:MM:SS) : 00:01:40

Here is an interpretation a pictorial interpretation of how an the system module,.

Time(HH:MM:SS) : 00:01:44

There are different Asics and different SDF files could be used for each of these ASIC blocks.

Time(HH:MM:SS) : 00:01:53

In this slide, we take a look at how the SDF file format looks like and what are the details that go into this file.

Time(HH:MM:SS) : 00:02:00

The data in the SDF file is represented in a tool independent way and can include the following things.

Time(HH:MM:SS) : 00:02:06

The following items.

Time(HH:MM:SS) : 00:02:07

Delays in the form of module, path interconnect path delays.

Time(HH:MM:SS) : 00:02:12

Timing Checks.

Time(HH:MM:SS) : 00:02:13

is in setup and hold items.

Time(HH:MM:SS) : 00:02:15

Recovery and removal.

Time(HH:MM:SS) : 00:02:16

Skew width and period timing constraints like the path, the skew, the period sum and the diff or the difference timing environment like intended operating timing environment, the incremental and absolute delays, the conditional and unconditional module, path.

Time(HH:MM:SS) : 00:02:34

Delays and timing checks the design instance specific or type library specific data.

Time(HH:MM:SS) : 00:02:42

The scaling the environmental and technology parameters.

Time(HH:MM:SS) : 00:02:46

So an example SDF file format screenshot is given here.

Time(HH:MM:SS) : 00:02:51

This is how typically the SDF file will look like.

Time(HH:MM:SS) : 00:02:57

I had shown earlier the changes to be made in the testbench in order to take in the delay that is gotten after the SDF annotation process.

Time(HH:MM:SS) : 00:03:06

So the task that is used, the system task that is to be included in the Testbench is dollar SDF annotate.

Time(HH:MM:SS) : 00:03:14

This is what I showed in the beginning slide when I started explaining about the SDF annotation process.

Time(HH:MM:SS) : 00:03:21

So there is a small snippet of code here wherein dollar SDF annotate system task is added, and the delays SDF and SDF dot log is generated.

Time(HH:MM:SS) : 00:03:31

And what is the delay that is to be considered maximum or minimum is specified in here.

Time(HH:MM:SS) : 00:03:39

More details about the dollar SDF annotate system task.

Time(HH:MM:SS) : 00:03:44

This is how you have to specify the SDF annotate task.

Time(HH:MM:SS) : 00:03:47

You must specify the arguments to this system task in the order shown in the syntax.

Time(HH:MM:SS) : 00:03:52

On the left side.

Time(HH:MM:SS) : 00:03:54

In the red font, I have shown the syntax.

Time(HH:MM:SS) : 00:03:57

You can skip an argument specification, but the number of comma separators must maintain the argument sequence.

Time(HH:MM:SS) : 00:04:03

For example, to specify only the first and last arguments, you should use the following syntax dollar SDF annotate SDF file scale type.

Time(HH:MM:SS) : 00:04:13

But in between you need to add all the comma separators for module, instance, config file, log file, etc.

Time(HH:MM:SS) : 00:04:21

in order to specify a placeholder.

Time(HH:MM:SS) : 00:04:24

So dollar sdf annotate sdf file is the full or relative path of the SDF file.

Time(HH:MM:SS) : 00:04:30

The Module, instance is optional.

Time(HH:MM:SS) : 00:04:33

It specifies the scope in which the annotation takes place.

Time(HH:MM:SS) : 00:04:37

The config file is also optional.

Time(HH:MM:SS) : 00:04:39

Parameter.

Time(HH:MM:SS) : 00:04:40

This specifies the name of the configuration file specified in quotation marks that the SDF annotator reads before annotating process begins.

Time(HH:MM:SS) : 00:04:50

The log file is also optional.

Time(HH:MM:SS) : 00:04:52

The name of the log file specified in quotation marks that the SDF annotator generates during annotation.

Time(HH:MM:SS) : 00:05:00

The MTM spec.

Time(HH:MM:SS) : 00:05:01

This is also optional.

Time(HH:MM:SS) : 00:05:03

One of the keywords specified in the table in the next slide, indicating the delay values that are annotated to the Verilog family tool that is MTM spec scale factors.

Time(HH:MM:SS) : 00:05:14

This is also optional.

Time(HH:MM:SS) : 00:05:16

The minimum, typical and maximum timing data values are passed to this parameter, specified in quotation marks, expressed as a set of three positive real number multipliers min multipliers.

Time(HH:MM:SS) : 00:05:28

Type multipliers.

Time(HH:MM:SS) : 00:05:30

Max multipliers.

Time(HH:MM:SS) : 00:05:32

For example 1.61.41.2 scale type.

Time(HH:MM:SS) : 00:05:39

This is also optional.

Time(HH:MM:SS) : 00:05:41

One of the following keyword specified in quotation marks to scale the timing specifications in SDF, which are annotated to the Verilog family tool.

Time(HH:MM:SS) : 00:05:53

The values that the MTM spec parameter takes up in the SDF annotate system task.

Time(HH:MM:SS) : 00:05:59

So this I mentioned in the previous slide.

Time(HH:MM:SS) : 00:06:02

Mtm spec.

Time(HH:MM:SS) : 00:06:03

So the keywords you can give to this are maximum.

Time(HH:MM:SS) : 00:06:06

This annotates the maximum delay value.

Time(HH:MM:SS) : 00:06:09

Minimum.

Time(HH:MM:SS) : 00:06:10

This annotates the minimum delay value or tool control, which is default.

Time(HH:MM:SS) : 00:06:14

It annotates the delay value that is determined by the Verilog XL and the default XL.

Time(HH:MM:SS) : 00:06:20

Command line options.

Time(HH:MM:SS) : 00:06:22

Minimum, typical and maximum values are always annotated to Vera time.

Time(HH:MM:SS) : 00:06:27

If none of the tool control command line options are specified, then the default keyword is typical, so typical means it annotates the typical delay value.

Time(HH:MM:SS) : 00:06:40

The values you can provide for the scale factor option, or the parameter that you pass in the SDF annotate system task are.

Time(HH:MM:SS) : 00:06:48

You can provide from maximum keyword which scales from the maximum timing specification.

Time(HH:MM:SS) : 00:06:53

You can provide from minimum keyword which scales from the minimum timing specification from MTM default.

Time(HH:MM:SS) : 00:07:01

So this scales from the minimum, typical and maximum timing specifications.

Time(HH:MM:SS) : 00:07:05

This is the default from typical scales from the typical timing specification.

VideoTitle :What are DRC and LVS

Time(HH:MM:SS) : 00:00:01

This slide shows a very detailed flow chart of the standard delay format annotation process.

Time(HH:MM:SS) : 00:00:07

What are all the different phases that happens in this process?

Time(HH:MM:SS) : 00:00:10

A little bit about the SDF annotator before we start on the flowchart.

Time(HH:MM:SS) : 00:00:15

The SDF annotator operates on many aspects of a design.

Time(HH:MM:SS) : 00:00:19

You can perform separate annotations to distinct hierarchical portions of a single design description.

Time(HH:MM:SS) : 00:00:25

For example, you can annotate from multiple SDF files, each of which corresponds to a separate integrated circuit or IC within a description of a board level design.

Time(HH:MM:SS) : 00:00:36

To call the SDF annotator from a Verilog family tool like Xcelium, you enter the SDF annotate system task at the interactive command line or in the Verilog family tools.

Time(HH:MM:SS) : 00:00:47

Description.

Time(HH:MM:SS) : 00:00:48

The SDF annotate system task specifications take precedence over specifications in the SDF file.

Time(HH:MM:SS) : 00:00:56

The first phase is the Verilog description, which makes the system call using the SDF, annotate and all the

parameters that have to be given to this.

Time(HH:MM:SS) : 00:01:04

I have explained in the previous few slides and the values that has to be taken by these parameters, which is passed to the system task are also explained by now a Verilog family tool like Xcelium responds to this SDF annotate system task which calls the SDF annotator.

Time(HH:MM:SS) : 00:01:23

This task is called from the Testbench.

Time(HH:MM:SS) : 00:01:25

As you can recall, there was a small code snippet I showed in the counter test.csv file where I have called the SDF Annotate system task.

Time(HH:MM:SS) : 00:01:34

Then the SDF annotator checks if there is a configuration file if it exists or not, it reads the configuration file if one exists.

Time(HH:MM:SS) : 00:01:43

The configuration file filters timing data before it is annotated to a Verilog family tool.

Time(HH:MM:SS) : 00:01:49

Then the SDF annotator reads the timing data from the SDF file, which is an Ascii text file that stores the timing data generated by the Verilog family tool.

Time(HH:MM:SS) : 00:01:59

So it reads the configuration file settings.

Time(HH:MM:SS) : 00:02:02

Otherwise, it uses the SDF default settings.

Time(HH:MM:SS) : 00:02:05

Then the SDF annotator tool processes the timing data according to the configuration file commands or the SDF annotator settings.

Time(HH:MM:SS) : 00:02:14

So it processes the timing data in the SDF file.

Time(HH:MM:SS) : 00:02:18

Then, the process data is annotated to the Verilog family tool.

Time(HH:MM:SS) : 00:02:22

That is, the control is passed annotation.

Time(HH:MM:SS) : 00:02:25

It passes the annotation back to the tool that used the SDF.

Time(HH:MM:SS) : 00:02:29

Annotate.

VideoTitle :Checking Connectivity

Time(HH:MM:SS) : 00:00:01

This slide, and the next few slides in this section will explain in detail the exact commands and the options you provide to the Xrun command using the Xcelium tool in order to run the gate level simulations.

Time(HH:MM:SS) : 00:00:14

So the command would be Xrun timescale.

Time(HH:MM:SS) : 00:00:16

You provide the timescale counter netlist, the netlist name, then your testbench counter test dot v v and the timing information file and the access RWC for read write connectivity.

Time(HH:MM:SS) : 00:00:30

Then you define the SDF test.

Time(HH:MM:SS) : 00:00:32

If not defined, then the verbosity with messages.

Time(HH:MM:SS) : 00:00:36

And if you want to invoke the GUI, then GUI.

Time(HH:MM:SS) : 00:00:39

Given a simple counter design and counter V and Testbench counter test V and the relevant testbench with SDF annotation information in it.

Time(HH:MM:SS) : 00:00:49

The delay file and the library v file.

Time(HH:MM:SS) : 00:00:52

Let us execute the above command to perform the gate level simulation.

Time(HH:MM:SS) : 00:00:56

Time scale is used to mention the time unit and time precision.

Time(HH:MM:SS) : 00:01:00

In this case, I had given one nanosecond by ten picoseconds.

Time(HH:MM:SS) : 00:01:05

That is a time unit by time precision.

Time(HH:MM:SS) : 00:01:08

Access is passed to the collaborator to provide the read, write, and connectivity.

Time(HH:MM:SS) : 00:01:12

Access to simulation objects Gui.

Time(HH:MM:SS) : 00:01:15

is used to invoke the Xrun in GUI mode.

Time(HH:MM:SS) : 00:01:19

Messages or mess is used to display all the messages in detail.

Time(HH:MM:SS) : 00:01:22

For the verbosity.

Time(HH:MM:SS) : 00:01:24

Sdf verbose is used to create instance level annotation details.

Time(HH:MM:SS) : 00:01:28

In the SDF log.

Time(HH:MM:SS) : 00:01:30

Define is used to provide SDF definition present in the Testbench.

Time(HH:MM:SS) : 00:01:35

Ve is used to provide the library in VE format.

Time(HH:MM:SS) : 00:01:39

So I have given the command in here in the terminal, and this is how I initiated the gate level simulation by providing this command x runtime scale followed by the netlist V, the testbench v for the library ve file access with RWC.

Time(HH:MM:SS) : 00:01:56

Then define is used to provide the SDF definition present in the Testbench.

Time(HH:MM:SS) : 00:02:01

Then the verbosity and the GUI for invoking the SimVision and.

Time(HH:MM:SS) : 00:02:08

Once you run the command that I showed in the previous slide, then the execution happens and the annotation is completed.

Time(HH:MM:SS) : 00:02:15

You get on your screen the SDF statistics that I have highlighted within this red rectangle.

Time(HH:MM:SS) : 00:02:20

And this is the statistics output of SDF annotation before the control is relinquished to the SimVision and.

Time(HH:MM:SS) : 00:02:27

This shows up on your screen.

Time(HH:MM:SS) : 00:02:32

These are the excerpts from the log file.

Time(HH:MM:SS) : 00:02:34

After the GLS is completed, the delay is introduced due to the I o paths.

Time(HH:MM:SS) : 00:02:40

Input output paths is shown in the SDF log and delays SDF files.

Time(HH:MM:SS) : 00:02:46

So SDF log annotating to instance G7 oh five of the module name absolute I, o path A and Y, and the delay introduced 0.04 is shown here.

Time(HH:MM:SS) : 00:02:58

We would be looking at this detail in the waveform in the next slide, and in the delays SDF you can again see the I o path A and Y and the delay introduced the MTM values of the rising edge and the falling edge from A to Y.

Time(HH:MM:SS) : 00:03:13

So these are the areas highlighted in the red rectangle here.

Time(HH:MM:SS) : 00:03:17

So this is the example which shows an instance G 705 of inverter inv1 which has two signals A and y.

Time(HH:MM:SS) : 00:03:25

And now the delay is introduced between these two.

Time(HH:MM:SS) : 00:03:31

This slide and the next few slides show how to run the gate level simulations or the GLS using SIM vision tool.

Time(HH:MM:SS) : 00:03:38

Sim vision is a GUI based tool used to run both RTL as well as gate level simulations or GLS.

Time(HH:MM:SS) : 00:03:46

The different windows which are profusely used for this purpose are the console window.

Time(HH:MM:SS) : 00:03:51

This is used to run the simulation.

Time(HH:MM:SS) : 00:03:54

The design browser window is used to analyze the signals in the design and also look at the hierarchy.

Time(HH:MM:SS) : 00:04:01

The waveform we do is used to pull all the required signals from the design browser into the window, then view the waveform and analyze the timing delay as a result of the history of annotation.

Time(HH:MM:SS) : 00:04:12

The different steps that are involved in running the gate level simulation.

Time(HH:MM:SS) : 00:04:16

Using SimVision and tool R, you start the SIM vision tool, you start the console window.

Time(HH:MM:SS) : 00:04:22

Then you use the Design browser window to analyze the different signals of the design and the hierarchy.

Time(HH:MM:SS) : 00:04:28

You select the required signals from the design browser, pull them into the waveform window.

Time(HH:MM:SS) : 00:04:34

You run the simulation in the console window, view the signals as well as the timing delay in the waveform window, and also view the instance level signals in the waveform window.

Time(HH:MM:SS) : 00:04:47

More information is provided on your screen before the SIM vision opens.

Time(HH:MM:SS) : 00:04:51

The timing checks the interconnect.

Time(HH:MM:SS) : 00:04:53

Delayed T Checks.

Time(HH:MM:SS) : 00:04:54

signals the simulation time scale.

Time(HH:MM:SS) : 00:04:57

Everything is provided under the design hierarchy summary.

Time(HH:MM:SS) : 00:05:01

Then the control is relinquished to the SIM vision tool.

Time(HH:MM:SS) : 00:05:05

Then the design browser opens as well as the console and SIM vision.

Time(HH:MM:SS) : 00:05:09

The SIM vision starts with these two windows, and in the console window you need to provide the force signals force DFT signals such as scan, enable, scan in and scan out and give the values as zero in here to start your gate level simulation process.

Time(HH:MM:SS) : 00:05:29

In the Design Browser window, you click on the counter test in the hierarchy and then select all the signals in the object window.

Time(HH:MM:SS) : 00:05:36

And by clicking the waveform icon, you send all these signals to the waveform window.

Time(HH:MM:SS) : 00:05:42

Then in the console, once you have forced the scan, enable SSE scan in and scan out to zero.

Time(HH:MM:SS) : 00:05:48

Then you click on the run button, the play button here which is run button and start the GLS gate level simulation.

Time(HH:MM:SS) : 00:05:59

Once the gate level simulation is ongoing and completed, you can go to the waveform and view.

Time(HH:MM:SS) : 00:06:05

This is a snapshot showing the counter waveform results.

Time(HH:MM:SS) : 00:06:09

After running the GLS.

Time(HH:MM:SS) : 00:06:14

In the design browser, you can view the instance level signals and the delays introduced.

Time(HH:MM:SS) : 00:06:19

And one of the examples here is the G7 oh five library cells and the A and Y.

Time(HH:MM:SS) : 00:06:25

And the delay between these two will be viewed in the next slide.

Time(HH:MM:SS) : 00:06:28

So the objects A and y signals.

Time(HH:MM:SS) : 00:06:31

And what is the SDF annotated or the back annotated delay between these two which was not present in the original logic simulation that will show up in the next slide in the SIM vision waveform.

Time(HH:MM:SS) : 00:06:42

So let's go ahead and see that.

Time(HH:MM:SS) : 00:06:47

So this is how we see the back annotated delay display in the waveform.

Time(HH:MM:SS) : 00:06:53

I have been mentioning about the signals A and y.

Time(HH:MM:SS) : 00:06:55

And I pulled those signals separately and see the delay in here.

Time(HH:MM:SS) : 00:06:59

So between the two markers I see the delay from A to Y is 0.04 ns, which is the back annotated delay.

Time(HH:MM:SS) : 00:07:07

Because of the SDF annotation and the GLS process, you will be executing this example in the lab at the end of this module.

VideoTitle :Checking DRC

Time(HH:MM:SS) : 00:00:00

To summarize, in this module, you identified the process of gate level simulation and described the reason for performing it in the design flow.

Time(HH:MM:SS) : 00:00:08

You identified the concept of SDF annotation in detail, and then you invoke the Xcelium simulator by using the Xrun command and simulated the gate level netlist of a simple counter design.

VideoTitle :Setting Up and Running Power and Rail Analysis

Time(HH:MM:SS) : 00:00:01

This slide lists several references or resources that you can find on Support.cadence.com or our Core is website.

Time(HH:MM:SS) : 00:00:11

Apart from the small module on GLS or gate level simulation, in this course you can find several training bytes.

Time(HH:MM:SS) : 00:00:18

Gate level simulation one.

Time(HH:MM:SS) : 00:00:19

Stop Page..

Time(HH:MM:SS) : 00:00:20

The tool user guides, Xcelium and GLS product manuals, and several articles and app notes written on GLS.

Time(HH:MM:SS) : 00:00:27

On the Core is website.

VideoTitle :Saving a GDSII File

Time(HH:MM:SS) : 00:00:01

We have a lab for you to run the gate level simulations on a simple counter design.

Time(HH:MM:SS) : 00:00:06

In this lab, you will be given a library file, a netlist, a gate level netlist, as well as the SDF file.

Time(HH:MM:SS) : 00:00:14

Using these, you will invoke the Xcelium simulator to perform the GLS on a simple counter design netlist.

Time(HH:MM:SS) : 00:00:21

So the same example was shown in the module earlier.

Time(HH:MM:SS) : 00:00:25

You could go through the slides which showed the screenshots of how to go through this process, and then start the lab.

Time(HH:MM:SS) : 00:00:31

Follow the instructions given in the lab manual, and then go ahead and perform and execute this lab.

VideoTitle :Quiz

Time(HH:MM:SS) : 00:00:00

After successful placement CTS and routing of the design, we need to check it against timing and reliability.

Time(HH:MM:SS) : 00:00:07

This module covers timing analysis and debug using Tempus.

Time(HH:MM:SS) : 00:00:12

Take a few moments to read through this module's objectives.

Time(HH:MM:SS) : 00:00:19

Let’s look at where to use tempus in the entire design flow, where does it basically fit.

VideoTitle :References

Time(HH:MM:SS) : 00:00:00

Static timing analysis is the process of adding the delays in the paths and verifying the path delay against the timing required for the design to work.

Time(HH:MM:SS) : 00:00:08

This illustration shows the calculation of the path delays.

Time(HH:MM:SS) : 00:00:12

The STA tool does this calculation automatically for the millions of paths in your design.

Time(HH:MM:SS) : 00:00:18

Analyzing and constraining these paths properly is the focus of this course.

Time(HH:MM:SS) : 00:00:22

Static timing analysis.

Time(HH:MM:SS) : 00:00:24

Sta provides a different type of timing verification to check whether the design will work when manufactured.

Time(HH:MM:SS) : 00:00:31

Complex designs running at high clock speeds are designed under tight timing constraints.

Time(HH:MM:SS) : 00:00:36

You can run static timing analysis to verify timing.

Time(HH:MM:SS) : 00:00:40

Why is static timing analysis an integral part of design closure?

Time(HH:MM:SS) : 00:00:44

Firstly, STA calculates the path delays for optimization tools.

Time(HH:MM:SS) : 00:00:49

Then, based on the path delays, the optimization tools choose cells from the timing library to create a circuit that meets your timing requirements.

Time(HH:MM:SS) : 00:00:58

Secondly, STA analyzes the timing of a circuit to verify that the circuit works at the specified frequency as we've seen before.

Time(HH:MM:SS) : 00:01:07

The STA tool is calculating the timing path delays.

Time(HH:MM:SS) : 00:01:10

These timing paths mainly consist of two basic elements.

Time(HH:MM:SS) : 00:01:14

One is the timing arc in the cells and the timing arcs of the nets.

Time(HH:MM:SS) : 00:01:18

The goal in timing is to restrict the path delay to a certain amount, as specified by the constraints.

Time(HH:MM:SS) : 00:01:24

In this example, we have a flop that provides timing check data, so the checks are also part of static timing analysis.

Time(HH:MM:SS) : 00:01:32

Timing libraries provide delays of cells interconnects, also known as nets and wires.

Time(HH:MM:SS) : 00:01:39

Apart from this, they also provide power and skew information.

Time(HH:MM:SS) : 00:01:43

The STA tool uses this information to calculate the path delays, and then verify these path delays against the constraint requirements that are provided.

Time(HH:MM:SS) : 00:01:52

The standard format that is widely used in the industry is the Liberty format, also known as dot lib.

Time(HH:MM:SS) : 00:01:58

There are other formats like Xtm or LBR that are used by different tools internally.

Time(HH:MM:SS) : 00:02:04

Most tools accept the Liberty format or dot lib as an input.

Time(HH:MM:SS) : 00:02:07

What are timing arcs?

Time(HH:MM:SS) : 00:02:09

If a change on the input is causing a change in the output, then we have a causal relationship.

Time(HH:MM:SS) : 00:02:15

If the input in this inverter is rising and the output is falling, or when the input is falling, the output is rising.

Time(HH:MM:SS) : 00:02:22

This dependency of the output on these inputs is called a causal relationship.

Time(HH:MM:SS) : 00:02:27

Each of these imaginary arcs between the input and the output is called a timing arc.

Time(HH:MM:SS) : 00:02:32

It's important to understand timing arcs so that you can determine the path delays correctly, because not every arc is symmetric.

Time(HH:MM:SS) : 00:02:39

The rising to falling arc is probably not the same as falling to rising arc.

Time(HH:MM:SS) : 00:02:44

We need to determine each arc based on how the output is actually behaving based on the inputs.

Time(HH:MM:SS) : 00:02:49

All of this information is provided in the timing library, specifically how to understand the purpose of each of these timing arcs.

Time(HH:MM:SS) : 00:02:56

Depending on the process technology, different physical elements have different levels of contribution.

Time(HH:MM:SS) : 00:03:02

Delays encountered in digital circuitry are composed of two main components cell delay and net delay.

Time(HH:MM:SS) : 00:03:08

Each stage delay, whether it's cell delay or net delay, represents the time required to propagate a signal from the input of one stage to the input of the next.

Time(HH:MM:SS) : 00:03:18

Historically, with process technologies above 90 nanometers, cell delay has been the major limiting factor in timing closure.

Time(HH:MM:SS) : 00:03:27

However, at process technologies below 90 nanometers, net delays dominate the cell delays.

Time(HH:MM:SS) : 00:03:33

All semiconductor devices take some time to switch between states.

Time(HH:MM:SS) : 00:03:38

Transition time is the time that a signal takes to change states from low to high or high to low.

Time(HH:MM:SS) : 00:03:43

The rise and fall transition times are the properties of a timing arc.

Time(HH:MM:SS) : 00:03:48

Usually, rise time and fall time are measured between 10 and 90%, or 20 and 80% of your input signal or your output signal, and calculated as rise times or fall times.

Time(HH:MM:SS) : 00:03:59

Technically, the rate of transition is measured in volts per nanosecond and is called Slew.

Time(HH:MM:SS) : 00:04:05

Transition time, such as the rise and fall times, are measured in terms of time, typically in nanoseconds.

Time(HH:MM:SS) : 00:04:11

Sta tools measure input and output transition time using Slew thresholds.

Time(HH:MM:SS) : 00:04:16

These Slew thresholds are defined in the library, and the input rise and fall times are calculated from the Slew thresholds.

Time(HH:MM:SS) : 00:04:24

The upper threshold value determines the actual time at which a device turns on and stays on.

Time(HH:MM:SS) : 00:04:30

The lower threshold value determines the time at which the device turns off and stays off.

Time(HH:MM:SS) : 00:04:35

In the illustration, the lower thresholds are 20% and the upper thresholds are 80%.

Time(HH:MM:SS) : 00:04:41

The rise transition is measured from 20% to 80% of the signal, and the fall transition is measured from 80% to 20%.

Time(HH:MM:SS) : 00:04:49

Constraints provide all the specifications that our design must meet through optimization so you can have constraints for clocks, external constraints, power constraints, yield constraints, net delay constraints, environmental constraints, and even constraints for design rules for manufacturing.

Time(HH:MM:SS) : 00:05:07

Now you can use a certain constraint to tell the tool what you need to achieve through these constraints, and then later on verify the same design through these constraints.

Time(HH:MM:SS) : 00:05:17

So for this course, we will only stick to the timing related constraints and ignore all the other ones.

Time(HH:MM:SS) : 00:05:23

The SDC constraints are the standard format for writing constraints in the design industry.

Time(HH:MM:SS) : 00:05:29

Sta tools have their own style of writing constraints.

Time(HH:MM:SS) : 00:05:32

If we were to group the SDC constraints, you would group them into operating conditions or constraints that were environmental related, your design rules or power related and timing related.

Time(HH:MM:SS) : 00:05:43

So all these different constraints are specified to achieve the common goal, which is to meet timing for your design in order to set the constraints.

Time(HH:MM:SS) : 00:05:51

Let's categorize the design into different design objects.

Time(HH:MM:SS) : 00:05:55

You might have your design, which is the container for the entire circuit.

Time(HH:MM:SS) : 00:05:58

You might have cells or blocks or have ports, which is an entry or exit point into the design and your pins, which are necessarily the ones that are around your blocks for an instance, or a hierarchical port or design, or a sub design.

Time(HH:MM:SS) : 00:06:12

Clocks are the sequential cell drivers.

Time(HH:MM:SS) : 00:06:15

Nets are the wires or the interconnect between the cells and the design ports.

VideoTitle :Labs

Time(HH:MM:SS) : 00:00:00

In order to complete the timing analysis, Tempest requires certain inputs, which can be mandatory.

Time(HH:MM:SS) : 00:00:05

Those are timing libraries.

Time(HH:MM:SS) : 00:00:07

The libs with timing data Verilog netlist design constraints in the form of SDC and SDF, and Spef file for reading the RC extraction data.

Time(HH:MM:SS) : 00:00:18

The optional input can be the XM, CS and Cdbs for noise analysis.

Time(HH:MM:SS) : 00:00:24

You can also provide a CPF to read the power domains, and the MMC definition file is used for MMC analysis.

Time(HH:MM:SS) : 00:00:31

Tempest also supports the Ocv analysis.

Time(HH:MM:SS) : 00:00:34

Its capability to do AOC V and SoC V, and for that it requires LVF.

Time(HH:MM:SS) : 00:00:40

Def and DEF files are provided.

Time(HH:MM:SS) : 00:00:43

They can be used for physical data and also for the sign off Eco.

Time(HH:MM:SS) : 00:00:47

At the end of the analysis, the output is in the form of timing and crosstalk reports.

Time(HH:MM:SS) : 00:00:52

The ECO files incremental delay format information in the form of SDF histograms and certain waveforms to model the results.

VideoTitle :Gate-Level Simulation

Time(HH:MM:SS) : 00:00:00

To use any of the Tempest products for timing analysis.

Time(HH:MM:SS) : 00:00:02

We need to start Tempest in a work environment.

Time(HH:MM:SS) : 00:00:05

In stylus See how we mode, the regular STA can be invoked using Tempest, followed by the stylus option.

Time(HH:MM:SS) : 00:00:12

You can also provide a command file with a given set of commands to be performed while invoking the Tempest using hyphen file options.

Time(HH:MM:SS) : 00:00:20

Next comes a distributed STA, which uses hyphen distributed as a switch, and it starts at distributed Static Timing Analysis.

Time(HH:MM:SS) : 00:00:28

Then Tempest ECO uses Tempest followed by the hyphen ECO command.

Time(HH:MM:SS) : 00:00:33

This ECO command prepares for the timing.

Time(HH:MM:SS) : 00:00:36

Sign off ECO flow.

Time(HH:MM:SS) : 00:00:38

to start and stop the graphical user interface, you can use Gui_show and GUI_hide.

Time(HH:MM:SS) : 00:00:43

When Tempest starts, it checks out the base license as Tempest L or Tempest XL, as shown in the figure highlighted in green.

Time(HH:MM:SS) : 00:00:51

It also shows the version and different options.

Time(HH:MM:SS) : 00:00:54

The software captures all the standard output in Tempest dot log and Tempest dot log of Tempest dot log V also shows the timestamps in a sequential manner.

Time(HH:MM:SS) : 00:01:04

Shown on the right is the red highlighted figure with that tempest log file.

Time(HH:MM:SS) : 00:01:08

Next comes the Tempest command.

Time(HH:MM:SS) : 00:01:10

The commands executed at the command prompt are captured in this file, as shown on the figure highlighted in blue on the right.

VideoTitle :Module Objectives

Time(HH:MM:SS) : 00:00:00

This slide shows the basic Analysis.

Time(HH:MM:SS) : 00:00:02

flow in the Tempus timing sign off solution.

Time(HH:MM:SS) : 00:00:05

First we read the target libraries.

Time(HH:MM:SS) : 00:00:08

Those are libs, NLM, XM, and CS libraries for specifying noise.

Time(HH:MM:SS) : 00:00:15

Then we read the Verilog netlist.

Time(HH:MM:SS) : 00:00:17

Initialize the design.

Time(HH:MM:SS) : 00:00:18

It schedules the design loading into Tempus.

Time(HH:MM:SS) : 00:00:21

Then we read the physical data that is in the form of LEF and DEF files.

Time(HH:MM:SS) : 00:00:26

Then we read the Spphs for RC extraction information and other data required for delay calculations.

Time(HH:MM:SS) : 00:00:32

Sdc is the file for reading the constraints.

Time(HH:MM:SS) : 00:00:35

Those constraints can be in the form of clock specifications, certain input and output delay requirements, and some of the constructs like don't touch and don't use on certain cells.

Time(HH:MM:SS) : 00:00:45

Then we set the timing analysis modes, set the operating conditions in the form of PVS.

Time(HH:MM:SS) : 00:00:50

And after all this data has been read, we analyze the timing.

Time(HH:MM:SS) : 00:00:54

We check the design results against the constraints specified.

Time(HH:MM:SS) : 00:00:58

If it doesn't meet, we modify the design and in case if it meets the constraints, we proceed further for chip finishing.

VideoTitle :Gate Level Simulation FlowChart

Time(HH:MM:SS) : 00:00:00

Static timing analysis uses different algorithms for delay computations.

Time(HH:MM:SS) : 00:00:04

Let's discuss the graph based analysis first.

Time(HH:MM:SS) : 00:00:08

The software by default uses a pessimistic algorithm for timing, which is based on the worst Slew propagation, or the Slew merging, referred to as graph based analysis or GBA.

Time(HH:MM:SS) : 00:00:18

In GBA mode, the software considers both the worst arrival and the worst Slew in a path during timing analysis.

Time(HH:MM:SS) : 00:00:25

Even if the worst Slew is corresponding to input pin different than the relevant pin for the current path.

Time(HH:MM:SS) : 00:00:31

This approach is used during the initial timing analysis before the final sign off, because it reduces Analysis.

Time(HH:MM:SS) : 00:00:37

runtime for the whole design.

Time(HH:MM:SS) : 00:00:39

The only drawback is it gives a pessimistic result in the figure on the right.

Time(HH:MM:SS) : 00:00:44

It is assumed for any input Slew.

Time(HH:MM:SS) : 00:00:47

The output Slew is 25% more than the input Slew.

Time(HH:MM:SS) : 00:00:51

If Slew at B is 500 picoseconds, then the Slew at z is 625 picoseconds.

Time(HH:MM:SS) : 00:00:57

So in GBA, for calculating delay through this and gate, the worst input Slew through B is always considered irrespective of the fact that the path is through pin A or B.

Time(HH:MM:SS) : 00:01:07

This is known as Slew merging.

Time(HH:MM:SS) : 00:01:09

Next comes the path based analysis or the PVA mode.

Time(HH:MM:SS) : 00:01:13

It involves retiming the components of a timing path based on the actual Slew that is propagated in the path.

Time(HH:MM:SS) : 00:01:19

It considers an actual arrival time for both base and S.I.

Time(HH:MM:SS) : 00:01:23

Delay calculation.

Time(HH:MM:SS) : 00:01:24

It removes the pessimism that is introduced to the Slew merging at various nodes in the design when the graph based analysis is run.

Time(HH:MM:SS) : 00:01:32

This is recommended to use during the final stages of timing closure to ensure accuracy, but it hits the area power dissipation and ECO cycles by a huge runtime.

Time(HH:MM:SS) : 00:01:43

On the figure towards the right, it compares GBA and PBA modes, so is shown the actual path Slews are taken in PBA, shown by the arrows.

Time(HH:MM:SS) : 00:01:53

Here the magnitude depicts the fast or the slow Slew, showing the increased delay for the slow paths.

Time(HH:MM:SS) : 00:01:58

This is a sample PBA report file.

Time(HH:MM:SS) : 00:02:02

Note that in the command, we have added the Retime option with path Slew propagation to enable PBA in the timing analysis.

Time(HH:MM:SS) : 00:02:09

In the timing report as highlighted, we can see two columns delay and the Retime delay corresponding to the cell delays.

Time(HH:MM:SS) : 00:02:16

Here, the Retime delay to respond to the PBA delays and are generally lesser than the actual delays.

Time(HH:MM:SS) : 00:02:22

The retime delays are taken into account, leading to fewer violations..

Time(HH:MM:SS) : 00:02:26

than the GBA results.

VideoTitle :What is GLS

Time(HH:MM:SS) : 00:00:00

Water on chip variations and where do they come from?

Time(HH:MM:SS) : 00:00:03

Chip variations are the intrinsic variability of semiconductor subjected to process variations as shown on the right.

Time(HH:MM:SS) : 00:00:10

Any changes in the physical parameters affect electrical parameter, in turn causing delay variations.

Time(HH:MM:SS) : 00:00:16

These can be the variation in length, width or thickness, or even the doping variations can cause to these variations.

Time(HH:MM:SS) : 00:00:23

Here the key device parameters are the channel length, threshold voltage, oxide thickness, and the channel width, while the interconnect parameters like metal thickness, dielectric thickness or the metal line width can attribute.

Time(HH:MM:SS) : 00:00:36

to these parameter variations.

Time(HH:MM:SS) : 00:00:38

Now how does this ocv affect our STA analysis.?

Time(HH:MM:SS) : 00:00:43

It affects wires and cell delays.

Time(HH:MM:SS) : 00:00:45

It also affects the clock and data paths differently.

Time(HH:MM:SS) : 00:00:49

It increases the pessimism in the design and it can be location and depth based variation for the design data.

Time(HH:MM:SS) : 00:00:56

Now the question arises how to model these on chip variations in the timing analysis?

Time(HH:MM:SS) : 00:01:01

The answer lies in the ocv AOV and the SOC V advanced approaches.

Time(HH:MM:SS) : 00:01:07

Fortunately, they all are supported in Tempus.

VideoTitle :SDF Annotation

Time(HH:MM:SS) : 00:00:00

What is advanced ocv or racv in general.

Time(HH:MM:SS) : 00:00:04

Racv is a design approach where the derate is applied to the cells vary with path depth or distance, or both.

Time(HH:MM:SS) : 00:00:10

But what's the need of racv?

Time(HH:MM:SS) : 00:00:13

The shortcoming of ocv is that timing closure gets difficult due to extra pessimism added with fixed rates for the cells.

Time(HH:MM:SS) : 00:00:21

Racv reduces the level of derating at each stage on the basis that successive stage will cancel out the variation.

Time(HH:MM:SS) : 00:00:28

So the saying goes like this.

Time(HH:MM:SS) : 00:00:30

Some stages will be faster, others slower.

Time(HH:MM:SS) : 00:00:33

So the more stages you have, the more it averages out.

Time(HH:MM:SS) : 00:00:37

In the figure shown on the right, it shows a derate table for a buffer cell based on the depth along with a timing graph.

Time(HH:MM:SS) : 00:00:44

So as discussed, these buffers along with the path, they are affected differently by the variations.

Time(HH:MM:SS) : 00:00:50

So there are different D rates applied to them based on the path depth.

Time(HH:MM:SS) : 00:00:54

For example, the buffer labeled as five gets the d rate value as 1.3 compared to the 1.6 value given to buffer one.

Time(HH:MM:SS) : 00:01:02

As the path advances, the d rate values approach towards one for both early and late checks..

Time(HH:MM:SS) : 00:01:09

So how is a calculation done in the AOV mode?

Time(HH:MM:SS) : 00:01:13

In this mode, the software uses derating libraries, wherein AOV factor is so chosen that when multiplied by the lib nominal daily value, you get close to the mean three sigma value for that many stages in the path.

Time(HH:MM:SS) : 00:01:26

As the number of stages increases, as shown in the figure, the sigma value relative to the mean decreases on the order of one by square root n, and it decreases the AOV value as shown in the table below.

Time(HH:MM:SS) : 00:01:39

Both early and late AOV tables approach towards one with increasing path depth.

Time(HH:MM:SS) : 00:01:44

What is the statistical ocv?

Time(HH:MM:SS) : 00:01:47

Soccer is a variation modeling technique that computes the impact of the local process, variations on the delay, and the Slew of each instance in the design at a given global variation corner.

Time(HH:MM:SS) : 00:01:58

This is a superset after Ocv and AOV.

Time(HH:MM:SS) : 00:02:02

So CV propagates the sigma of arrival and required times through the timing graph, and then computes the statistical characteristics of slack at all the timing pins.

Time(HH:MM:SS) : 00:02:12

This solves the shortcomings of Racv approach, which is inefficient at ultra low voltage operation and the cell timing dependency on slews and loads was not considered very well in Racv, but is considered perfectly in s o v s o v analysis.

Time(HH:MM:SS) : 00:02:27

requires variation libraries and the typical sign off timing and closure configuration in the form of cadence.

Time(HH:MM:SS) : 00:02:33

Soc V library format, known as Library Variation Format or LVF.

Time(HH:MM:SS) : 00:02:38

Lvf includes arc level absolute violations..

Time(HH:MM:SS) : 00:02:41

of cell delays, output transitions and timing checks as functions of input Slew and load.

Time(HH:MM:SS) : 00:02:47

The figure on the right shows the inputs for the SoC v Analysis., which takes in LVF and SoC V libraries along with other design inputs coming on to Liberty Variation Format or the LVF.

Time(HH:MM:SS) : 00:03:00

The file on the left shows the LVF configuration.

Time(HH:MM:SS) : 00:03:04

It takes into account the OC v Sigma cell rise and fall parameters.

Time(HH:MM:SS) : 00:03:09

The figure on the right shows the distribution taken into account for SoC V.

Time(HH:MM:SS) : 00:03:14

It also discusses the smoothening of the waveforms and the abrupt, sharp changes as it proceeds to different cells during the path.

VideoTitle :Standard Delay Format Annotation Process Flowchart

Time(HH:MM:SS) : 00:00:00

transcript is not present for this video.

VideoTitle :Invoking the Xcelium Tool to Run GLS

Time(HH:MM:SS) : 00:00:00

Let's look at how to generate reports in Tempus timing sign off solution.

Time(HH:MM:SS) : 00:00:04

This submodule covers generating and analyzing the timing reports.

VideoTitle :Module Summary

Time(HH:MM:SS) : 00:00:00

Report Timing is the basic command used for timing analysis, and it generates a timing report that provides information about the various paths in the design.

Time(HH:MM:SS) : 00:00:08

The timing report contains this information.

Time(HH:MM:SS) : 00:00:11

The slack times of the arrival signal at the end node and the associated transitions.

Time(HH:MM:SS) : 00:00:16

The start and the end nodes of each path, the signal required times, and the actual signal arrival times helping in calculation of the slack values.

Time(HH:MM:SS) : 00:00:24

The summary of propagated versus the ideal status of launching and capturing clocks is shown.

Time(HH:MM:SS) : 00:00:29

Any phase shift or kpwr values applied to the timing checks.

Time(HH:MM:SS) : 00:00:33

are also displayed.

Time(HH:MM:SS) : 00:00:34

The figure on the right shows a sample report timing in Tempus.

Time(HH:MM:SS) : 00:00:38

Let's look at the report generated from report timing in more detail.

Time(HH:MM:SS) : 00:00:42

This shows the end point and the begin point for the particular path.

Time(HH:MM:SS) : 00:00:47

Any path groups applied are shown like MDS Ram clock as the path group applied on this path.

Time(HH:MM:SS) : 00:00:53

It shows the setup, the phase shift required and the arrival times.

Time(HH:MM:SS) : 00:00:57

The clock network latency.

Time(HH:MM:SS) : 00:00:58

The instances, the cell types, and the corresponding Responding to lay in the slack calculation column, we can see that the arrival time is later than that required time, leading to a slack value of -0.959.

Time(HH:MM:SS) : 00:01:13

Report Critical insights is the command being used to report any timing critical instance, commonly referred to as bottleneck instance.

Time(HH:MM:SS) : 00:01:21

The timing report includes the total negative slack for each instance, the total count of connected start and end points, and the total count of paths through that particular instance.

Time(HH:MM:SS) : 00:01:31

In the figure shown on the right, we have used the command report Critical insights.

Time(HH:MM:SS) : 00:01:35

Hyphen Max insights is set to five.Hyphen include.

Time(HH:MM:SS) : 00:01:38

Sequential allows that the report to show the sequential instances in the design.

Time(HH:MM:SS) : 00:01:44

Certain timing paths in the design may be assigned Path exceptions.

Time(HH:MM:SS) : 00:01:49

Report path exceptions is the command which generates a report about these exceptions, which may be specified using set false path set min delay, set max delay, or the set multi-cycle path command.

Time(HH:MM:SS) : 00:02:01

Multiple path exceptions that match a given path are prioritized and applied and shown in the reports.

Time(HH:MM:SS) : 00:02:07

When the software is in multi-mode multi corner timing analysis mode, the report generates path exception information for every active analysis view toward the right.

Time(HH:MM:SS) : 00:02:17

The example shows the Multi-cycle exception being ignored due to the false path on V clock one.

Time(HH:MM:SS) : 00:02:23

This is due to the Multi-cycle exception path given less priority than the false path.

Time(HH:MM:SS) : 00:02:30

The report clock command reports the clock waveform, the arrival point, and the generated clock information.

Time(HH:MM:SS) : 00:02:38

Here in this example, it shows the clock description, the source, the time period, and whether it's generated or propagated Clock.

Time(HH:MM:SS) : 00:02:47

For generated clock description, it is providing the master source and the clock source pin the frequency multiplier in the edges.

Time(HH:MM:SS) : 00:02:56

Clock timing is very crucial for correct timing analysis.

Time(HH:MM:SS) : 00:03:00

The report clock timing command generates a clock skew report for the current design and displays the latency skew numbers, the CBR values and different clock pins associated.

Time(HH:MM:SS) : 00:03:11

The software generates a separate report for each specified clock on a pair of clocks.

Time(HH:MM:SS) : 00:03:16

There can be certain specifications which will be provided, like.

Time(HH:MM:SS) : 00:03:19

The clocks will be provided using hyphen clock and the clock pairs using hyphen from clock and hyphen to clock parameters.

Time(HH:MM:SS) : 00:03:26

Hyphen early and hyphen late parameters are used to specify the type of skew to be reported.

Time(HH:MM:SS) : 00:03:32

The hyphen view parameter is used to report the clock timing for a user specified view in multimode multi-core mode.

Time(HH:MM:SS) : 00:03:39

To remove the common path pessimism from the reported skew or the kpwr number, so that the report contains a separate column for the common path adjustment values, we can use set analysis mode hyphen Kpwr both and then we can do report clock timing.

Time(HH:MM:SS) : 00:03:54

This is a sample clock report shown.

Time(HH:MM:SS) : 00:03:56

This is a sample report clock timing shown in more detail.

Time(HH:MM:SS) : 00:04:00

Here it shows the skewed numbers.

Time(HH:MM:SS) : 00:04:03

The jitter values and also the inter clock skew with the latency.

VideoTitle :References

Time(HH:MM:SS) : 00:00:00

Given the timing reports.

Time(HH:MM:SS) : 00:00:01

Let's see how to debug the timing analysis results.

Time(HH:MM:SS) : 00:00:04

This submodule covers debugging the timing reports and different commands used for it.

VideoTitle :Labs

Time(HH:MM:SS) : 00:00:00

In the design.

Time(HH:MM:SS) : 00:00:00

Certain disabled arcs lead to unsuccessful timing analysis and computation in the timing analysis and debug information about disabled timing and timing.

Time(HH:MM:SS) : 00:00:10

Check arcs is given by the report inactive Arcs command.

Time(HH:MM:SS) : 00:00:14

This reports all the Arcs which are disabled due to user specified exceptions such as set disabled timing or set case analysis.

Time(HH:MM:SS) : 00:00:22

This command is particularly useful for querying specific instances for the status of the arcs.

Time(HH:MM:SS) : 00:00:28

It also reports about arcs disabled by constant propagation during timing analysis, snip loop arcs, or the arcs disabled in the timing library.

Time(HH:MM:SS) : 00:00:37

It is not very good for determining possible causes of the lack of timing data at certain location.

Time(HH:MM:SS) : 00:00:43

For that, we can use report fanout or report fan in with the trace through option to search for blockages.

Time(HH:MM:SS) : 00:00:49

The figure on the right shows that there is an arc disabled across U2 from A to Y pin of the cell.

Time(HH:MM:SS) : 00:00:55

Certain timing paths may have more than one path exception applied, which can be queried with the report.

Time(HH:MM:SS) : 00:01:00

Path exceptions command.

Time(HH:MM:SS) : 00:01:02

This command proves useful for detecting problems with the constraints which were not reported immediately to the console or may have been hidden.

Time(HH:MM:SS) : 00:01:10

It's also good for false path debugging, since the false paths have the highest priority among the path exceptions.

Time(HH:MM:SS) : 00:01:16

The table on the right shows the priorities for the path exceptions applied to the same path.

Time(HH:MM:SS) : 00:01:22

After set false path set max delay takes the priority followed with set Multi-cycle path.

Time(HH:MM:SS) : 00:01:28

Also within these commands, the pin list explicit command has higher priority than the clock list command.

Time(HH:MM:SS) : 00:01:35

By default, this command reports activated path exceptions to report the path exceptions that have been ignored by the timing analysis.

Time(HH:MM:SS) : 00:01:43

You can specify hyphen ignored parameter to the command.

Time(HH:MM:SS) : 00:01:47

Another useful debugging technique is to use the hyphen path exceptions option with the report timing command.

Time(HH:MM:SS) : 00:01:53

This option is very useful for determining which exceptions were applied by the timer, when there were overlapping

types of constraints against a particular timing path.

Time(HH:MM:SS) : 00:02:02

In the example shown on the right, it has a mix of constraints set max delay, set Multi-cycle path and set false path, all applied against a reg to reg path from block Br1 to block Br2.

Time(HH:MM:SS) : 00:02:15

As expected, the set false path Constraint has precedence, causing the path to be initially blocked by turning on the report timing hyphen unconstrained option.

Time(HH:MM:SS) : 00:02:24

Along with hyphen path exceptions, we can see all three exceptions reported along with their state.

Time(HH:MM:SS) : 00:02:30

The exceptions report format is similar to that provided by the global report from the report path Exceptions Command.

Time(HH:MM:SS) : 00:02:37

The report Analysis Coverage command provides details about the timing checks and the design.

Time(HH:MM:SS) : 00:02:42

We can use the hyphen verbose option to get a quick indication of the reason for the endpoint being unconstrained for each timing check.

Time(HH:MM:SS) : 00:02:50

The command reports a number of checks that meet or violate the constraints, or that are untested in the summary section, as shown in the figure on the first half.

Time(HH:MM:SS) : 00:02:58

Then the detailed section includes data about signal or reference pins, check types, slack and the reason for being untested is shown in below section.

Time(HH:MM:SS) : 00:03:08

It shows that the d pin of F1 has got a false path, and this setup timing has been untested.

VideoTitle :Timing Analysis and Debug

Time(HH:MM:SS) : 00:00:00

The report.

Time(HH:MM:SS) : 00:00:00

Analysis.

Time(HH:MM:SS) : 00:00:01

coverage command provides details about the timing, checks.

Time(HH:MM:SS) : 00:00:04

and the design.

Time(HH:MM:SS) : 00:00:05

We can use the hyphen verbose option to get a quick indication of the reason for the endpoint being unconstrained.

Time(HH:MM:SS) : 00:00:11

We can use report cell instance timing on the particular cell to query more about the settings.

Time(HH:MM:SS) : 00:00:16

When the particular schematic path has been tested against the report.

Time(HH:MM:SS) : 00:00:20

Analysis.

Time(HH:MM:SS) : 00:00:20

coverage hyphen verbose option.

Time(HH:MM:SS) : 00:00:22

We find that a false path or a disable constraint and a constant constraint has been applied.

Time(HH:MM:SS) : 00:00:28

Now how to get this information in more detail.

Time(HH:MM:SS) : 00:00:31

So the first figure on the right shows that a constant zero has been applied.

Time(HH:MM:SS) : 00:00:36

Coming to the second figure it shows the report case Analysis.

Time(HH:MM:SS) : 00:00:39

hyphen verbose option along with hyphen propagated followed by the clock pin.

Time(HH:MM:SS) : 00:00:44

It shows and we can see how the constant got there by checking the propagation using this command.

Time(HH:MM:SS) : 00:00:49

This slide shows further querying on the report analysis coverage results.

Time(HH:MM:SS) : 00:00:54

So as shown we have got a false path at the d pin of flip flop one.

Time(HH:MM:SS) : 00:00:59

This can be tested further using report timing hyphen unconstrained hyphen path exceptions all hyphen to this particular pin, so it results in applied exception being the false path showing the particular view.

Time(HH:MM:SS) : 00:01:12

For this, the timing has been tested.

Time(HH:MM:SS) : 00:01:15

Coming to the disabled constraint on the d pin of flip flop two, we can find the commands affecting an instance with the report and active arcs.

Time(HH:MM:SS) : 00:01:23

It shows the arc types and the reason that it's a user disabled setting.

Time(HH:MM:SS) : 00:01:27

Following on the particular views.

VideoTitle :Module Objectives

Time(HH:MM:SS) : 00:00:00

Let's discuss the global timing debug interface within Tempest.

Time(HH:MM:SS) : 00:00:04

In this submodule, we will explore the global timing debug interface provided by Tempest, and how to generate and analyze the timing reports in this interface.

VideoTitle :Concept BSTA Timing parameters

Time(HH:MM:SS) : 00:00:00

One of the major issues faced by the designers in today's world is that the designs are very huge, with thousands of complex failing paths having high ones.

Time(HH:MM:SS) : 00:00:08

It takes several iterations to identify the root causes and develop or implement the fixes.

Time(HH:MM:SS) : 00:00:14

So a better approach to this timing debug problem is the global timing debug interface in Tempus.

Time(HH:MM:SS) : 00:00:21

This feature proves useful for debugging the results, and the various timing debug forms provide easy visual access to the timing reports and debugging tools, so the features include unique path categorization capabilities which can be visualized.

Time(HH:MM:SS) : 00:00:35

An enhanced path centric debug features are provided in this interface.

Time(HH:MM:SS) : 00:00:40

The goal is to reduce thousands of failing paths to a smaller number of unique problems to be solved, then rerun the software after implementing those fixes.

VideoTitle :Input and Output Files for Tempus

Time(HH:MM:SS) : 00:00:00

Next comes how to display or generate the timing reports.

Time(HH:MM:SS) : 00:00:03

This can be done through timing and AC menu option.

Time(HH:MM:SS) : 00:00:07

We need to go with the Debug Timing tab.

Time(HH:MM:SS) : 00:00:10

The Display Generate Timing report form can be used to specify the violation report to be read in.

Time(HH:MM:SS) : 00:00:15

For debugging the timing results in the Display Generate Timing Reports tab.

Time(HH:MM:SS) : 00:00:20

The generate option creates a violation report file in the machine readable format.

Time(HH:MM:SS) : 00:00:25

Check type can be selected for the type of check like setup or hold.

Time(HH:MM:SS) : 00:00:29

Retime.

Time(HH:MM:SS) : 00:00:30

Specifies the advanced timing methods, that is AOV, STA path Slew propagation and waveform propagation.

Time(HH:MM:SS) : 00:00:37

After the timing reports are generated, the debug form is used to view the debugging timing results.

Time(HH:MM:SS) : 00:00:44

This form is displayed in the analysis tab once the reports are generated and loaded in the software.

Time(HH:MM:SS) : 00:00:50

Use the path histogram to identify all the categories.

Time(HH:MM:SS) : 00:00:54

The idea is to start looking at the big picture overall histogram at once.

Time(HH:MM:SS) : 00:00:59

It shows information like summary of timing results, path lists, path categories and can be used to start debugging the results.

Time(HH:MM:SS) : 00:01:06

The paths and the timing debug form can be further analyzed using the timing path analyzer form provided in that GTD interface.

Time(HH:MM:SS) : 00:01:15

This form can be accessed by double clicking on a particular path in the timing debug form.

Time(HH:MM:SS) : 00:01:20

This form is used to identify the issues related to a path using slack calculation bar, timing bars, and the hierarchy view.

Time(HH:MM:SS) : 00:01:28

The idea is to make the obvious problems very visible.

Time(HH:MM:SS) : 00:01:32

So in the first slack calculation column, it displays path arrival time and required time calculations in the color bars.

Time(HH:MM:SS) : 00:01:40

Next, the data delay column displays the details of the selected path in the violation report.

Time(HH:MM:SS) : 00:01:47

Next comes the timing bar and it displays the instance and net delays.

Time(HH:MM:SS) : 00:01:50

Size of the bar indicates that delay associated the hierarchy.

Time(HH:MM:SS) : 00:01:55

View displays a paths traversal a path against a logical hierarchy on a time axis.

Time(HH:MM:SS) : 00:02:01

Let's discuss these columns in more detail in the next slide.

Time(HH:MM:SS) : 00:02:04

The slack calculation bars are referred for arrival and required times.

Time(HH:MM:SS) : 00:02:09

You can identify clock skew issues, latency balancing, or large clock uncertainty issues using these bars.

Time(HH:MM:SS) : 00:02:16

Detailed path tab show path details, including launch and capture.

Time(HH:MM:SS) : 00:02:21

The path SDC tab displays the SDC constraints related to the selected path like create clocks, set false path, set Multi-cycle path, set Max delay, and others.

Time(HH:MM:SS) : 00:02:32

The Timing Interpretation tab provides rule based path analysis to help you discover sources of potential timing problems in the path.

Time(HH:MM:SS) : 00:02:40

The simulation tab provides an option to generate the SPICE netlist for the specified net.

Time(HH:MM:SS) : 00:02:45

Transfer this data to a circuit simulator like SPICE, and then correlate the timing in Tempus with the SPICE timing reports.

Time(HH:MM:SS) : 00:02:52

Internally, it involves the creation of a SPICE deck within Tempest.

Time(HH:MM:SS) : 00:02:57

Next, the schematic tab displays the gate level schematic view of the critical path.

Time(HH:MM:SS) : 00:03:03

Next comes the path delay timing bars.

Time(HH:MM:SS) : 00:03:05

It is used to analyze delays associated with instances and nets in a path.

Time(HH:MM:SS) : 00:03:10

Use this information to identify the issues related to large instance or net delays.

Time(HH:MM:SS) : 00:03:15

Repeater chains paths with large number of buffers, and large macro delays.

Time(HH:MM:SS) : 00:03:20

At the end, hierarchical representation of the path is shown the hierarchy view field.

Time(HH:MM:SS) : 00:03:26

This representation of the path delay shows the traversal of a path through the design hierarchy drawn on the time axis.

Time(HH:MM:SS) : 00:03:33

A longer arrow means that there are more instances on its path.

Time(HH:MM:SS) : 00:03:37

Use this information to see the module, where the path is spending more time, or to identify inter partition timing problems.

Time(HH:MM:SS) : 00:03:44

This slide shows how the slack calculation bar be used to debug the timing and the timing path analyzer.

Time(HH:MM:SS) : 00:03:50

In the first example, the launch latency and the capture latency components are not aligned.

Time(HH:MM:SS) : 00:03:56

Therefore, there can be a large clock latency mismatch in the path.

Time(HH:MM:SS) : 00:04:00

In the second one, the cycle adjustment bar in the required time indicates the presence of a multi cycle path.

Time(HH:MM:SS) : 00:04:07

The next one shows a large input delay in an input output path, represented by the light blue bar in the arrival time.

Time(HH:MM:SS) : 00:04:14

The last one shows the path delay bar in the required time, which indicates a set max delay Constraint.

Time(HH:MM:SS) : 00:04:25

In this video, we are going to look at the Timing Path Analyzer window that is generated from the analysis tab of the timing analysis.

Time(HH:MM:SS) : 00:04:36

Currently we see the all category and all the paths are actually laid out kind of 1 to 50.

Time(HH:MM:SS) : 00:04:44

And so basically after running bottleneck analysis I have these two extra categories and I can actually just open them up.

Time(HH:MM:SS) : 00:04:56

And as you can see here, it's part number 18 through um, 382.

Time(HH:MM:SS) : 00:05:04

And there is only one page here.

Time(HH:MM:SS) : 00:05:07

So the pagination wise there is only one page.

Time(HH:MM:SS) : 00:05:10

And I'm going to go through all of them.

Time(HH:MM:SS) : 00:05:12

It's about 78 paths let's say.

Time(HH:MM:SS) : 00:05:16

And that would be too much to basically debug.

Time(HH:MM:SS) : 00:05:19

So what I could actually do is because this is bottleneck analysis, I could try and locate whatever is causing the bottleneck and look at that and try to fix that particular cell.

Time(HH:MM:SS) : 00:05:32

So what I'm doing here is just double clicking and opening up the timing path analyzer.

Time(HH:MM:SS) : 00:05:41

And in here we have a lot of data.

Time(HH:MM:SS) : 00:05:45

And we can look at the information here little by little.

Time(HH:MM:SS) : 00:05:50

But basically one of the things to observe here is that when you use these, um, buttons, they kind of move within the category.

Time(HH:MM:SS) : 00:06:01

So it's going from 18 to 40 6 to 50 3 to 63.

Time(HH:MM:SS) : 00:06:05

So these path numbers are actually the same.

Time(HH:MM:SS) : 00:06:08

And you're actually moving through the paths in the same category.

Time(HH:MM:SS) : 00:06:14

So the assumption is that the whatever the problems in this category are and basically they're similar.

Time(HH:MM:SS) : 00:06:24

So they have a similar issue whatever it might have been.

Time(HH:MM:SS) : 00:06:29

So hopefully we can identify whatever the problem point is and try to fix that.

Time(HH:MM:SS) : 00:06:35

So with that said, let's actually go into the details of the contents of the timing path analyzer.

Time(HH:MM:SS) : 00:06:45

Uh, in this window, uh, you obviously see the path number and you see what is in the report in terms of what type of violation it is and what type of check we are performing.

Time(HH:MM:SS) : 00:06:59

First of all, and what type of violation it is.

Time(HH:MM:SS) : 00:07:03

So therefore if it is um, uh, basically, uh, negative slack, if there is a skew issue, um, what is the cprw.

Time(HH:MM:SS) : 00:07:15

What is the start point?

Time(HH:MM:SS) : 00:07:16

End point.

Time(HH:MM:SS) : 00:07:16

So all of these are actually kind of, uh, you know, markers to try to identify whatever the problem might be.

Time(HH:MM:SS) : 00:07:24

And also it notes the views.

Time(HH:MM:SS) : 00:07:28

So all of these things you see in the reports as well, and the slack calculation bar is showing what is the required time and what is the arrival time.

Time(HH:MM:SS) : 00:07:40

So basically the arrival time is kind of shown here.

Time(HH:MM:SS) : 00:07:44

And the required time is shown on the bottom.

Time(HH:MM:SS) : 00:07:47

So basically, whatever the problem is, if there is a negative slack, it would be identified in the required time area.

Time(HH:MM:SS) : 00:07:57

Uh, like so.

Time(HH:MM:SS) : 00:07:59

So uh, you also can identify a few things from the slack calculation bar as well.

Time(HH:MM:SS) : 00:08:07

Uh, the launch latency capture latency.

Time(HH:MM:SS) : 00:08:09

These are actually latencies that are always in that kind of blue.

Time(HH:MM:SS) : 00:08:13

Uh, you also have incremental delay, kind of shown in kind of a gray right there, phase shift or clock period, depending on if it's multiple clocks, clock one to clock two kind of situation, then it's actually recorded as phase shift.

Time(HH:MM:SS) : 00:08:29

But if it's actually the same clock, instead of calling it period, we still call it a phase shift.

Time(HH:MM:SS) : 00:08:38

And then whatever the negative slack or positive slack is actually identified here.

Time(HH:MM:SS) : 00:08:43

Uh, your delay through the data path is actually shown here the data delay.

Time(HH:MM:SS) : 00:08:47

And so the setup times and or any uncertainty is also recorded here.

Time(HH:MM:SS) : 00:08:54

So what this um basically means is that the color coding is pretty constant.

Time(HH:MM:SS) : 00:09:02

And it is kind of easier to identify what the problem might be.

Time(HH:MM:SS) : 00:09:07

As you can see here, there is a slight skew between the launch latency and the capture latency.

Time(HH:MM:SS) : 00:09:13

So it's it gets easier and easier to identify what the problem might be.

Time(HH:MM:SS) : 00:09:19

As you look at more and more of these slack calculation bars.

Time(HH:MM:SS) : 00:09:26

So approximately the same skew between these particular, um, you know, uh, paths is actually, uh, you know, pretty common right there.

Time(HH:MM:SS) : 00:09:35

And um, the data path is shown in the reports.

Time(HH:MM:SS) : 00:09:40

And so it's similarly laid out here.

Time(HH:MM:SS) : 00:09:44

We also have all these different, um, columns here.

Time(HH:MM:SS) : 00:09:50

What is the arc?

Time(HH:MM:SS) : 00:09:50

What is the cell?

Time(HH:MM:SS) : 00:09:51

This is what you would see in the timing reports as well.

Time(HH:MM:SS) : 00:09:56

Um, but one thing to note is the the status is actually helpful in certain cases.

Time(HH:MM:SS) : 00:10:02

If there is a fixed cell or something, the status would actually easily identify what that problem might be in that particular um, design or uh, path.

Time(HH:MM:SS) : 00:10:14

So in this case it's fixed or pre-placed.

Time(HH:MM:SS) : 00:10:18

So that is actually one more thing to note.

Time(HH:MM:SS) : 00:10:20

And then there are multiple statuses.

Time(HH:MM:SS) : 00:10:23

And it's a good idea to check if there is any cells that cannot be moved or upsized or downsized or anything like that before.

Time(HH:MM:SS) : 00:10:32

You may try to make any changes to your design.

Time(HH:MM:SS) : 00:10:37

Uh, you also have multiple tabs here.

Time(HH:MM:SS) : 00:10:39

We'll go into details of each of the tabs.

Time(HH:MM:SS) : 00:10:42

Um, also to notice the path delay bar and the path delay bar, uh, is color coded in such a way that you have, uh, the inverters and buffers are actually easily identifiable.

Time(HH:MM:SS) : 00:10:59

So anything with a thick yellow, um, you know, bar is actually a buffer, as you can see here.

Time(HH:MM:SS) : 00:11:08

And then if you select this, it's an inverter.

Time(HH:MM:SS) : 00:11:12

The green ones and the rest are all basically other kind of, um, cells that you would find in your design.

Time(HH:MM:SS) : 00:11:20

And also if you try to zoom in into an area, uh, you will also notice a thinner yellow bar.

Time(HH:MM:SS) : 00:11:28

And the thinner yellow bar is basically the, uh, the net delay.

Time(HH:MM:SS) : 00:11:32

So if there is a net, uh, that particular net delay is actually captured there.

Time(HH:MM:SS) : 00:11:37

So that is what I Here are the key identifiers in the Patlabor.

Time(HH:MM:SS) : 00:11:43

So you can right click and fit.

Time(HH:MM:SS) : 00:11:46

And basically the entire um Patlabor is supposed to fit there.

Time(HH:MM:SS) : 00:11:52

So this is helpful in identifying especially the large delay cells.

Time(HH:MM:SS) : 00:11:59

So in this case because we are looking for the bottleneck instance, it could be the one with the highest delay.

Time(HH:MM:SS) : 00:12:09

Or it could be the one that is actually, you know, one of the other ones, but has the but is the cause of the most negative delay.

Time(HH:MM:SS) : 00:12:19

So we can scroll and look for whatever the offending party is.

Time(HH:MM:SS) : 00:12:26

And in our case it's J3.

Time(HH:MM:SS) : 00:12:30

And when you select J3 it's actually highlighting that particular instance right here.

Time(HH:MM:SS) : 00:12:35

So it's kind of, you know, kind of semi not grayed out but kind of blurred out.

Time(HH:MM:SS) : 00:12:42

So same thing happens when you select something else.

Time(HH:MM:SS) : 00:12:46

It's kind of blurred whereas the others are more defined.

Time(HH:MM:SS) : 00:12:51

Then the hierarchy view also can help in the sense that what hierarchy has the most delay.

Time(HH:MM:SS) : 00:12:58

So that can actually this is a time based hierarchical view.

Time(HH:MM:SS) : 00:13:02

So you can actually see that m core zero is the is encapsulating the entire path.

Time(HH:MM:SS) : 00:13:10

Whereas you know, at a lower level hb0 and M control zero are actually the ones with the most delay.

Time(HH:MM:SS) : 00:13:20

And then at this level, the highest delay is the I19622.

Time(HH:MM:SS) : 00:13:27

So kind of showing you what the view looks like down below in the hierarchy.

Time(HH:MM:SS) : 00:13:37

So let's look at some of the other tabs here now.

Time(HH:MM:SS) : 00:13:44

So the next part is the launch clock.

Time(HH:MM:SS) : 00:13:50

The launch clock is actually identified.

Time(HH:MM:SS) : 00:13:52

And you can actually see different statuses here.

Time(HH:MM:SS) : 00:13:57

Uh this is a clock kind of thing.

Time(HH:MM:SS) : 00:13:59

And so there are actually, um, you know, fixed cells in here and so on.

Time(HH:MM:SS) : 00:14:05

So this is something to also note.

Time(HH:MM:SS) : 00:14:09

And because there are actual cells in this clock path, it means that the, you know, design is in a propagated state.

Time(HH:MM:SS) : 00:14:19

And uh, you know, what is the cells, what are the cells that are causing the most delay.

Time(HH:MM:SS) : 00:14:25

So on and so forth.

Time(HH:MM:SS) : 00:14:26

You can actually look to see if there are any issues here.

Time(HH:MM:SS) : 00:14:30

Same thing with the capture clock.

Time(HH:MM:SS) : 00:14:33

And so again the capture clock would also show the same thing.

Time(HH:MM:SS) : 00:14:39

And then the first time you click on the path SDC tab, you should see a slight delay.

Time(HH:MM:SS) : 00:14:46

And that is because the tool computes all the problems with the SDC and correlates them to this particular path.

Time(HH:MM:SS) : 00:14:58

So it does that for all of the paths that are in your timing report all at once.

Time(HH:MM:SS) : 00:15:04

So that way it might take a while before the path SDC tab shows anything on the graphical interface.

Time(HH:MM:SS) : 00:15:13

So in this case it's only showing all the sdcs that are relevant to this path.

Time(HH:MM:SS) : 00:15:19

So the clocks are propagated.

Time(HH:MM:SS) : 00:15:22

What are the clocks that are pertinent to this particular path?

Time(HH:MM:SS) : 00:15:25

What is the latency and any uncertainties on that are going to this particular path?

Time(HH:MM:SS) : 00:15:32

Then we also have another tab called the Timing Interpretation tab.

Time(HH:MM:SS) : 00:15:36

And in this tab you can modify some of these settings.

Time(HH:MM:SS) : 00:15:43

So in we are flagging any paths that actually, you know, exceed the violation limit that we specify.

Time(HH:MM:SS) : 00:15:52

So in this case the net fan out is greater than five.

Time(HH:MM:SS) : 00:15:56

We will flag it.

Time(HH:MM:SS) : 00:15:57

But I can actually right click on it and modify.

Time(HH:MM:SS) : 00:16:01

And basically I can um change the values that you actually see here.

Time(HH:MM:SS) : 00:16:08

You can actually say modify it.

Time(HH:MM:SS) : 00:16:10

And you can um, basically just go on to net to long um, thousand or coupling capacitance, whatever the condition is.

Time(HH:MM:SS) : 00:16:21

Net fan out of five, I can actually modify this to ten.

Time(HH:MM:SS) : 00:16:27

And so you can actually specify that.

Time(HH:MM:SS) : 00:16:30

And so therefore, the next time around, uh, or, you know, I can when I open up the timing interpretation again, it would give me a different kind of look and feel for the timing interpretation.

Time(HH:MM:SS) : 00:16:48

So as you can see here, the net fan out greater than ten is what is going to be flagged.

Time(HH:MM:SS) : 00:16:54

So uh, it also shows other things like distance buffer tree.

Time(HH:MM:SS) : 00:17:01

Um, what is the repeater chain?

Time(HH:MM:SS) : 00:17:03

Uh, you know, distance.

Time(HH:MM:SS) : 00:17:05

And uh, what is the distance from start to end?

Time(HH:MM:SS) : 00:17:07

This is all based on floor plan.

Time(HH:MM:SS) : 00:17:09

And there are some things based on drv analysis..

Time(HH:MM:SS) : 00:17:12

Some things are based on constraints, large SKUs and so on.

Time(HH:MM:SS) : 00:17:16

And some are based on your structure of your path.

Time(HH:MM:SS) : 00:17:22

So if it's containing any hard macros, level shifters, isolation cells, so on and so forth.

Time(HH:MM:SS) : 00:17:28

So all of those things can be flagged.

Time(HH:MM:SS) : 00:17:32

Then on the simulation tab allows you to run the simulation after you can generate the SPICE deck, but this would require a virtuoso license.

Time(HH:MM:SS) : 00:17:43

And then you can basically run your simulation.

Time(HH:MM:SS) : 00:17:48

So once you generate your SPICE deck, it would basically, you know, require the models for generating your spice deck.

Time(HH:MM:SS) : 00:17:59

And all the components in that particular path need to have the spice models.

Time(HH:MM:SS) : 00:18:05

And then once the analysis is complete, you can run your simulation.

Time(HH:MM:SS) : 00:18:12

And once you run the simulation, you should be able to, uh, do a report timing.

Time(HH:MM:SS) : 00:18:21

And based on that report timing, you can actually see what the difference is between the two, um, you know, um, simulation SPICE simulation versus your actual timing report.

Time(HH:MM:SS) : 00:18:35

So what is the difference?

Time(HH:MM:SS) : 00:18:37

You can actually see and finally, we also have the schematic.

Time(HH:MM:SS) : 00:18:41

The schematic actually, uh, you know, can be zoomed in.

Time(HH:MM:SS) : 00:18:46

Uh, you can actually look at only this path in more detail, um, from one particular component to the next, uh, in the form that Genus.

Time(HH:MM:SS) : 00:18:57

might show.

Time(HH:MM:SS) : 00:18:58

So that is what, um, this is actually, uh, showing you.

Time(HH:MM:SS) : 00:19:06

Uh, once we identify whatever the problem is, after you do your analysis, you should be able to, um, you know, move on to other things.

Time(HH:MM:SS) : 00:19:16

Uh, with respect to the category creation, you know, one bucket at a time.

VideoTitle :Quiz

Time(HH:MM:SS) : 00:00:00

After identifying the timing problems, you may want to group all paths with the same problem under a single category.

Time(HH:MM:SS) : 00:00:06

This can be done through the Create Path category form, which is used to define the categories as per the design requirements and use to analyze the critical paths and provide the fixes.

Time(HH:MM:SS) : 00:00:16

This can be reached through the analysis tab in the create pull down category.

Time(HH:MM:SS) : 00:00:21

Use create as an option.

Time(HH:MM:SS) : 00:00:23

This can be used to create standard path group like conditions or other conditions not supported in path group.

Time(HH:MM:SS) : 00:00:29

Definition.

Time(HH:MM:SS) : 00:00:30

The Equivalent.

Time(HH:MM:SS) : 00:00:31

tickle command for this function is create path category.

Time(HH:MM:SS) : 00:00:34

Next comes creating path groups.

Time(HH:MM:SS) : 00:00:37

The path analysis form is used to create standard path categories according to basic path groups.

Time(HH:MM:SS) : 00:00:43

This can be reached through analysis tab within Analysis.

Time(HH:MM:SS) : 00:00:47

dropdown menu.

Time(HH:MM:SS) : 00:00:48

Select path Group analysis.

Time(HH:MM:SS) : 00:00:50

The basic path groups created are Reg to reg input to register.

Time(HH:MM:SS) : 00:00:55

Register to output.

Time(HH:MM:SS) : 00:00:56

Input to output.

Time(HH:MM:SS) : 00:00:58

In this kind of definition, registers can be macros, latches, or sequential cell types.

Time(HH:MM:SS) : 00:01:04

How to create clock Analysis.

Time(HH:MM:SS) : 00:01:05

categories.

Time(HH:MM:SS) : 00:01:07

The path analysis.

Time(HH:MM:SS) : 00:01:08

form is used to create categories according to the launch clock capture clock combinations, and this can be reached through the clock Analysis.

Time(HH:MM:SS) : 00:01:15

option within the Analysis.

Time(HH:MM:SS) : 00:01:16

dropdown menu.

Time(HH:MM:SS) : 00:01:18

The following categories are created in this kind of Analysis.

Time(HH:MM:SS) : 00:01:21

the paths with clock fully contained in a single domain.

Time(HH:MM:SS) : 00:01:24

Clock one are the paths with clock starting in one clock domain and ending at another, like clock one to clock two.

Time(HH:MM:SS) : 00:01:30

In this slide, it shows all the paths which have been sorted by, from and to clock.

Time(HH:MM:SS) : 00:01:36

Next feature is to create view Analysis.

Time(HH:MM:SS) : 00:01:38

categories.

Time(HH:MM:SS) : 00:01:40

The path Analysis.

Time(HH:MM:SS) : 00:01:41

form can be used to create categories according to the view that each path is related to.

Time(HH:MM:SS) : 00:01:46

This can be done using the view Analysis.

Time(HH:MM:SS) : 00:01:48

option within the Analysis.

Time(HH:MM:SS) : 00:01:50

dropdown.

Time(HH:MM:SS) : 00:01:51

The impact of each view on the performance can be analyzed using this category in a single report or the report containing all views together.

Time(HH:MM:SS) : 00:01:59

Each endpoint is reported once.

Time(HH:MM:SS) : 00:02:02

When you load several reports that were generated for different views, each endpoint may appear in each report with a different slack value.

Time(HH:MM:SS) : 00:02:09

This shows a view Analysis.

Time(HH:MM:SS) : 00:02:11

creation and shows all the paths sorted by views.

Time(HH:MM:SS) : 00:02:14

Creating bottleneck instances is another dimension in this feature.

Time(HH:MM:SS) : 00:02:19

The path analysis form can be used to create categories according to the bottleneck instances of the critical path.

Time(HH:MM:SS) : 00:02:25

This can be done by selecting bottleneck Analysis within the analysis dropdown.

Time(HH:MM:SS) : 00:02:30

This analysis detects the instances that are often involved in the critical paths in the design.

Time(HH:MM:SS) : 00:02:36

Any change in timing for these instances impacts multiple critical paths in the design, so you can use this information to change your design according to these instance reporting.

Time(HH:MM:SS) : 00:02:47

The bottleneck analysis computes the number of occurrences that an instance appears in the path that you display in the timing debug window.

Time(HH:MM:SS) : 00:02:58

In this video, we're going to look at how to create predefined path categories for timing analysis.

Time(HH:MM:SS) : 00:03:05

After opening the path histogram in the analysis tab you can start creating categories.

Time(HH:MM:SS) : 00:03:11

So currently we only have one category namely the all category.

Time(HH:MM:SS) : 00:03:17

And the ones is -2.541.

Time(HH:MM:SS) : 00:03:21

And TNS is around 503.

Time(HH:MM:SS) : 00:03:24

So if we do the analysis and let's choose the path group analysis.

Time(HH:MM:SS) : 00:03:31

And in this case we don't need any other options.

Time(HH:MM:SS) : 00:03:36

And basically it will create registered register and put register registered output depending on the paths that are in your design.

Time(HH:MM:SS) : 00:03:46

So in this case it only created the register edge.

Time(HH:MM:SS) : 00:03:53

And it's looking at the failing paths.

Time(HH:MM:SS) : 00:03:55

So there are 702 failing paths.

Time(HH:MM:SS) : 00:03:59

And obviously that covers all of the failing paths in your design.

Time(HH:MM:SS) : 00:04:05

And so the edge to edge analysis is not helpful to us in this case, except to eliminate that there are no issues with input to register and registered output.

Time(HH:MM:SS) : 00:04:19

So let's look at the clock analysis.

Time(HH:MM:SS) : 00:04:24

And clock analysis has different choices here.

Time(HH:MM:SS) : 00:04:28

You can choose to use different edges or views or modes.

Time(HH:MM:SS) : 00:04:33

In our case we only have one view and there are no other modes except for that one view.

Time(HH:MM:SS) : 00:04:41

So let's just say okay.

Time(HH:MM:SS) : 00:04:44

And we don't need to bother with the edges in this case.

Time(HH:MM:SS) : 00:04:48

So let's just look at the categories that have been created here.

Time(HH:MM:SS) : 00:04:54

So after examining all the thousand paths, this is what it came up with for the categories.

Time(HH:MM:SS) : 00:05:01

And you have 24 paths failing paths in the pCloud category and about 363 in the my Glock two x.

Time(HH:MM:SS) : 00:05:14

And for my clock you have about 315.

Time(HH:MM:SS) : 00:05:19

So we can start with the most negative.

Time(HH:MM:SS) : 00:05:23

In this case if we double click on peak clock it actually shows the category peak clock in the path list.

Time(HH:MM:SS) : 00:05:30

And the number of paths that are shown would not exceed 24.

Time(HH:MM:SS) : 00:05:36

So all the 24 paths are actually shown here.

Time(HH:MM:SS) : 00:05:39

And the path numbers are actually different because it is only showing based on the value of the slack.

Time(HH:MM:SS) : 00:05:48

So therefore, um, you know, this is the worst.

Time(HH:MM:SS) : 00:05:52

163 63rd, uh, slack path.

Time(HH:MM:SS) : 00:05:58

So that is basically why the path numbers are actually different.

Time(HH:MM:SS) : 00:06:03

Um, but still, it actually is helpful later on when we do further analysis.

Time(HH:MM:SS) : 00:06:09

But for now these are the path numbers that it's showing.

Time(HH:MM:SS) : 00:06:12

And then uh, you can actually do further analysis on that.

Time(HH:MM:SS) : 00:06:18

So because we double clicked on this one, the category summary also shows only the relevant data.

Time(HH:MM:SS) : 00:06:25

So 24 failing paths and the worst negative slack and the TNS.

Time(HH:MM:SS) : 00:06:32

And so basically it's only going to show that.

Time(HH:MM:SS) : 00:06:36

So let's look at the category in the histogram.

Time(HH:MM:SS) : 00:06:42

So I've added the category into the histogram.

Time(HH:MM:SS) : 00:06:46

And everything that is purple is actually shown.

Time(HH:MM:SS) : 00:06:50

And that's basically what the color of the histogram is.

Time(HH:MM:SS) : 00:06:54

So when I right click I can actually choose what color I want.

Time(HH:MM:SS) : 00:07:00

So I can change the category color to let's say yellow.

Time(HH:MM:SS) : 00:07:04

And I can say okay.

Time(HH:MM:SS) : 00:07:06

So that basically is what um, it is able to do.

Time(HH:MM:SS) : 00:07:11

So let's go back to the purple color.

Time(HH:MM:SS) : 00:07:15

And so in our case um, that's the only category listed in the histogram.

Time(HH:MM:SS) : 00:07:22

There aren't many paths obviously.

Time(HH:MM:SS) : 00:07:24

And so therefore we can actually double click on these and open up the timing path analyzer.

Time(HH:MM:SS) : 00:07:34

Then let's look at the Hierarchical flow vPlan one.

Time(HH:MM:SS) : 00:07:38

Uh, in our case we only have one top level hierarchical uh instance, But let's just for kicks do the analysis on this one.

Time(HH:MM:SS) : 00:07:53

And also let's choose black box analysis as well.

Time(HH:MM:SS) : 00:08:01

And so therefore um you know and any macros.

Time(HH:MM:SS) : 00:08:07

So.

Time(HH:MM:SS) : 00:08:12

All right.

Time(HH:MM:SS) : 00:08:15

Let's actually do the analysis.

Time(HH:MM:SS) : 00:08:18

And if we look at the analysis um, there is no black boxes or macros.

Time(HH:MM:SS) : 00:08:25

And there is only one hierarchical instance at the top level.

Time(HH:MM:SS) : 00:08:29

And that is the code instance.

Time(HH:MM:SS) : 00:08:31

And so therefore this is, uh, not very useful for our analysis.

Time(HH:MM:SS) : 00:08:37

So let's look at the Hierarchical port one.

Time(HH:MM:SS) : 00:08:41

You have the choice of creating based on hierarchical port.

Time(HH:MM:SS) : 00:08:48

So if you have any specific ports that you want to look at, and based on that, you can generate the analysis..

Time(HH:MM:SS) : 00:08:57

Uh, now, uh, we also can do the view analysis.

Time(HH:MM:SS) : 00:09:02

And as I mentioned before, there is only one view.

Time(HH:MM:SS) : 00:09:06

And so let's just do the analysis.

Time(HH:MM:SS) : 00:09:08

And basically it's showing the functional Slew max view.

Time(HH:MM:SS) : 00:09:13

And that's the only view that you actually get.

Time(HH:MM:SS) : 00:09:16

So you can uh, merge the same path in different views.

Time(HH:MM:SS) : 00:09:23

And that would actually show when you right click the same path with the same endpoint and across different views, when you have multiple views using this method, you can actually compare the same endpoint across multiple views.

Time(HH:MM:SS) : 00:09:40

Now go back to the other view in order to run the critical faults path analysis, you need to have Conformal Constraint designer license and you have two options.

Time(HH:MM:SS) : 00:09:53

One is to create a category of only the faults paths that are part of your, uh, timing violation report.

Time(HH:MM:SS) : 00:10:03

And or you can actually create, uh, category, which excludes the faults paths.

Time(HH:MM:SS) : 00:10:12

So let's actually run the false path categories and see if there is any category.

Time(HH:MM:SS) : 00:10:19

If we look at the, uh, report right here, it says it's loading the Libraries in Conformal Constraint designer and ran the critical false path analysis.

Time(HH:MM:SS) : 00:10:29

But there are no paths.

Time(HH:MM:SS) : 00:10:31

So because there are no paths, it's actually not creating a group.

Time(HH:MM:SS) : 00:10:36

Um, okay.

Time(HH:MM:SS) : 00:10:37

So let's look at the um category that excludes the false paths.

Time(HH:MM:SS) : 00:10:44

So if we run this one, it should include all the paths, the 702 that we have in our design.

Time(HH:MM:SS) : 00:10:53

And basically the false paths, um, are zero.

Time(HH:MM:SS) : 00:10:58

So we have all the 702 paths included in the exclude false path list.

Time(HH:MM:SS) : 00:11:08

So we have that analysis..

Time(HH:MM:SS) : 00:11:12

And let's look at um, the other ones.

Time(HH:MM:SS) : 00:11:15

So let's create a bottleneck analysis.

Time(HH:MM:SS) : 00:11:18

Bottleneck analysis basically looks for um paths that are considered, um, you know, critical paths, but at the same time that go through a certain instance.

Time(HH:MM:SS) : 00:11:32

And so therefore that instance becomes a bottleneck.

Time(HH:MM:SS) : 00:11:37

So let's actually look for those kind of instances and create categories based on those instances.

Time(HH:MM:SS) : 00:11:46

So I'm specifying only the negative slack paths.

Time(HH:MM:SS) : 00:11:49

And so create two categories.

Time(HH:MM:SS) : 00:11:53

So here I have basically 78 paths that go through G3 instance.

Time(HH:MM:SS) : 00:12:02

So as you can see in the name of the category itself, G3 is the name of the instance that is considered the bottleneck here.

Time(HH:MM:SS) : 00:12:11

And then in this case, um, the x reg says one data.

Time(HH:MM:SS) : 00:12:17

One uh, DF is the offending party.

Time(HH:MM:SS) : 00:12:23

So let's look at each of these.

Time(HH:MM:SS) : 00:12:26

Um, so let's start with this one and the start point here in this case.

Time(HH:MM:SS) : 00:12:35

So data one uh, Was the offending party, as you know, or the bottleneck.

Time(HH:MM:SS) : 00:12:41

So therefore in this case it is the start point.

Time(HH:MM:SS) : 00:12:44

So we have on that particular path, um, and all of the other paths, the 120 of them basically have this particular flop in its path.

Time(HH:MM:SS) : 00:12:59

And then let's look at the other one.

Time(HH:MM:SS) : 00:13:04

In the case of the other one, the, um, somewhere along the path, you should actually have the G3 instance.

Time(HH:MM:SS) : 00:13:15

And basically let's look for that G3 instance.

Time(HH:MM:SS) : 00:13:19

And there it is.

Time(HH:MM:SS) : 00:13:21

G3 is basically one of those, um, instances in that particular path.

Time(HH:MM:SS) : 00:13:28

And as you can see, the delay through that instance is 0.46 and obviously a bunch of paths, 78 of them.

Time(HH:MM:SS) : 00:13:37

The same particular instance.

Time(HH:MM:SS) : 00:13:41

So let's continue our analysis to drive analysis in the case of drive analysis.

Time(HH:MM:SS) : 00:13:50

The tool needs to generate the violation reports for capacitance transition as well as the fan out.

Time(HH:MM:SS) : 00:13:56

And it is going to give you the categories based on that.

Time(HH:MM:SS) : 00:14:04

So once the violations are generated so it creates its own category and the drive train.

Time(HH:MM:SS) : 00:14:13

We can also see the violations actually present themselves in the timing interpretation tab as well.

Time(HH:MM:SS) : 00:14:22

So um, finally um, we also have uh a noise analysis.

Time(HH:MM:SS) : 00:14:31

And these three viewers, we haven't run the noise analysis for the GUI.

Time(HH:MM:SS) : 00:14:39

to view itself, but we should be able to view the clock matrix Viewer..

Time(HH:MM:SS) : 00:14:45

We don't have any hierarchy, so therefore hierarchical Analysis.

Time(HH:MM:SS) : 00:14:49

Viewer.

Time(HH:MM:SS) : 00:14:49

also might not be visible, but when you open the clock matrix viewer, you can actually look at the different ones between the clocks and so on.

Time(HH:MM:SS) : 00:15:00

Okay.

Time(HH:MM:SS) : 00:15:01

You can also see what is the skew between the clocks.

Time(HH:MM:SS) : 00:15:05

Okay.

Time(HH:MM:SS) : 00:15:05

And if there are any paths that are going from clock one to clock two, there should be data in here as well.

Time(HH:MM:SS) : 00:15:12

But in this case there is nothing like that.

Time(HH:MM:SS) : 00:15:15

So the clock matrix Viewer.

Time(HH:MM:SS) : 00:15:17

is between the same clock.

Time(HH:MM:SS) : 00:15:20

That is actually a helpful neat analysis tool that you can actually look at.

Time(HH:MM:SS) : 00:15:25

That's the end of the video.

Time(HH:MM:SS) : 00:15:26

Please look for more videos in the Tempest series.

VideoTitle :Construct Tempus Start

Time(HH:MM:SS) : 00:00:01

Often in the timing, analysis and debug, it's useful to write a text file containing the category names, the total number of paths, number of paths failing or passing along with their worst negative slack, and the total negative slack.

Time(HH:MM:SS) : 00:00:15

This can be done using the right category summary command followed with the report name.

Time(HH:MM:SS) : 00:00:21

Here it shows a sample report result of this command showing different view names with the paths failing and passing ones.

VideoTitle :Flowchart Analysis FLow

Time(HH:MM:SS) : 00:00:01

transcript is not present for this video.

VideoTitle :GBA versus PBA

Time(HH:MM:SS) : 00:00:00

Timing, analysis and debug is all about identifying the issues and implementing the fixes for the design closure.

Time(HH:MM:SS) : 00:00:06

Let's look through the manual ECO is, which can be done in Tempus timing.

Time(HH:MM:SS) : 00:00:09

In this submodule, let's explore the manual ECO is related commands and set up and run manual ECO's and Tempus timing.

VideoTitle :What are On Chip Variations

Time(HH:MM:SS) : 00:00:00

During manual ECO is certain settings need to be done.

Time(HH:MM:SS) : 00:00:04

These are the different timing ECO.

Time(HH:MM:SS) : 00:00:06

commands supported in Tempest stylus.

Time(HH:MM:SS) : 00:00:08

We can instruct the tool to honor, don't touch or don't use or the fixed status in the echoes.

Time(HH:MM:SS) : 00:00:14

We can also go for checking logical equivalence and setting it to true or false as per the design specifications.

Time(HH:MM:SS) : 00:00:21

We can do echoes by adding repeaters, deleting them, or updating the cell as an upsize or downsizing them.

Time(HH:MM:SS) : 00:00:28

Then it also supports the net characteristics like connect net, disconnect net and creating and deleting instances.

Time(HH:MM:SS) : 00:00:34

We can also read an echo file using read echo, or can write out an echo file in Tempest or Innovus platform using write echo, followed by the format specified.

VideoTitle :Concept AOCV SOCV

Time(HH:MM:SS) : 00:00:00

Repeater edition is one of the most common fixes in interactive ECO is.

Time(HH:MM:SS) : 00:00:04

This can be done in Tempest using the add repeater page of the interactive ECO form, which is used to add either a single buffer or two inverters to a net.

Time(HH:MM:SS) : 00:00:13

This can be reached through ECO menu.

Time(HH:MM:SS) : 00:00:16

Within that, go for interactive ECO, and within that dropdown select Add Repeater tab.

Time(HH:MM:SS) : 00:00:21

The add repeater fields and options are.

Time(HH:MM:SS) : 00:00:23

Net you need to type the Net name or click on the Net in the layout tab and click Get selected.

Time(HH:MM:SS) : 00:00:29

In the new cell, enter the cell type name of the buffer cell to be added, or click on the arrow to the right of the field and select a buffer from the drop down list.

Time(HH:MM:SS) : 00:00:38

Add to What If list allows you to add your specifications with ECO commands.

Time(HH:MM:SS) : 00:00:42

in series.

Time(HH:MM:SS) : 00:00:43

The avail option evaluates the effect on timing if you add a new cell.

Time(HH:MM:SS) : 00:00:48

How to add an instance in the ECO.

Time(HH:MM:SS) : 00:00:50

The add instance page of the interactive ECO form is used to add instances to a.

Time(HH:MM:SS) : 00:00:55

Net, and this can be done using add instance tab selection within the interactive ECO.

Time(HH:MM:SS) : 00:01:00

VCO form.

Time(HH:MM:SS) : 00:01:01

The different fields and options are.

Time(HH:MM:SS) : 00:01:03

Cell list, which specifies a name of a collection of cells used as a reference for the new instances added to search for the cell.

Time(HH:MM:SS) : 00:01:10

Enter the cell name in the textbooks and click the tick button.

Time(HH:MM:SS) : 00:01:14

Cell type it specifies the type of cell to be used as a reference for the new instances added.

Time(HH:MM:SS) : 00:01:20

Cell name specifies the master of the instance.

Time(HH:MM:SS) : 00:01:24

The instance name specifies the name of the instance to be added and placed.

Time(HH:MM:SS) : 00:01:28

Next comes modifying a cell.

Time(HH:MM:SS) : 00:01:31

The change cell page of the interactive ECO form is used to upsize or downsize an instance.

Time(HH:MM:SS) : 00:01:37

The different fields and options are.

Time(HH:MM:SS) : 00:01:38

In instance, enter the hierarchical instance name to be changed.

Time(HH:MM:SS) : 00:01:42

Get selected.

Time(HH:MM:SS) : 00:01:43

It displays the name of the selected instance in options.

Time(HH:MM:SS) : 00:01:47

Select from the various options either to upsize or downsize a specified cell.

Time(HH:MM:SS) : 00:01:53

Add to What If list allows you to add your specifications with ECO commands.

Time(HH:MM:SS) : 00:01:57

in series in a val, it evaluates the effect on timing.

Time(HH:MM:SS) : 00:02:02

If you upsize or downsize the cell or change to a different cell, and these values are reported but not applied.

Time(HH:MM:SS) : 00:02:08

Next is to delete the repeaters.

Time(HH:MM:SS) : 00:02:11

The Del repeater page.

Time(HH:MM:SS) : 00:02:12

of the interactive ECO form is used to delete redundant buffers, which are added based on wireload models but cause an extra delay.

Time(HH:MM:SS) : 00:02:20

The fields and options are.

Time(HH:MM:SS) : 00:02:21

Instance.

Time(HH:MM:SS) : 00:02:22

Enter the hierarchical instance name to be removed.

Time(HH:MM:SS) : 00:02:26

In get selected, it displays the name of the selected instance.

Time(HH:MM:SS) : 00:02:30

If you click on the Get Selected button on this form without for selecting an instance in the layout tab, the software issues an error message.

Time(HH:MM:SS) : 00:02:37

The add to What If list allows you to add your specifications with ECO commands.

Time(HH:MM:SS) : 00:02:42

in series, and the avail switch evaluates the effect on timing.

Time(HH:MM:SS) : 00:02:45

If you do delete cell after identifying the fixes, the save ECO history page of the interactive ECO form is used to save the history in an output file.

Time(HH:MM:SS) : 00:02:55

You can save this file in Tempus or other database format.

Time(HH:MM:SS) : 00:02:59

The fields and options are.

Time(HH:MM:SS) : 00:03:00

The ECO history file specifies the name of the file that this menu command produces.

Time(HH:MM:SS) : 00:03:06

The file style generates an ECO.

Time(HH:MM:SS) : 00:03:09

file in the specified format.

Time(HH:MM:SS) : 00:03:11

Append to selected file.

Time(HH:MM:SS) : 00:03:13

Appends the changes to an existing ECO file.

Time(HH:MM:SS) : 00:03:16

The Equivalent.

Time(HH:MM:SS) : 00:03:17

tickle command is right.

Time(HH:MM:SS) : 00:03:18

Eco.

Time(HH:MM:SS) : 00:03:22

Hi.

Time(HH:MM:SS) : 00:03:23

In this video we will look at how to perform interactive ECO is in Tempus with stylus coy mode.

Time(HH:MM:SS) : 00:03:31

An interactive ECO can be done to fix the timing Violations..

Time(HH:MM:SS) : 00:03:35

weather, be it set up or hold.

Time(HH:MM:SS) : 00:03:38

Here we are loading the design in tempo stylus mode using this Xrun article.

Time(HH:MM:SS) : 00:03:44

We first read the libraries followed by the noise libraries.

Time(HH:MM:SS) : 00:03:49

Then we read the netlist by setting the top module, as DTMF receiver code.

Time(HH:MM:SS) : 00:03:55

Then in stylus mode, we need to do init design to initialize the design, then throw this source command.

Time(HH:MM:SS) : 00:04:02

We are loading the MMC information for the design.

Time(HH:MM:SS) : 00:04:06

We are reading the constraints following by setting the power rails for ground and power.

Time(HH:MM:SS) : 00:04:12

Then we are reading the specs to load the RC extraction information.

Time(HH:MM:SS) : 00:04:16

For the design.

Time(HH:MM:SS) : 00:04:18

We are setting the timing analysis type as Ocv.

Time(HH:MM:SS) : 00:04:21

That is the on chip variation followed by the update timing.

Time(HH:MM:SS) : 00:04:26

Let's open the GUI..

Time(HH:MM:SS) : 00:04:32

Let's go to the analysis tab and generate the timing reports.

Time(HH:MM:SS) : 00:04:39

Here we find that there are 83 failing paths in our design, with the worst negative slack given by 0.95.

Time(HH:MM:SS) : 00:04:49

To begin the analysis, let's go to analysis tab and select the bottleneck analysis.

Time(HH:MM:SS) : 00:04:56

Let's give the categories as three Followed by the max lag as zero.

Time(HH:MM:SS) : 00:05:04

Here we find that total three bottleneck instances have been generated.

Time(HH:MM:SS) : 00:05:09

G5 zero five instance is leading to 75 violating paths.

Time(HH:MM:SS) : 00:05:13

So let's have a look at this.

Time(HH:MM:SS) : 00:05:16

By double clicking on this instance we get all the violations particular to that instance.

Time(HH:MM:SS) : 00:05:22

Let's double click on this to open the timing window analyzer.

Time(HH:MM:SS) : 00:05:27

So here it gives the start and end point followed by the slack.

Time(HH:MM:SS) : 00:05:31

This is the bottleneck instance.

Time(HH:MM:SS) : 00:05:33

Having a Slew of 1.463 and a delay of 1.094.

Time(HH:MM:SS) : 00:05:40

Let's check for this library cell.

Time(HH:MM:SS) : 00:05:44

So it gives the cell and the library from which it has been categorized.

Time(HH:MM:SS) : 00:05:49

We see that the cell type is inverter.

Time(HH:MM:SS) : 00:05:53

Then doing the import instance library it gives some more information about the cell.

Time(HH:MM:SS) : 00:06:01

So here we see that it leads to 75 violations...

Time(HH:MM:SS) : 00:06:06

And if we fix this particular instance we might help to reduce these many violations..

Time(HH:MM:SS) : 00:06:11

So let's do an interactive ECO on this cell.

Time(HH:MM:SS) : 00:06:15

Right click on this and go to interactive ECO.

Time(HH:MM:SS) : 00:06:18

Let's go with the chain cell here.

Time(HH:MM:SS) : 00:06:22

It gives the instance and the cell type.

Time(HH:MM:SS) : 00:06:26

Let's upsize the cell to reduce the delay.

Time(HH:MM:SS) : 00:06:30

Let's go for inverter x for MTR.

Time(HH:MM:SS) : 00:06:34

Before applying this change let's evaluate the slack with the changes.

Time(HH:MM:SS) : 00:06:40

Here it shows that the slack will be reduced to 0.144.

Time(HH:MM:SS) : 00:06:45

If we have done this change, let's try for some more upsize options.

Time(HH:MM:SS) : 00:06:55

So this inverter six leads to a positive slack of 0.08.

Time(HH:MM:SS) : 00:07:01

Let's see further.

Time(HH:MM:SS) : 00:07:06

So increasing the cell even to inverter a option, it gives a much more positive slack.

Time(HH:MM:SS) : 00:07:13

So let's commit the change with inverters eight and click apply.

Time(HH:MM:SS) : 00:07:19

So on the prompt we see that this particular cell has been resized from inverter to X to inverter eight family.

Time(HH:MM:SS) : 00:07:27

Let's close this window.

Time(HH:MM:SS) : 00:07:31

With this change, let's regenerate the reports and see the effect of this ECO on the design.

Time(HH:MM:SS) : 00:07:41

What a great change.

Time(HH:MM:SS) : 00:07:42

We find that out of 83 Violations..

Time(HH:MM:SS) : 00:07:45

with just one change, it has led us to only eight failing paths.

Time(HH:MM:SS) : 00:07:50

So this is the effect of the ECO we have done.

Time(HH:MM:SS) : 00:07:54

Thank you.

VideoTitle :Course Quiz Timing Analysis

Time(HH:MM:SS) : 00:00:00

Here are some additional reference materials for Tempus.

VideoTitle :Sub-Module Objective

Time(HH:MM:SS) : 00:00:00

In this lab.

Time(HH:MM:SS) : 00:00:00

Test your understanding on the global timing debug interface to debug the timing results.

VideoTitle :Construct Generating Reports

Time(HH:MM:SS) : 00:00:01

Course Conclusions.

Time(HH:MM:SS) : 00:00:06

In this course, you.

Time(HH:MM:SS) : 00:00:08

Implemented the RTL of a design from its specification.

Time(HH:MM:SS) : 00:00:12

Simulated a design using the Xcelium™ Simulator tool.

Time(HH:MM:SS) : 00:00:16

Verified Code Coverage using the Integrated Metrics Center.

Time(HH:MM:SS) : 00:00:20

Synthesized the design from RTL to Gates using Genus™ Synthesis Solution.

Time(HH:MM:SS) : 00:00:25

Inserted test structures to be able to test the design using the Genus Synthesis Solution and verify the test coverage using the Encounter® test.

Time(HH:MM:SS) : 00:00:34

Compared the design against the RTL using Conformal® Equivalence Checker.

Time(HH:MM:SS) : 00:00:39

Ran the digital implementation flow with the Innovus™ Implementation System.

Time(HH:MM:SS) : 00:00:44

Created a floorplan.

Time(HH:MM:SS) : 00:00:46

Implemented power structures and clock trees.

Time(HH:MM:SS) : 00:00:49

Performed Place and Route on the design.

Time(HH:MM:SS) : 00:00:52

Verified the design.

Time(HH:MM:SS) : 00:00:54

Ran signoff checks to make sure that the design chip can be fabricated.

Time(HH:MM:SS) : 00:01:01

Other Courses References.